

Graphisme dans le jeu vidéo

Table des matières

Session 1 : Géométrie et Découverte de l'API	4
Présentation du cours	4
Découverte de l'environnement : Babylon.js et le Playground	5
Rappels mathématiques : Vecteurs et Matrices	5
La géométrie bas niveau	6
Atelier pratique : Création d'un mesh à la main	7
Démonstration : Construire un maillage de A à Z	9
Session 2 : Transformations et Espaces de Coordonnées	12
Le voyage d'un Vertex : Le Pipeline de Transformation	12
Comprendre les Matrices : L'approche « Calculatrice »	15
La Matrice Identité : La machine au repos	15
La Matrice de Scale : Le multiplicateur	15
Le laboratoire de test : L'ordre des opérations	16
Anatomie des matrices de transformation	16
La Matrice de Translation et l'astuce de la coordonnée W	17
Atelier Desmos : Visualiser les matrices en 2D	18
Le lien avec la géométrie : La boucle est bouclée	19
Hiérarchie de scène : Le Parenting	20
Atelier : Le Train et le Passager	20
Concepts clés à retenir (Les pièges classiques)	21
Session 3 : Création de Shaders dans Babylon.js (Partie 1)	23
De la copie manuelle à la presse d'imprimerie	23
Qu'est-ce qu'un Fragment Shader ?	23
Pourquoi les shaders sont-ils rapides ?	23
Pourquoi les shaders font peur ?	24
Le Pipeline de Rendu Programmable	24
GLSL : Le langage des Shaders	24
Shaders dans Babylon.js : ShaderMaterial	25
Uniforms et Varyings	26
Session 4 : Création de Shaders dans Babylon.js (Partie 2)	27
La fonction step	27
La terre vue par NASA	27
Session 5 : Lumière et matériaux	28
Partie 1 : L'Éclairage Mathématique — La Loi de Lambert	28
Partie 2 : Sources de Lumières et Spéculaire	33
Concepts clés à retenir	40
Session 6 : L'Éclairage Statique	41
Partie 1 : Le Problème — L'Éclairage Temps Réel est Trop Cher	41
Partie 2 : Les Lightmaps — L'Éclairage Cuit au Four	41
Partie 3 : L'Ambient Occlusion (AO)	44
Partie 4 : Les Light Probes — L'Éclairage pour les Objets Dynamiques	46
Partie 5 : L'Éclairage Mixte (Mixed Lighting)	48

Partie 6 : Reflection Probes — Les Réflexions Baked	48
Partie 7 : Le Pipeline Complet d'Éclairage Statique	49
Partie 8 : Le Texture Baking pour Personnages	49
Concepts clés à retenir	51
Travail Pratique : Baked Lighting en Three.js	52
Session 7 : Textures — Les Fondamentaux	55
Partie 1 : Qu'est-ce qu'une Texture ?	55
Partie 2 : UV Mapping — Le Dépliage dans Blender	56
Partie 3 : Le Filtrage	57
Partie 4 : Le Wrapping	58
Partie 5 : Les Formats de Texture	58
Partie 6 : Le Set de Textures PBR	59
Partie 7 : Les Normal Maps en Détail	61
Partie 8 : Bump, Normal et Displacement — Les Trois Approches du Relief	62
Partie 9 : Export depuis Blender	65
Partie 10 : TP — Set PBR Complet dans Babylon.js	65
Session 8 : Textures Avancées	67
Partie 1 : Textures Procédurales	67
Partie 2 : Render Targets	78
Partie 3 : Cube Maps et Environment Maps	81
Partie 4 : Texture Atlases et Arrays	82
Partie 5 : UV Avancés — Triplanar Mapping	85
Partie 6 : Parallax Mapping	87
Partie 7 : Detail Textures et Blending	89
Partie 8 : Pratique	90
Session 9 : Baking de Textures avec Blender	91
Partie 1 : Le point de départ — un modèle issu de l'IA	91
Partie 2 : Le Baking — Principe Général	91
Partie 3 : Baking de Diffuse — Transférer vers de nouveaux UVs	92
Partie 4 : Refaire un UV Propre	93
Partie 5 : Baking de Normal Map — Détail High-Poly → Low-Poly	95
Partie 6 : Démonstration en Direct	97
Partie 7 : Pratique — Éclairage Statique en Moteur	98
Session 10 : Ombres en Temps Réel — le Shadow Mapping	101
Partie 1 : Pourquoi les Ombres ?	101
Partie 2 : Le Shadow Mapping — Principe	102
Partie 3 : Le Shader d'Ombre	104
Partie 4 : Les Artefacts — Acne et Peter Panning	105
Partie 5 : Adoucir les Ombres — PCF	107
Partie 6 : PCSS — Ombres Douces Physiquement Plausibles	109
Partie 7 : Cascaded Shadow Maps (CSM)	111
Partie 8 : Ombres pour Lumières Ponctuelles — Cube Shadow Maps	113
Partie 9 : Implémentation en Three.js	114
Partie 10 : Ombres dans Godot et Unity — Aperçu	116
Partie 11 : Performance et Qualité	117
Partie 12 : Variante — Variance Shadow Maps (VSM)	118

Partie 13 : Démonstration en Direct — Three.js	119
Partie 14 : Pratique — Ajouter des Ombres à une Scène Three.js	120
Conclusion	122
Session 11 : Post-Processing et Render Targets	124
Partie 1 : Le Pipeline de Post-Processing	124
Partie 2 : Les Effets Principaux	126
Partie 3 : Implémentation en Three.js	130
Partie 4 : Performance et Bonnes Pratiques	131
Partie 5 : Démonstration en Direct — Three.js	132
Partie 6 : Pratique — Ajouter du Post-Processing	132
Conclusion	132
Session 12 : Optimisation — Structures Spatiales, LOD et Billboards	134
Partie 1 : Le Problème des Grandes Scènes	134
Partie 2 : Culling — Ne Pas Dessiner l’Invisible	135
Partie 3 : LOD — Level of Detail	137
Partie 4 : Billboards — la 3D dans une Texture	139
Partie 5 : Impostors — le LOD Extrême	141
Partie 6 : Implémentation en Three.js	142
Partie 7 : Démonstration en Direct	143
Partie 8 : Pratique — Scène Optimisée	143
Conclusion	144

Session 1 : Géométrie et Découverte de l'API

🔗 Objectifs de la session

Présenter le cours et son fonctionnement, découvrir l'écosystème Babylon.js, rappeler les notions mathématiques essentielles, et construire un maillage 3D à la main pour comprendre la structure intime d'un modèle.

Présentation du cours

Orientations et évaluation

Ce cours explore la programmation graphique 3D temps réel appliquée au jeu vidéo, en utilisant entre autres Babylon.js comme moteur, mais aussi d'autres outils et frameworks (Unity, Unreal Engine, Blender, Godot).

Évaluation :

- Travaux pratiques (TP) : Mise en pratique des concepts de chaque session.
- Projet final : Création d'un projet utilisant l'ensemble des principes abordés.

Approche pédagogique : Chaque session combine théorie et atelier pratique. Le code est écrit en JavaScript/TypeScript et s'exécute directement dans le navigateur. Des démonstrations seront aussi faites sur les engins commerciaux, pour montrer les analogies, la programmation graphique étant indépendante du moteur utilisé.

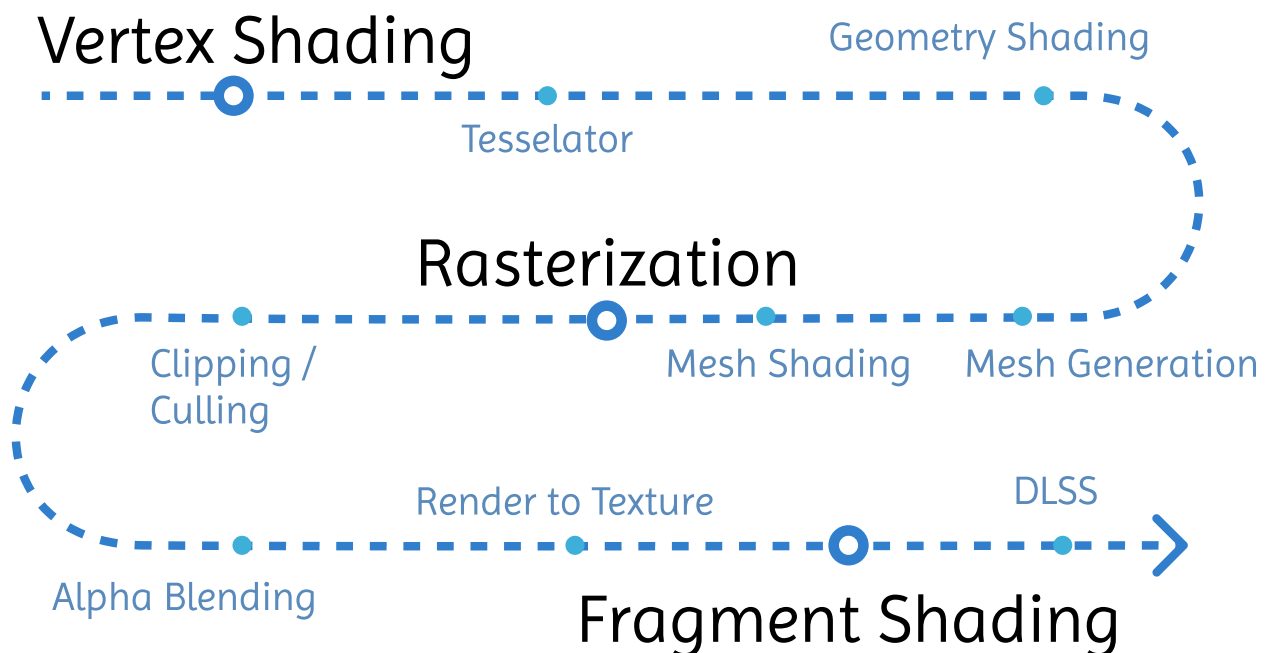


Fig. 1. – Notre parcours dans l'univers du GPU.

Introduction à la programmation graphique

La programmation graphique consiste à décrire au GPU (carte graphique) comment transformer des données mathématiques (points, vecteurs, matrices) en pixels colorés à l'écran. C'est la rencontre entre les mathématiques, la physique (optique) et l'informatique.

Contrairement à la programmation classique (CPU, séquentielle), la programmation graphique exploite le parallélisme massif du GPU : des milliers de calculs exécutés simultanément.

Découverte de l'environnement : Babylon.js et le Playground

L'écosystème Babylon.js

Babylon.js est un moteur 3D open-source écrit en TypeScript, s'exécutant dans le navigateur via WebGL et WebGPU. Il offre :

- Un moteur de rendu temps réel (PBR, lumières, ombres, post-process).
- Un éditeur visuel de shaders (Node Material Editor).
- Une API complète pour la création de scènes, d'animations et d'interactions.

Avantage clé : Aucune installation requise. Tout s'exécute dans le navigateur, et il est possible de partager facilement les exemples via URL.

🔗 Le Babylon.js Playground

Le **Playground** (<https://playground.babylonjs.com>) est l'outil idéal pour prototyper :

- Zéro installation : on écrit du code JS directement dans le navigateur.
- Itération rapide : modification en direct, rendu instantané.
- Partage : chaque scène a une URL unique partageable.
- Modèles : des centaines d'exemples officiels pour apprendre.

C'est l'environnement que nous utiliserons pour les ateliers pratiques.

Rappels mathématiques : Vecteurs et Matrices

Vecteurs (Vector3)

Un vecteur représente une direction et une magnitude dans l'espace 3D. Dans Babylon.js, on utilise la classe `BABYLON.Vector3`.

Opérations essentielles :

```
const a = new BABYLON.Vector3(1, 2, 3);
const b = new BABYLON.Vector3(4, 5, 6);

a.add(b);           // Addition
a.subtract(b);      // Soustraction
a.scale(2);         // Multiplication par scalaire
BABYLON.Vector3.Dot(a, b); // Produit scalaire
BABYLON.Vector3.Cross(a, b); // Produit vectoriel
a.normalize();      // Vecteur unitaire
a.length();         // Magnitude (norme)
```

- **Produit scalaire** : Mesure l'alignement entre deux vecteurs. Utilisé pour l'éclairage (angle entre normale et lumière).
- **Produit vectoriel** : Produit un vecteur perpendiculaire aux deux autres. Utilisé pour calculer les normales de surface.

Matrices (Matrix)

Une matrice 4x4 représente une **transformation** dans l'espace (translation, rotation, scale, ou combinaison). Babylon.js utilise `BABYLON.Matrix`.

Transformations courantes :

```
// Translation
const T = BABYLON.Matrix.Translation(1, 0, 0);

// Rotation (autour de Y)
const R = BABYLON.Matrix.RotationY(Math.PI / 4);

// Scale
const S = BABYLON.Matrix.Scaling(2, 2, 2);

// Composition : T * R * S (l'ordre compte !)
const world = T.multiply(R).multiply(S);
```

Pourquoi 4x4 ? La 4ème coordonnée (w) permet de unifier translations et rotations dans une seule matrice. Un point (x, y, z) devient $(x, y, z, 1)$ en coordonnées homogènes.

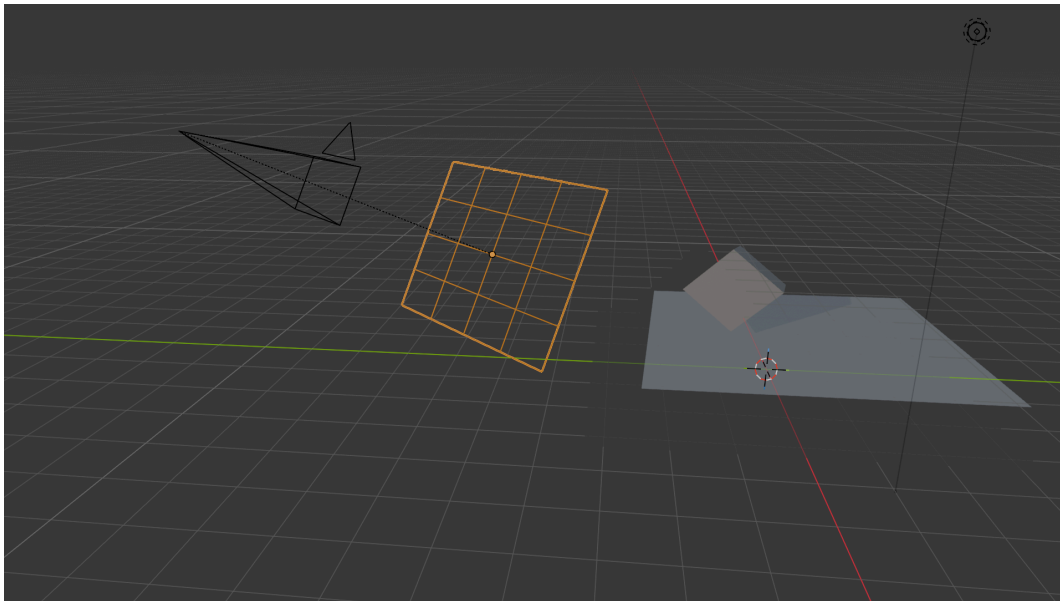


Fig. 2. – L'écran est une projection du monde en 3 dimensions.

La géométrie bas niveau

Composantes d'un Vertex

Un **vertex** (sommet) n'est pas qu'un point dans l'espace. Il peut contenir plusieurs attributs :

- **Position** : Coordonnées (x, y, z) dans l'espace objet. Obligatoire.
- **Normale** : Vecteur perpendiculaire à la surface, utilisé pour l'éclairage. Direction (n_x, n_y, n_z) .
- **UV** : Coordonnées de texture (u, v) dans $[0, 1]$, reliant le sommet à un pixel de la texture 2D.
- **Tangente** : Vecteur tangent à la surface, utilisé pour le normal mapping (espace tangent).
- **Couleur** : Couleur RGBA par sommet (r, g, b, a) , utilisée pour le vertex coloring.

Ces attributs sont stockés dans des **Vertex Buffers** séparés et envoyés au GPU.

Types de primitives

En graphisme temps réel, toute surface est décomposée en triangles. Les types de primitives GPU sont :

- **Points** : Un point par vertex (rarement utilisé).
- **Lignes** : Une ligne par paire de vertices.
- **Line Strip** : Lignes connectées en chaîne.
- **Triangles** : Un triangle par triplet de vertices. C'est le type dominant.
- **Triangle Strip** : Triangles connectés partageant des arêtes (plus économe en vertices).
- **Triangle Fan** : Triangles partageant un vertex central.

Le triangle est la primitive de base car :

- Trois points définissent toujours un plan (pas d'ambiguïté).
- L'interpolation des attributs (UV, normales) est bien définie à l'intérieur.

Atelier pratique : Création d'un mesh à la main

! Objectif de l'atelier

Construire un maillage triangulaire en manipulant directement les **Vertex Buffers** et l'**Index Buffer** via l'objet `VertexData` de Babylon.js. Comprendre la structure intime d'un modèle 3D.

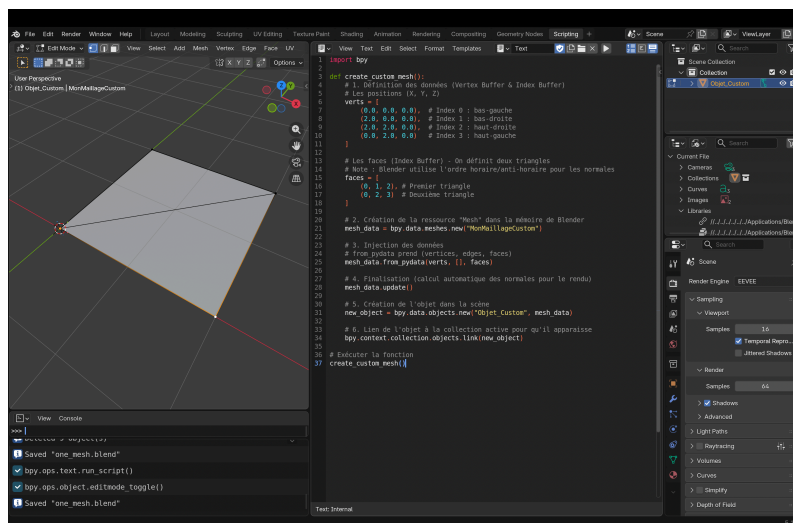


Fig. 3. – Le même maillage peut être créé visuellement dans Blender — les concepts de vertices, normales et UVs sont universels, indépendants du moteur utilisé.

VertexData : L'objet de construction

BABYLON.VertexData est le conteneur bas niveau pour les données géométriques. Il permet de définir manuellement chaque buffer avant de les appliquer à un mesh.

Buffers principaux :

- positions : Tableau de floats ($x_0, y_0, z_0, x_1, y_1, z_1, \dots$)
- normals : Tableau de floats ($nx_0, ny_0, nz_0, \dots$)
- uvs : Tableau de floats ($u_0, v_0, u_1, v_1, \dots$)
- indices : **Index Buffer** — tableau d'entiers désignant quels vertices forment chaque triangle.

Créer un triangle à la main.

```
// 1. Créer un mesh vide
const customMesh = new BABYLON.Mesh("custom", scene);

// 2. Définir les Vertex Buffers
const positions = [
    -1, -1, 0, // Vertex 0 : bas-gauche
    1, -1, 0, // Vertex 1 : bas-droite
    0, 1, 0 // Vertex 2 : haut-centre
];

const normals = [
    0, 0, 1, // Normale du Vertex 0 (vers la caméra)
    0, 0, 1, // Normale du Vertex 1
    0, 0, 1 // Normale du Vertex 2
];

const uvs = [
    0, 0, // UV du Vertex 0
    1, 0, // UV du Vertex 1
    0.5, 1 // UV du Vertex 2
];

// 3. Définir l'Index Buffer (quels vertices forment les triangles)
const indices = [0, 1, 2]; // Un seul triangle

// 4. Assembler le VertexData
const vertexData = new BABYLON.VertexData();
vertexData.positions = positions;
vertexData.normals = normals;
vertexData.uvs = uvs;
vertexData.indices = indices;

// 5. Appliquer au mesh
vertexData.applyToMesh(customMesh);

// 6. Assigner un matériau
const mat = new BABYLON.StandardMaterial("mat", scene);
mat.diffuseColor = new BABYLON.Color3(1, 0, 0);
mat.wireframe = false;
customMesh.material = mat;
```

L'Index Buffer : Pourquoi et Comment ?

L'**Index Buffer** (ou tableau d'indices) indique au GPU dans quel ordre lire les vertices pour former des triangles.

Pourquoi des indices ?

- Un vertex peut être partagé par plusieurs triangles (ex: un cube a 8 sommets mais 12 triangles).
- Sans index buffer, on dupliquerait les vertices partagés (gaspillage mémoire).
- Avec index buffer : 8 vertices + 36 indices au lieu de 36 vertices.

Convention de winding (sens d'enroulement) :

- Babylon.js utilise le **counter-clockwise** (sens inverse horaire) vu de l'extérieur.
- indices = [0, 1, 2] : le triangle est visible depuis +Z.
- indices = [0, 2, 1] : le triangle est visible depuis -Z (backface culling).

Démonstration : Construire un maillage de A à Z

Script complet pour le Babylon.js Playground. Ce code peut être collé directement dans le Playground. Il illustre la création d'un triangle et d'un carré (Quad) en manipulant les buffers.

```
export const createScene = function () {
    // 1. Initialisation de la scène basique
    const scene = new BABYLON.Scene(engine);

    // Caméra orbitale pour tourner autour de nos créations
    const camera = new BABYLON.ArcRotateCamera("camera", -Math.PI / 2, Math.PI / 2.5, 5,
BABYLON.Vector3.Zero(), scene);
    camera.attachControl(canvas, true);

    // Lumière basique
    const light = new BABYLON.HemisphericLight("light", new BABYLON.Vector3(0, 1, 0), scene);
    light.intensity = 0.7;

    // =====
    // EXEMPLE 1 : LE TRIANGLE
    // =====
    const triangleMesh = new BABYLON.Mesh("monTriangle", scene);

    const positionsTri = [
        -1, -1, 0,    // 0: bas-gauche
        1, -1, 0,     // 1: bas-droite
        0, 1, 0       // 2: haut-centre
    ];

    // Dans Babylon (Left-Handed), normales vers -1 en Z pour faire face à la caméra
    const normalsTri = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsTri = [
        0, 0,
        1, 0,
```

```

        0.5, 1
    ];

    const indicesTri = [0, 1, 2];

    const vertexDataTri = new BABYLON.VertexData();
    vertexDataTri.positions = positionsTri;
    vertexDataTri.normals = normalsTri;
    vertexDataTri.uvs = uvsTri;
    vertexDataTri.indices = indicesTri;

    vertexDataTri.applyToMesh(triangleMesh);

    // Matériau rouge pour le triangle
    const matTri = new BABYLON.StandardMaterial("matTri", scene);
    matTri.diffuseColor = new BABYLON.Color3(1, 0, 0);
    // matTri.wireframe = true; // Décommenter pour tester
    triangleMesh.material = matTri;

    triangleMesh.position.x = -1.5;

    // =====
    // EXEMPLE 2 : LE CARRÉ (QUAD) AVEC TEXTURE
    // =====
    const quadMesh = new BABYLON.Mesh("monCarre", scene);

    const positionsQuad = [
        -1, -1, 0, // 0 : bas-gauche
        1, -1, 0, // 1 : bas-droite
        1, 1, 0, // 2 : haut-droite
        -1, 1, 0 // 3 : haut-gauche
    ];

    const normalsQuad = [
        0, 0, -1,
        0, 0, -1,
        0, 0, -1,
        0, 0, -1
    ];

    const uvsQuad = [
        0, 0, // 0
        1, 0, // 1
        1, 1, // 2
        0, 1 // 3
    ];

    const indicesQuad = [0, 1, 2, 0, 2, 3];

    const vertexDataQuad = new BABYLON.VertexData();
    vertexDataQuad.positions = positionsQuad;
    vertexDataQuad.normals = normalsQuad;
    vertexDataQuad.uvs = uvsQuad;
    vertexDataQuad.indices = indicesQuad;

```

```

vertexDataQuad.applyToMesh(quadMesh);

// Matériau avec texture pour prouver que les UV fonctionnent
const matQuad = new BABYLON.StandardMaterial("matQuad", scene);
matQuad.diffuseTexture = new BABYLON.Texture("textures/checkerboard_basecolor.png",
scene);
quadMesh.material = matQuad;

quadMesh.position.x = 1.5;

// =====
// OUTILS PEDAGOGIQUES
// =====
// scene.debugLayer.show(); // Afficher l'inspecteur pour montrer les buffers

return scene;
};
    
```

💡 Points interactifs à observer

1. **Le Backface Culling en direct :** Faites pivoter la caméra dans le Playground pour regarder derrière le triangle et le carré. Les objets deviendront invisibles. Cela illustre parfaitement que l'ordre des indices définit une face orientée d'un seul côté, optimisant ainsi le rendu GPU.
2. **Jouer avec le Wireframe :** Décommentez la ligne `matTri.wireframe = true;`. C'est un excellent moyen visuel de prouver que le carré (à droite) est bel et bien composé de deux triangles (la diagonale apparaît).
3. **Casser l'Index Buffer :** Changez `indicesQuad = [0, 1, 2, 0, 2, 3]` par `[0, 1, 2]`. Un seul triangle du carré s'affiche, prouvant le rôle des indices.
4. **Casser les UVs :** Changez la coordonnée UV du point 2 (haut-droite) de 1, 1 à 0.5, 0.5. La texture du damier se déformera instantanément, ce qui permet de constater le fonctionnement du « texture mapping ».

Session 2 : Transformations et Espaces de Coordonnées

🔗 Objectifs de la session

Comprendre le voyage d'un vertex à travers les espaces de coordonnées, maîtriser les matrices de transformation, et exploiter la hiérarchie de scène (parenting) pour déplacer et animer des objets complexes.

Le voyage d'un Vertex : Le Pipeline de Transformation

Un vertex possède plusieurs positions

Un vertex ne possède pas « une » position, mais une série de positions selon le référentiel dans lequel on l'observe. À chaque étape, une matrice transforme les coordonnées d'un espace à l'autre.



Fig. 4. – Un vertex voyage d'un espace à l'autre jusqu'à ce qu'il devienne un pixel à l'écran.

Espace Local (Object Space)

L'origine est au centre de l'objet $(0, 0, 0)$. C'est l'espace dans lequel l'artiste modélise le mesh. Les coordonnées sont relatives à l'objet lui-même, indépendamment de sa position dans la scène.

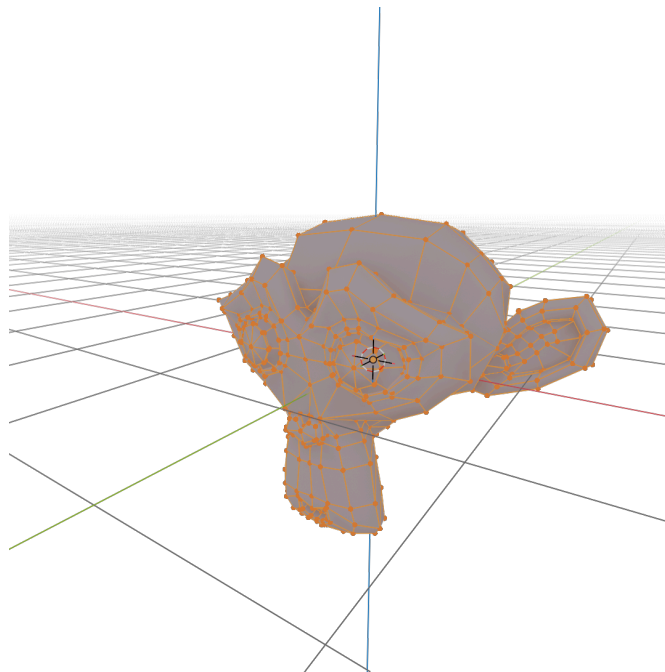


Fig. 5. – Espace Local : l'origine est au centre de l'objet.

Espace Monde (World Space)

La position de l'objet dans la scène globale. Tous les objets y sont exprimés dans un repère commun.

Transformation : Matrice Monde (Model Matrix) — combine la position, rotation et échelle de l'objet pour passer de l'espace local à l'espace monde.

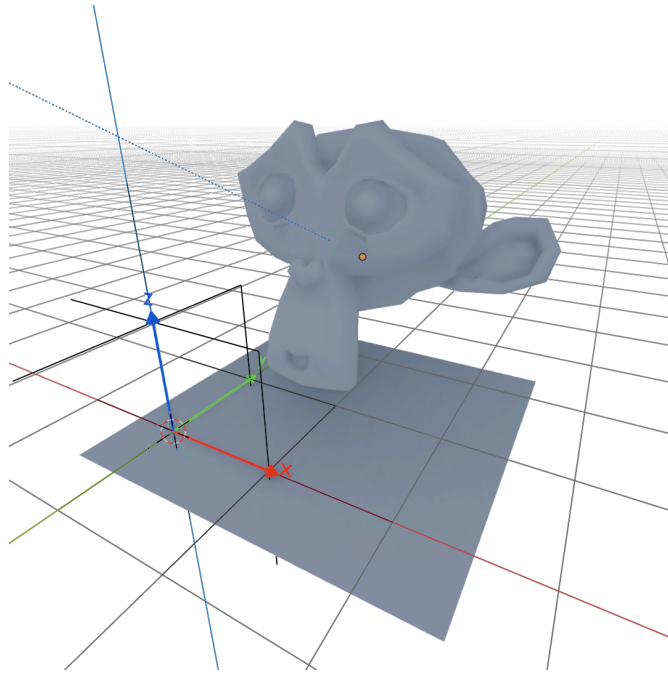


Fig. 6. – Espace Monde : tous les objets partagent un repère commun.

Espace Caméra (View Space)

Le monde vu depuis l'œil du joueur. La caméra devient l'origine $(0, 0, 0)$, et les axes sont alignés sur son orientation.

Transformation : Matrice Vue (View Matrix) — transforme tout le monde pour qu'il soit vu depuis la caméra.

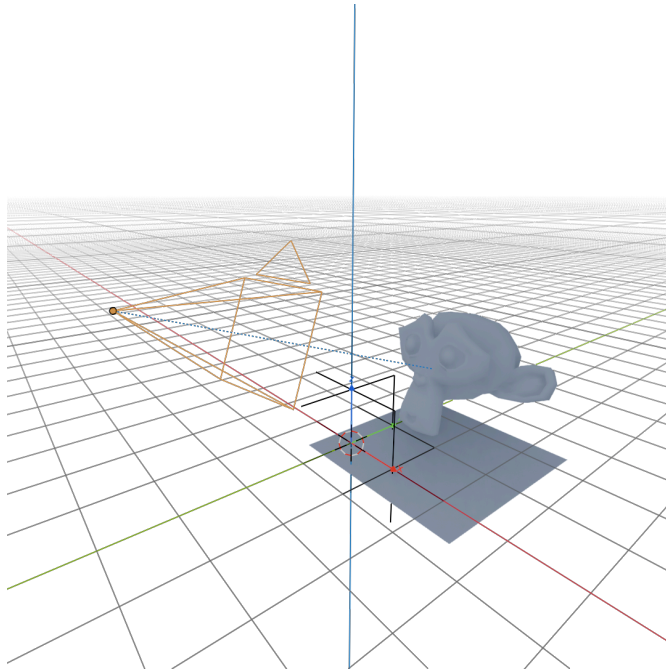


Fig. 7. – Espace Caméra : la caméra devient l'origine, le monde est vu depuis son point de vue.

Espace Projection (Clip Space)

Conversion du volume 3D en un cube plat $[-1, 1]$. C'est ici qu'on définit la perspective : plus un objet est loin, plus il apparaît petit.

Transformation : Matrice de Projection — projette les coordonnées 3D en coordonnées 2D normalisées (NDC).

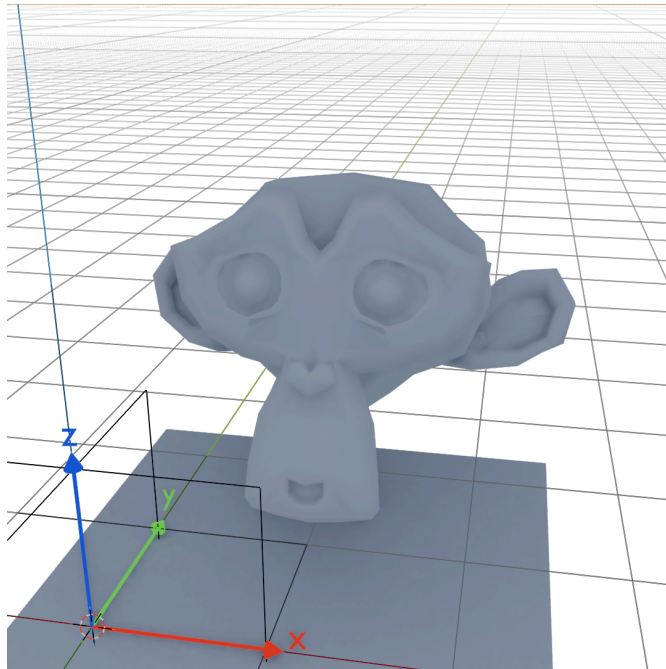


Fig. 8. – Espace Projection : le volume 3D est projeté dans un cube normalisé $[-1, 1]$.

Comprendre les Matrices : L'approche « Calculatrice »

La matrice comme machine

Pour commencer, oubliez les modèles 3D et les caméras. Ouvrez simplement la console de votre **Playground** Babylon.js. Nous allons observer la matrice pour ce qu'elle est réellement : une simple « machine » mathématique qui prend un point en entrée, le transforme, et recrache un nouveau point.

La Matrice Identité : La machine au repos

Définition

La Matrice Identité est une grille 4x4 avec une diagonale de 1 et des 0 partout ailleurs. En algèbre matricielle, c'est l'équivalent du chiffre 1 dans une multiplication classique ($5 \times 1 = 5$).

Micro-exercice 1 : Prouvez que cette matrice ne fait rien. Consigne : Créez un point (un Vector3) à la position (3, 4, 5). Créez ensuite une Matrice Identité. Faites passer votre point dans cette matrice à l'aide de la fonction TransformCoordinates et affichez le résultat dans la console.

```
let p = new BABYLON.Vector3(3, 4, 5);
let mIdentite = BABYLON.Matrix.Identity();

// On transforme le point avec la matrice
let resultat = BABYLON.Vector3.TransformCoordinates(p, mIdentite);
console.log("Résultat Identité :", resultat); // Affiche {X:3, Y:4, Z:5}
```

La Matrice de Scale : Le multiplicateur

Principe

Si vous remplacez les 1 de la diagonale par d'autres chiffres, la machine devient un multiplicateur. Le premier chiffre touche l'axe X, le deuxième le Y, et le troisième le Z.

Micro-exercice 2 : Aplatis un point. Consigne : Prenez le point (2, 2, 2). Fabriquez une matrice de **Scale** qui double sa taille en X et Y, mais l'écrase complètement à 0 sur l'axe Z. Affichez le résultat.

```
let p2 = new BABYLON.Vector3(2, 2, 2);
let mScale = BABYLON.Matrix.Scaling(2, 2, 0); // Z est multiplié par 0 !

let resultatScale = BABYLON.Vector3.TransformCoordinates(p2, mScale);
console.log("Résultat Scale :", resultatScale); // Affiche {X:4, Y:4, Z:0}
```

Micro-exercice 3 : Le téléporteur. Consigne : Vous êtes à l'origine (0, 0, 0). Utilisez une matrice de translation pour vous téléporter aux coordonnées (10, 5, -2).

```
let p3 = new BABYLON.Vector3(0, 0, 0);
let mTrans = BABYLON.Matrix.Translation(10, 5, -2);

let resultatTrans = BABYLON.Vector3.TransformCoordinates(p3, mTrans);
console.log("Résultat Translation :", resultatTrans); // Affiche {X:10, Y:5, Z:-2}
```

Le laboratoire de test : L'ordre des opérations

Combiner les transformations

Maintenant que vous possédez les briques de base, nous allons combiner ces opérations. C'est ici que le pouvoir de la matrice prend tout son sens : on peut fusionner des dizaines de transformations en une seule « Super Matrice ».

Pour fusionner deux matrices, il faut les multiplier. Mais attention : en mathématiques, l'ordre dans lequel vous multipliez vos matrices change complètement le résultat. $A \times B \neq B \times A$.

Micro-exercice 4 : Le piège de l'ordre. **Consigne :** Vous avez un point de départ à $x = 1$. Vous disposez d'une matrice **Scale** $\times 2$ et d'une matrice **Translation** $+3$.

- **Cas A :** Fusionnez les matrices pour appliquer le **Scale** d'abord, puis la **Translation**. Quel est le résultat ?
- **Cas B :** Fusionnez les matrices pour appliquer la **Translation** d'abord, puis le **Scale**. Quel est le résultat ?

```
let point = new BABYLON.Vector3(1, 0, 0);
let S = BABYLON.Matrix.Scaling(2, 1, 1);
let T = BABYLON.Matrix.Translation(3, 0, 0);

// Cas A : T * S (On scale d'abord, puis on translate)
// En math : (1 * 2) + 3 = 5
let matriceA = S.multiply(T);
let resA = BABYLON.Vector3.TransformCoordinates(point, matriceA);
console.log("Cas A :", resA.x); // Affiche 5

// Cas B : S * T (On translate d'abord, puis on scale)
// En math : (1 + 3) * 2 = 8
let matriceB = T.multiply(S);
let resB = BABYLON.Vector3.TransformCoordinates(point, matriceB);
console.log("Cas B :", resB.x); // Affiche 8
```

! Observation clé

Bouger puis grossir ne donne pas la même position finale que grossir puis bouger ! C'est ce qui crée des comportements étranges (les « orbites ») quand vous animez des objets complexes.

Anatomie des matrices de transformation

Matrice de Translation

Déplace un point. La 4ème colonne contient les valeurs de déplacement (t_x, t_y, t_z) .

$$T = \begin{pmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrice de Scale (Échelle)

Multiplie les coordonnées par un facteur. La diagonale contient (s_x, s_y, s_z) .

$$S = \begin{pmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Matrices de Rotation

Rotation autour de chaque axe. Les axes perpendiculaires sont affectés par $\cos \theta$ et $\sin \theta$.

Rotation X

$$R_x = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \theta & -\sin \theta & 0 \\ 0 & \sin \theta & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation Y

$$R_y = \begin{pmatrix} \cos \theta & 0 & \sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ -\sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

Rotation Z

$$R_z = \begin{pmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$$

🔍 Observation

La matrice de **Translation** utilise la 4ème colonne (d'où l'importance de $w = 1$), tandis que les matrices de **Rotation** et **Scale** utilisent la zone 3x3 supérieure gauche. C'est cette séparation qui rend les coordonnées homogènes indispensables.

La Matrice de Translation et l'astuce de la coordonnée W

Pourquoi 4x4 ?

Vous vous demandez sûrement pourquoi nous utilisons des matrices 4x4 pour un espace en 3D (X, Y, Z) ? Le problème est que multiplier la diagonale ne permet pas **d'ajouter** (donc de déplacer) un point. La solution se trouve dans la 4ème colonne de la matrice. Pour que le calcul active cette colonne de déplacement, votre point 3D possède en réalité une 4ème coordonnée cachée : $w = 1$. L'opération mathématique finale fera donc : NouveauX = $(X \times 1) + (\text{TranslationX} \times 1)$.

Versions 2D des matrices de transformation

Les mêmes transformations existent en 2D (X, Y) avec des matrices 3x3. On retrouve la même structure : la translation dans la 3ème colonne, le scale sur la diagonale, et la rotation dans la zone 2x2.

Translation 2D

$$\begin{pmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{pmatrix}$$

Scale 2D

$$\begin{pmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Rotation 2D

$$\begin{pmatrix} \cos \theta & -\sin \theta & 0 \\ \sin \theta & \cos \theta & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Shear X

$$\begin{pmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Shear Y

$$\begin{pmatrix} 1 & 0 & 0 \\ k_y & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

Atelier Desmos : Visualiser les matrices en 2D

Outil

Ouvrez Desmos, calculatrice matricielle : <https://www.desmos.com/matrix>. Desmos permet de définir des matrices et de les multiplier directement. C'est l'outil idéal pour visualiser l'effet d'une matrice sur une forme géométrique avant de passer au 3D.

Étape 1 : Définir un point et la matrice Identité. Consigne : Dans Desmos, créez une matrice $M = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}$ et un point $P = (2, 3)$. Multipliez $M \times P$ pour obtenir un nouveau point P' .

- Entrez : $M = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$
- Entrez : $P = (2, 3)$
- Entrez : $P' = M * P$

Résultat attendu : $P' = (2, 3)$. La matrice Identité ne change rien — le point reste à la même position.

Étape 2 : Scale (Étirer la forme). Consigne : Remplacez la matrice M par une matrice de scale qui double la taille en X et réduit de moitié en Y.

- Entrez : $M = \begin{bmatrix} 2 & 0 \\ 0 & 0.5 \end{bmatrix}$
- Observez P' : le point s'est déplacé de $(2, 3)$ à $(4, 1.5)$.

Expérience : Créez un polygone (un carré) avec plusieurs points et appliquez la matrice à chacun. Vous verrez la forme s'étirer et s'aplatir en temps réel quand vous modifiez les valeurs de M .

Étape 3 : Rotation (Faire tourner la forme). Consigne : Utilisez la matrice de rotation 2D avec un angle θ que vous pouvez animer.

- Entrez : $M = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$
- Ajoutez un curseur pour θ (cliquez sur θ puis activez le curseur).
- Créez un carré avec 4 points et appliquez M à chacun.

Résultat : En faisant varier θ de 0 à 2π , la forme tourne autour de l'origine. C'est exactement ce que fait le GPU avec vos modèles 3D, mais avec une matrice 4x4 et un axe supplémentaire.

Étape 4 : Combiner Translation + Rotation (Le piège !). Consigne : En 2D, la translation nécessite une matrice 3x3 avec coordonnées homogènes. Définissez une matrice de translation et une matrice de rotation, puis testez les deux ordres de multiplication.

- **Translation :** $T = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & 2 \\ 0 & 0 & 1 \end{bmatrix}$
- **Rotation :** $R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$
- **Cas A (Rotation puis Translation) :** $M_A = T * R$
- **Cas B (Translation puis Rotation) :** $M_B = R * T$

Observation : Animez θ et comparez les deux trajectoires. Dans le Cas A, la forme tourne sur elle-même puis est déplacée. Dans le Cas B, la forme orbite autour de l'origine. C'est la démonstration visuelle que $A \times B \neq B \times A$.

! Ce que vous venez de comprendre

Desmos vous a permis de voir, en 2D, exactement ce que le GPU fait en 3D avec des matrices 4x4. Chaque point (vertex) de votre modèle est multiplié par une matrice pour déterminer sa position finale à l'écran. L'ordre des multiplications détermine le comportement de l'animation.

Le lien avec la géométrie : La boucle est bouclée

La matrice applique la même règle à tout l'univers

Rappelez-vous : un modèle 3D (Mesh) n'est qu'un immense tableau géant rempli de Vector3. La matrice est la formule magique que l'on va appliquer dans une boucle for sur **absolument tous** ces points.

Micro-exercice 5 : Incarnez la carte graphique (GPU) !. **Consigne :** Voici le code du carré plat de la Session 1. Avant d'envoyer ces positions pour créer la face 3D, vous allez faire le travail du Vertex Shader. Créez une matrice de Scale(2, 0.5, 1). Faites passer chaque point de votre tableau (grâce à une boucle) dans cette matrice pour déformer la géométrie à la main.

```
// Les coordonnées de base de notre carré
let positions = [
    new BABYLON.Vector3(-1, -1, 0),
    new BABYLON.Vector3(1, -1, 0),
    new BABYLON.Vector3(1, 1, 0),
    new BABYLON.Vector3(-1, 1, 0)
];

// Notre matrice de transformation (Super Matrice)
let mScale = BABYLON.Matrix.Scaling(2, 0.5, 1);

// Vous faites ici ce que le Vertex Shader fera des millions de fois par seconde :
for(let i = 0; i < positions.length; i++) {
    positions[i] = BABYLON.Vector3.TransformCoordinates(positions[i], mScale);
}

// Vérifions le premier point :
console.log("Ancien bas-gauche (-1,-1,0) est devenu :", positions[0]);
// Affiche {X:-2, Y:-0.5, Z:0}

// À ce stade, vous pouvez extraire ces valeurs (X,Y,Z) et les injecter
// dans le VertexData.positions
```

! Félicitations

Vous venez de comprendre la base absolue de la programmation graphique. Quand, plus tard dans le cours, vous utiliserez `monObjet.position.x = 5`, vous saurez que derrière ce code simple, le moteur génère une matrice de translation et l'applique à des milliers de sommets exactement comme vous venez de le faire !

GPU

Graphics Processing Unit

10496 Cores

Lent - Tâches simples

CPU

Graphics Processing Unit

24 Cores

Rapide - Tâches complexes

Fig. 9. – Le GPU peut effectuer un grand nombre de multiplications de matrices.

Hiérarchie de scène : Le Parenting

Le concept du référentiel local

Les objets 3D sont organisés en hiérarchie parent-enfant. Le concept fondamental à retenir est que l'univers d'un enfant, c'est son parent.

L'origine (0, 0, 0) de l'enfant n'est plus le centre du monde, mais le centre de son parent. Ses axes (X, Y, Z) s'alignent sur ceux de son parent. C'est ce qu'on appelle le **Scene Graph**.

❗ Pourquoi ne peut-on pas juste additionner les positions ?

Si un parent est en $X = 10$ et que l'enfant est en $X = 2$ par rapport au parent, on pourrait penser que l'enfant est en $X = 12$ dans le monde.

C'est faux ! Que se passe-t-il si le parent a tourné de 90 degrés ? L'axe X de l'enfant pointe désormais vers l'axe Z du monde. C'est pour cela que la simple addition ne fonctionne pas.

Le GPU doit obligatoirement utiliser la multiplication de matrices pour calculer la vraie position finale :

$$\text{MatriceMonde}_{\text{Enfant}} = \text{MatriceMonde}_{\text{Parent}} \times \text{MatriceLocale}_{\text{Enfant}}$$

Atelier : Le Train et le Passager

Les propriétés de base dans Babylon.js

- `.position` : Toujours **locale** par rapport au parent.
- `.rotation` (ou `.rotationQuaternion`) : Orientation **locale**.
- `.scaling` : Échelle **locale**.

💡 L'objectif de l'atelier

Nous allons créer un « Train » (le parent) et un « Passager » (l'enfant). Nous allons observer ce qui se passe dans la console pour comprendre la différence entre la position que vous tapez dans le code (Locale) et la position réelle calculée par le GPU (Monde).

Micro-exercice 6 : Embarquement immédiat. **Consigne :** Créez une boîte rectangulaire (`train`) et placez-la à $X = 10$. Créez une petite sphère (`passager`). Appliquez la propriété `.parent` pour lier le passager au train. Placez le passager à la position locale $X = 2$. Observez les coordonnées.

```
// 1. Le Train (Le repère parent)
const train = BABYLON.MeshBuilder.CreateBox("train", {width: 4, height: 1, depth: 2}, scene);
train.position.x = 10; // Le train est à 10 unités de l'origine du monde

// 2. Le Passager (Le repère enfant)
const passager = BABYLON.MeshBuilder.CreateSphere("passager", {diameter: 0.5}, scene);

// LA MAGIE OPÈRE ICI :
passager.parent = train;

// La position locale du passager (Dans le référentiel du train)
passager.position = new BABYLON.Vector3(2, 1, 0);

// --- OBSERVATION DANS LA CONSOLE ---

// 1. Où est le passager par rapport au train ?
console.log("Position Locale : ", passager.position.x);
// Affiche : 2

// 2. Où est le passager par rapport au monde 3D ?
passager.computeWorldMatrix(true); // On force le moteur à calculer la multiplication des matrices
let worldPos = passager.getAbsolutePosition();

console.log("Position Monde : ", worldPos.x);
// Affiche : 12 (car le train n'a pas de rotation, l'addition fonctionne exceptionnellement ici)
```

Micro-exercice 7 : Le crash test (Rotation). **Consigne :** Ajoutez une rotation au train de 90 degrés sur l'axe Y ($\text{Math.PI} / 2$). Affichez à nouveau la position absolue du passager.

```
train.rotation.y = Math.PI / 2; // Tourne le train de 90 degrés

passager.computeWorldMatrix(true);
console.log("Nouvelle Position Monde X : ", passager.getAbsolutePosition().x);
// Affiche : 10 ! (Et non plus 12)

console.log("Nouvelle Position Monde Z : ", passager.getAbsolutePosition().z);
// Affiche : -2 !
```

Explication : La position locale du passager est toujours $x=2$. Mais comme le train a tourné, l'axe X du train correspond maintenant à l'axe Z négatif du monde. La matrice a calculé cette conversion complexe pour vous !

Concepts clés à retenir (Les pièges classiques)

🔗 L'héritage du Scale (L'effet Œuf)

Testez ceci : Appliquez `train.scaling.x = 2` pour allonger votre train. Que se passe-t-il ?

Votre passager (la sphère) se transforme en œuf aplati ! Pourquoi ? Parce que la matrice monde du train contient un **Scale x2**. En multipliant cette matrice, le GPU multiplie aussi les sommets de l'enfant par 2 sur cet axe.

Bonne pratique : En production de jeux vidéo, on évite presque toujours d'appliquer un « Scale » à un objet parent, pour éviter de déformer accidentellement tous ses enfants.

Transformation de vecteurs vs points

Rappelez-vous la coordonnée w . Les normales (l'orientation des faces pour la lumière) sont des **vecteurs** ($w = 0$), pas des points ($w = 1$). Elles ne subissent donc **pas** la translation de la matrice — seulement la rotation. C'est pourquoi une face orientée vers le haut pointe toujours vers le haut, même si on déplace l'objet de 1000 kilomètres.

💡 Question de réflexion pour la prochaine session

« Si je déplace ma caméra de 5 unités vers la droite, que doit faire le GPU avec tous les objets du monde pour qu'ils apparaissent correctement à l'écran ? »

Réponse : Il doit tous les déplacer de 5 unités vers la **gauche**. C'est le rôle de la Matrice Vue (View Matrix) — dans le moteur 3D, la caméra est fixe à l'origine, c'est l'univers entier qui bouge en sens inverse autour d'elle pour simuler le mouvement !

Session 3 : Création de Shaders dans Babylon.js (Partie 1)

🔗 Objectifs de la session

Comprendre ce qu'est un shader, pourquoi le GPU est si rapide, et écrire ses premiers shaders GLSL dans Babylon.js.

De la copie manuelle à la presse d'imprimerie

Analogie : Gutenberg et le Shader

Avant Gutenberg, chaque livre était copié à la main, lettre par lettre. C'est comme dessiner un cercle, puis un rectangle, puis une ligne : on donne des instructions une par une, exécutées en série.

Le shader est l'équivalent de la presse d'imprimerie : au lieu de dessiner chaque élément individuellement, on écrit **une fois** un programme (la matrice de caractères), et il s'applique partout simultanément.

Qu'est-ce qu'un Fragment Shader ?

Définition

Un shader est un programme exécuté sur la carte graphique (GPU). Contrairement aux API de dessin classiques où l'on dessine cercle, rectangle, triangle un par un, un shader exécute **les mêmes instructions sur chaque pixel de l'écran**.

Le programme est une fonction à laquelle on passe une position et qui renvoie une couleur :

$$\text{couleur} = f(\text{position})$$

Une fois compilé, ce programme s'exécute extrêmement rapidement.

Pourquoi les shaders sont-ils rapides ?

CPU : un tuyau, GPU : une grille de tuyaux

Imaginez le CPU comme un gros tuyau : les opérations passent une par une. Les ordinateurs récents ont plusieurs tuyaux (threads), mais ils restent limités.

Le GPU (Graphics Processing Unit) est une **grille** de milliers de petits microprocesseurs qui tournent en parallèle. Chaque pixel est une petite opération ; au lieu de les traiter séquentiellement, le GPU les traite tous simultanément.

💬 Le calcul en parallèle

Sur un écran 800x600 à 60 FPS : $480\,000 \text{ pixels} \times 60 = \mathbf{28\,800\,000 \text{ calculs par seconde}}$.

Sur un écran Rétina 2880x1800 à 60 FPS : $\mathbf{311\,040\,000 \text{ calculs par seconde}}$.

Un CPU seul ne peut pas suivre. Le GPU résout ce problème par le traitement parallèle : des milliers de petits cœurs traitent les pixels simultanément.

💡 Bonus : accélération matérielle

Le GPU accélère aussi certaines fonctions mathématiques directement au niveau du matériel : transformations de matrices, fonctions trigonométriques, etc. Ces opérations sont traitées à la vitesse de l'électricité.

Pourquoi les shaders font peur ?

Les deux contraintes des threads GPU

Pour fonctionner en parallèle, chaque thread doit être **indépendant** des autres. Cela implique deux contraintes :

- **Cécité** : un thread ne peut pas voir ce que font les autres threads. Il est impossible de vérifier le résultat d'un autre thread ou de transmettre des données entre threads.
- **Amnésie** : dès qu'un thread a fini son traitement, le GPU lui ré-assigne une autre opération. Le thread ne garde **aucune mémoire** de ce qu'il faisait la frame précédente. Il pourrait dessiner un bouton, puis un bout de ciel, puis du texte.

💡 Conséquence pour le code

Coder un shader demande un niveau d'abstraction différent : il faut écrire une fonction **générique** qui sait rendre une image entière uniquement à partir de la variation d'une position, sans aucun état persistant.

Le Pipeline de Rendu Programmable

Étapes du pipeline

1. **Vertex Shader** : Transforme chaque sommet de l'Object Space au Clip Space.
2. **Rasterization** : Le GPU convertit les triangles en pixels.
3. **Fragment Shader** : Calcule la couleur de chaque pixel (la fonction $f(\text{position}) \rightarrow \text{couleur}$).
4. **Output Merger** : La couleur finale est écrite dans le framebuffer.

GLSL : Le langage des Shaders

Qu'est-ce que le GLSL ?

GLSL (OpenGL Shading Language) est le langage pour écrire des shaders OpenGL. Les spécifications sont maintenues par le **Khronos Group**. Il existe d'autres langages selon les plateformes (HLSL pour DirectX, MSL pour Metal), mais nous nous concentrerons sur GLSL.

Syntaxe GLSL de base

```
// Vertex Shader : transforme la position du sommet
attribute vec3 position;
uniform mat4 worldViewProjection;

void main() {
    gl_Position = worldViewProjection * vec4(position, 1.0);
}

// Fragment Shader : la fonction f(position) -> couleur
// Ici, tous les pixels seront rouges
void main() {
    gl_FragColor = vec4(1.0, 0.0, 0.0, 1.0); // Rouge
}
```

Décryptage du code

- `gl_FragColor` est une variable **globale** fournie par GLSL. Elle représente la couleur de sortie du pixel. Le GPU lit cette variable à la fin de `main()` et l'écrit dans le framebuffer.
- `vec4(1.0, 0.0, 0.0, 1.0)` est un vecteur à 4 composantes : (R, G, B, A) . Les valeurs vont de 0.0 à 1.0 (pas de 0 à 255).
 - $R = 1.0$: rouge au maximum.
 - $G = 0.0, B = 0.0$: pas de vert ni de bleu.
 - $A = 1.0$: opacité complète (pixel opaque).
- **Résultat** : tous les pixels de la géométrie sont colorés en rouge pur.

💡 Rappelez-vous

Le fragment shader s'exécute sur **chaque pixel** simultanément. Chaque instance est aveugle et amnésique : elle ne connaît que sa propre position. `gl_FragColor` est le seul moyen de communiquer le résultat au GPU.

Shaders dans Babylon.js : ShaderMaterial

Créer un *ShaderMaterial*.

```
BABYLON.Effect.ShadersStore["customVertexShader"] = `
    attribute vec3 position;
    uniform mat4 worldViewProjection;
    varying vec3 vPosition;

    void main() {
        vPosition = position;
        gl_Position = worldViewProjection * vec4(position, 1.0);
    }
`;
```

```
BABYLON.Effect.ShadersStore["customFragmentShader"] = `
    varying vec3 vPosition;

    void main() {
```

```

    gl_FragColor = vec4(vPosition.x, vPosition.y, vPosition.z, 1.0);
  }
`;

const shaderMaterial = new BABYLON.ShaderMaterial(
  "shader", scene, { vertex: "custom", fragment: "custom" },
  { attributes: ["position"], uniforms: ["worldViewProjection"] }
);
mesh.material = shaderMaterial;

```

Uniforms et Varyings

Communication entre shaders

- **Uniforms** : Variables passées du CPU (JavaScript) au GPU (shader). Constantes pour tous les sommets/pixels.
- **Attributes** : Données par sommet (position, normal, UV).
- **Varyings** : Variables passées du Vertex Shader au Fragment Shader (interpolées par pixel).

Session 4 : Création de Shaders dans Babylon.js (Partie 2)

🔗 Objectifs de la session

Découvrir le Node Material Editor de Babylon.js

<https://iquilezles.org/>

La fonction step

<https://nme.babylonjs.com/#G31DQU>

La terre vue par NASA

Vous pouvez télécharger des textures de la terre sur ce site : <https://www.solarsystemscope.com/textures/>

<https://www.solarsystemscope.com/textures/#earth>

Vous trouverez le resultat du shader dans ce lien : <https://nme.babylonjs.com/#NSSZS9#1>

<https://nme.babylonjs.com/#NSSZS9#3>

Session 5 : Lumière et matériaux

🔔 Objectifs de la session

Comprendre la mathématique derrière l'éclairage — la loi de Lambert et le produit scalaire $\vec{N} \cdot \vec{L}$ — puis explorer les sources de lumières de Babylon.js et le modèle spéculaire de Phong/Blinn-Phong.

🔔 Rappel : pourquoi la lumière est essentielle

Rappelez-vous la Session 3 : le Fragment Shader calcule la couleur de chaque pixel. Mais sans lumière, un objet 3D n'est qu'un bloc uniforme — tous les pixels ont la même couleur, peu importe la géométrie. C'est la lumière qui révèle la forme, la profondeur et la texture des objets. L'éclairage est le pont entre la géométrie (les normales vues en Session 1) et l'apparence finale (la couleur calculée par le shader).

💡 Le rappel critique : les normales

En Session 1, nous avons défini les **normales** — des vecteurs perpendiculaires à chaque face, pointant vers l'extérieur. Ces normales sont l'information essentielle que le GPU utilise pour calculer l'éclairage. Sans normales correctes, la lumière n'a aucun sens. Le produit scalaire ($\vec{N} \cdot \vec{L}$) entre la normale de surface et la direction de la lumière détermine **l'intensité** de la lumière reçue par cette surface.

Partie 1 : L'Éclairage Mathématique — La Loi de Lambert

1. Le problème : combien de lumière reçoit une surface ?

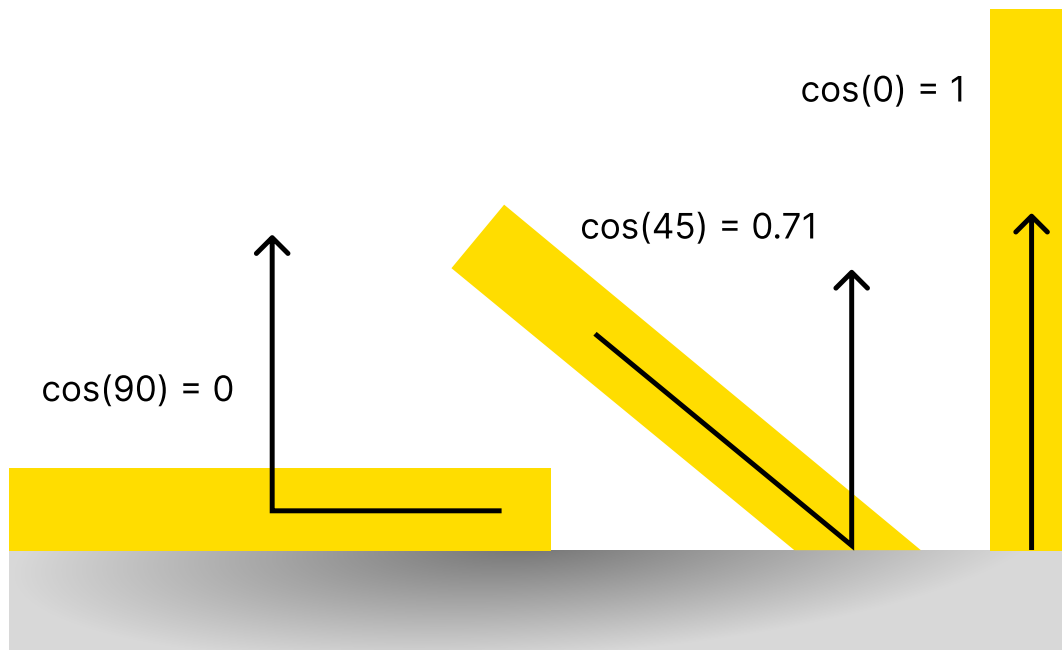


Fig. 10. – Loi de Lambert : L'intensité est proportionnelle au cosinus de l'angle entre la normale $\vec{N}$ et la direction de la lumière $\vec{L}$.

L'intuition physique

Imaginez une feuille de papier blanche tenue face au soleil. Si vous l'inclinez progressivement, elle devient de plus en plus sombre. Pourquoi ? Parce que la même quantité de lumière se répartit sur une **plus grande surface** projetée — chaque cm^2 reçoit moins d'énergie. C'est la **loi de Lambert** : l'intensité reçue est proportionnelle au cosinus de l'angle entre la normale de la surface et la direction de la lumière (voir Fig. 10).

La loi de Lambert (1760)

Pour une surface mate (diffuse), l'intensité lumineuse reçue vaut :

$$k_d = \max(\vec{N} \cdot \vec{L}, 0)$$

où $\vec{N}$ est la **normale unitaire** de la surface et $\vec{L}$ la **direction unitaire de la surface vers la lumière**. Le $\max(\dots, 0)$ coupe les valeurs négatives : une surface dos à la lumière ne reçoit rien.

2. Le produit scalaire, au cœur de tout

Rappel : le produit scalaire

Le produit scalaire de deux vecteurs u et v s'écrit :

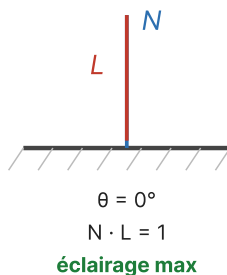
$$u \cdot v = \|u\| \|v\| \cos(\theta)$$

Si u et v sont **normalisés** ($\|u\| = \|v\| = 1$), alors :

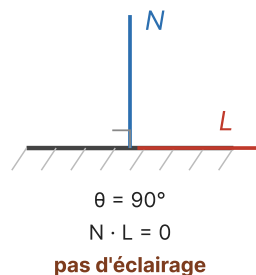
$$u \cdot v = \cos(\theta)$$

C'est la formule fondamentale de toute l'informatique graphique temps réel : un simple produit scalaire donne le cosinus de l'angle entre deux vecteurs.

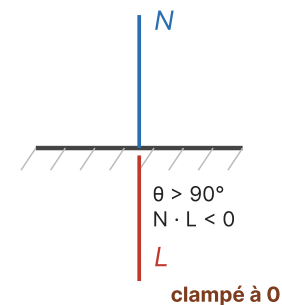
(a) Surface face à la lumière



(b) Lumière rasante



(c) Surface dos à la lumière



Les trois cas de la loi de Lambert : N (bleu) est la normale, L (rouge) est la direction de la lumière.

Fig. 11. – Les trois cas de la loi de Lambert : (a) surface face à la lumière, (b) lumière rasante, (c) surface dos à la lumière.

Les trois cas de la loi de Lambert

- Si $\vec{N}$ et $\vec{L}$ pointent dans la même direction (surface face à la lumière) : $\theta = 0$, $\vec{N} \cdot \vec{L} = 1$, éclairage **maximal**.
- Si $\vec{N}$ est perpendiculaire à $\vec{L}$ (lumière rasante) : $\theta = 90^\circ$, $\vec{N} \cdot \vec{L} = 0$, **pas d'éclairage**.
- Si $\vec{N}$ pointe à l'opposé de $\vec{L}$ (surface dos à la lumière) : $\theta > 90^\circ$, $\vec{N} \cdot \vec{L} < 0$, mais on utilise $\max(\dots, 0) \rightarrow$ **zéro** (voir Fig. 11).

C'est pourquoi une sphère éclairée présente un dégradé continu du côté éclairé vers le côté sombre : chaque fragment a une normale différente, donc un $\vec{N} \cdot \vec{L}$ différent.

3. La couleur finale : multiplication lumière $\times$ matériau

La formule complète du diffus

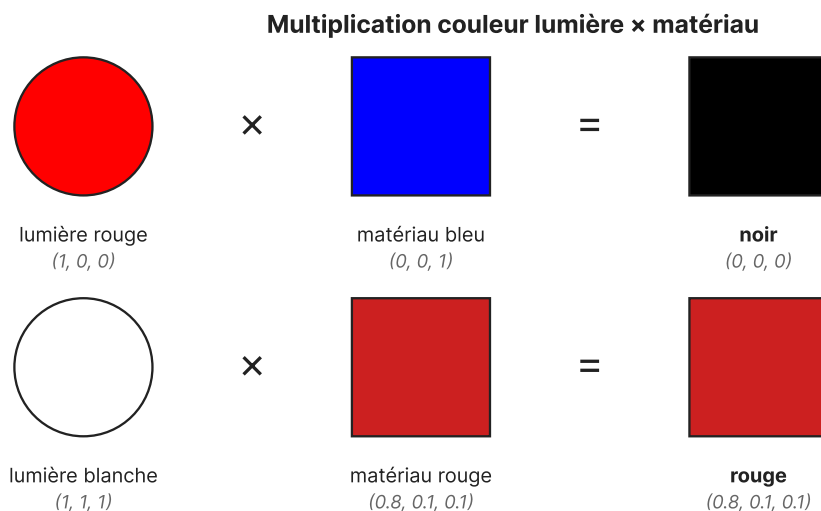
La couleur finale d'un fragment éclairé par la lumière diffuse est :

$$\text{color}_{\text{frag}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$$

- $\text{color}_{\text{light}}$: couleur de la source (ex. blanc (1, 1, 1)).
- $\text{color}_{\text{material}}$: couleur intrinsèque de la surface (ex. rouge (1, 0, 0)).
- $\max(\vec{N} \cdot \vec{L}, 0)$: le facteur d'angle de Lambert.

💡 Astuce : la couleur de la lumière se multiplie

La couleur de la lumière (diffuse) se **multiplie** avec la couleur du matériau. Une lumière rouge (1, 0, 0) sur un matériau bleu (0, 0, 1) donne... du noir (0, 0, 0) ! Le bleu ne réfléchit pas le rouge. C'est la physique réelle : un objet bleu sous lumière rouge apparaît noir (voir Fig. 12).



Un objet bleu sous lumière rouge apparaît noir : le bleu ne réfléchit pas le rouge.

Fig. 12. – La couleur finale est le produit composante par composante de la couleur de la lumière et de la couleur du matériau.

4. Écriture en GLSL : le shader de Lambert

Fragment shader minimal (Lambert seul).

```
// Varyings interpolés depuis le vertex shader
in vec3 vNormal;          // Normale du fragment (espace monde)
in vec3 vLightDir;        // Direction de la lumière vers le fragment

uniform vec3 uLightColor;  // Couleur de la lumière
uniform vec3 uMaterialColor; // Couleur du matériau

out vec4 fragColor;

void main() {
    // 1. Normaliser (les varyings ne sont plus unitaires après interpolation)
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);

    // 2. Produit scalaire = cos(theta), clampé à [0, 1]
    float diffuse = max(dot(N, L), 0.0);

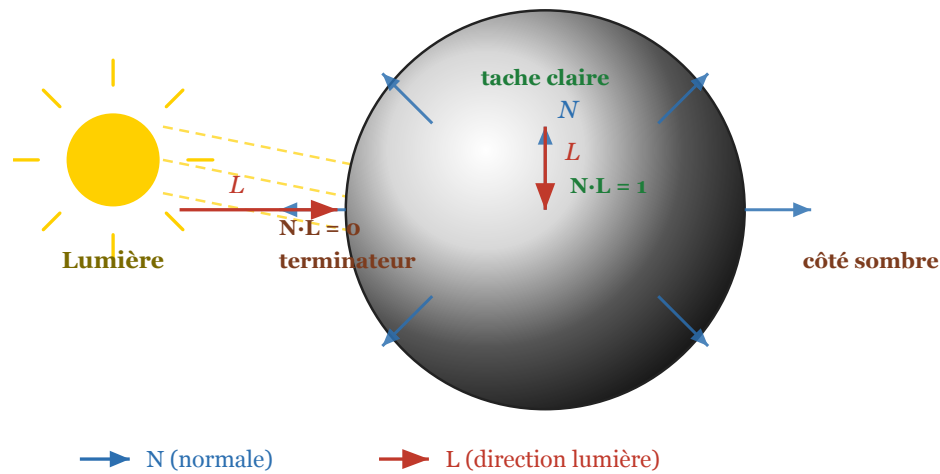
    // 3. Couleur finale = lumière × matériau × facteur d'angle
    vec3 color = uLightColor * uMaterialColor * diffuse;

    fragColor = vec4(color, 1.0);
}
```

❗ Pourquoi normaliser dans le fragment shader ?

Les normales sont interpolées **linéairement** entre les sommets du triangle. Cette interpolation ne préserve pas la norme : un vecteur unitaire aux sommets ne l'est plus forcément au milieu du fragment. Il faut donc **toujours** appeler `normalize(N)` dans le fragment shader, même si les normales étaient unitaires dans le vertex shader. Oublier cette étape est un bug classique qui donne un éclairage trop terne ou trop brillant.

5. Pourquoi Lambert fait un dégradé sur une sphère



Le dégradé sur la sphère vient de la variation de $N \cdot L$: chaque fragment a une normale différente.

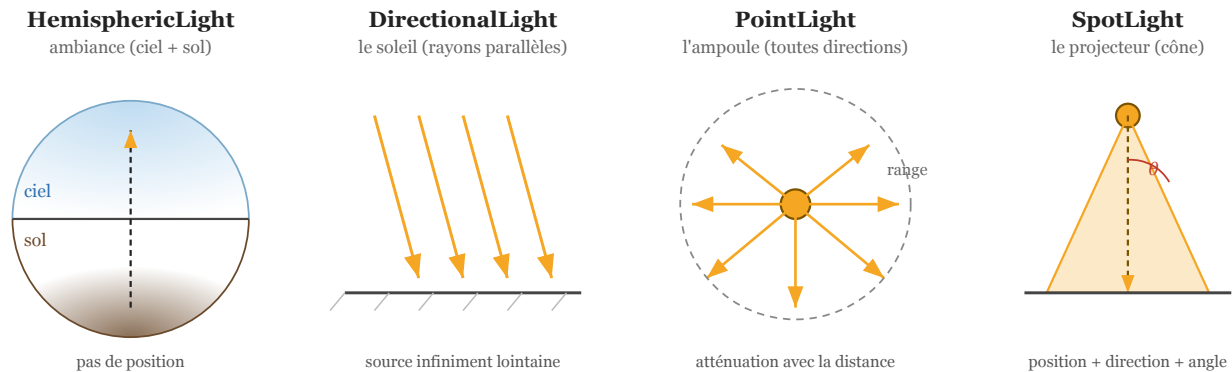
Fig. 13. – Une sphère éclairée : chaque fragment a une normale radiale différente. Au centre éclairé $\vec{N} \cdot \vec{L} = 1$ (tache claire), au terminator $\vec{N} \cdot \vec{L} = 0$, et le côté opposé reste sombre.

Géométrie d'une sphère éclairée

Sur une sphère, chaque fragment a une normale différente : elle pointe radialement vers l'extérieur. Au centre de la face éclairée, $\vec{N}$ et $\vec{L}$ sont alignés ($\vec{N} \cdot \vec{L} = 1$) → tache la plus claire. Au bord, $\vec{N}$ est perpendiculaire à $\vec{L}$ ($\vec{N} \cdot \vec{L} = 0$) → ombre. Entre les deux, $\vec{N} \cdot \vec{L}$ varie doucement de 1 à 0, créant le dégradé caractéristique (voir Fig. 13). C'est exactement ce que reproduit le shader ci-dessus, fragment par fragment.

Partie 2 : Sources de Lumières et Spéculaire

6. Les quatre types de lumières dans Babylon.js



Les quatre types de lumières intégrées de Babylon.js.

Fig. 14. – Les quatre types de lumières intégrées de Babylon.js : HemisphericLight (ambiance), DirectionalLight (soleil), PointLight (ampoule), SpotLight (projecteur).

HemisphericLight — La lumière ambiante

Simule la lumière du ciel et du sol. Elle n'a pas de position — elle éclaire toutes les surfaces en fonction de leur orientation (la normale).

- **Ciel** : Couleur pour les surfaces pointant vers le haut (groundColor inversé).
- **Sol** : Couleur pour les surfaces pointant vers le bas.
- **Direction** : Définit quel axe sépare le « haut » du « bas ».
- **Usage** : Lumière d'ambiance de base, évite que les zones non éclairées soient totalement noires.

Créer une *HemisphericLight*.

```
const hemiLight = new BABYLON.HemisphericLight("hemi", new BABYLON.Vector3(0, 1, 0), scene);

// Couleur du ciel (surfaces vers le haut)
hemiLight.diffuse = new BABYLON.Color3(0.8, 0.9, 1.0); // Bleu pâle

// Couleur du sol (surfaces vers le bas)
hemiLightgroundColor = new BABYLON.Color3(0.3, 0.2, 0.1); // Brun

// Couleur spéculaire
hemiLight.specular = new BABYLON.Color3(0, 0, 0); // Pas de reflet spéculaire

hemiLight.intensity = 0.6;
```

DirectionalLight — Le soleil

Lumière parallèle qui simule une source infiniment lointaine (comme le soleil). Tous les rayons arrivent dans la même direction. Elle n'a pas de position — seule la direction compte.

- **Direction** : Un vecteur normalisé qui indique la direction des rayons.
- **Pas de portée** : Éclaire toute la scène uniformément.
- **Usage** : Soleil, lumière naturelle, ombres nettes.

Créer une DirectionalLight.

```
const sunLight = new BABYLON.DirectionalLight("sun", new BABYLON.Vector3(-1, -2, -1), scene);

sunLight.diffuse = new BABYLON.Color3(1, 0.95, 0.8); // Lumière chaude
sunLight.specular = new BABYLON.Color3(1, 1, 1);      // Reflet blanc
sunLight.intensity = 1.2;

// La direction est normalisée automatiquement par Babylon.js
// mais il est recommandé de la normaliser soi-même :
sunLight.direction.normalize();
```

PointLight — L'ampoule

Lumière ponctuelle qui émet dans toutes les directions à partir d'une position dans l'espace. Plus un objet est loin, moins il est éclairé (atténuation).

- **Position** : Emplacement de la source dans la scène.
- **Range** : Portée maximale de la lumière. Au-delà, l'intensité tombe à zéro.
- **Atténuation** : L'intensité diminue avec la distance selon une courbe.
- **Usage** : Ampoule, torche, feu, lampe de poche (sans cône).

Créer une PointLight.

```
const bulb = new BABYLON.PointLight("bulb", new BABYLON.Vector3(0, 3, 0), scene);

bulb.diffuse = new BABYLON.Color3(1, 0.9, 0.7); // Lumière chaude
bulb.specular = new BABYLON.Color3(1, 1, 1);
bulb.intensity = 1.0;
bulb.range = 15; // Portée de 15 unités

// On peut animer la position pour simuler un mouvement
scene.registerBeforeRender(() => {
    bulb.position.x = Math.sin(performance.now() * 0.001) * 3;
});
```

SpotLight — Le projecteur

Lumière conique qui combine une position et une direction, avec un angle d'ouverture. C'est la lumière la plus complexe des quatre.

- **Position** : Origine du cône.
- **Direction** : Axe central du cône.
- **Angle** : Angle d'ouverture du cône (en radians, demi-angle).
- **Exponent (penumbra)** : Contrôle le flou du bord du cône. Plus la valeur est élevée, plus le bord est net.
- **Range** : Portée maximale.

- **Usage** : Lampe torche, projecteur de scène, phare de voiture.

Créer une *SpotLight*.

```
const spotlight = new BABYLON.SpotLight(
    "spot",
    new BABYLON.Vector3(0, 5, 0),    // Position (en haut)
    new BABYLON.Vector3(0, -1, 0),   // Direction (vers le bas)
    Math.PI / 4,                    // Angle : 45 degrés
    2,                              // Exponent (penumbra)
    scene
);

spotlight.diffuse = new BABYLON.Color3(1, 1, 0.8);
spotlight.intensity = 2.0;
spotlight.range = 20;
```

7. Paramètres communs à toutes les lumières

Propriétés partagées

- `light.diffuse` : Couleur principale de la lumière (`Color3`). C'est la couleur qui sera mélangée avec la couleur diffuse du matériau.
- `light.specular` : Couleur du reflet spéculaire (`Color3`). Souvent blanc, mais peut être coloré pour des effets créatifs.
- `light.intensity` : Multiplicateur d'intensité (`float`). Valeur par défaut : 1.0.
- `light.range` : Portée de la lumière (`float`). Utilisé par `PointLight` et `SpotLight`. Ignoré par `HemisphericLight` et `DirectionalLight`.
- `light.direction` : Direction de la lumière (`Vector3`). Utilisé par `HemisphericLight`, `DirectionalLight`, et `SpotLight`.

8. La lumière spéculaire (Modèle de Phong)

Au-delà du diffus : la brillance

La loi de Lambert explique les surfaces **mates** (bois, papier, béton). Mais une bille de verre, une voiture peinte ou un métal poli réfléchissent aussi la lumière comme un miroir — c'est la composante **spéculaire**. Elle ne dépend pas seulement de l'angle $\vec{N} \cdot \vec{L}$, mais aussi de la position de la **caméra** : le reflet se déplace quand on tourne autour de l'objet.

Le reflet : $\vec{R} \cdot \vec{V}$

La composante spéculaire simule le reflet brillant d'une lumière sur une surface lisse. Elle dépend de trois vecteurs :

- $\vec{L}$: Direction de la surface vers la lumière.
- $\vec{R}$: Rayon réfléchi (le miroir de $\vec{L}$ par rapport à la normale $\vec{N}$).
- $\vec{V}$: Direction de la surface vers la caméra.

$$k_s = \max(\vec{R} \cdot \vec{V}, 0)^{\text{shininess}}$$

L'exposant "shininess" contrôle la taille du reflet :

- Valeur faible (ex: 2) : reflet large et flou (plastique mat).
- Valeur élevée (ex: 128) : reflet petit et net (métal poli).

Le vecteur réfléchi $\vec{R}$

Le rayon réfléchi se calcule par la formule :

$$\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$$

Géométriquement : on décompose $\vec{L}$ en une composante normale $(\vec{N} \cdot \vec{L})\vec{N}$ et une composante tangente. La réflexion inverse la composante tangente, donc on double la composante normale et on soustrait $\vec{L}$. C'est la loi de la réflexion de Descartes — l'angle d'incidence égal l'angle de réflexion (voir Fig. 15).

Note : cette formule n'a de sens que lorsque $\vec{N} \cdot \vec{L} > 0$ (surface face à la lumière). Quand $\vec{N} \cdot \vec{L} < 0$, la surface est dos à la lumière et ne reçoit rien — le shader masque alors la spéculaire avec $\text{step}(0.0, \text{NdotL})$.

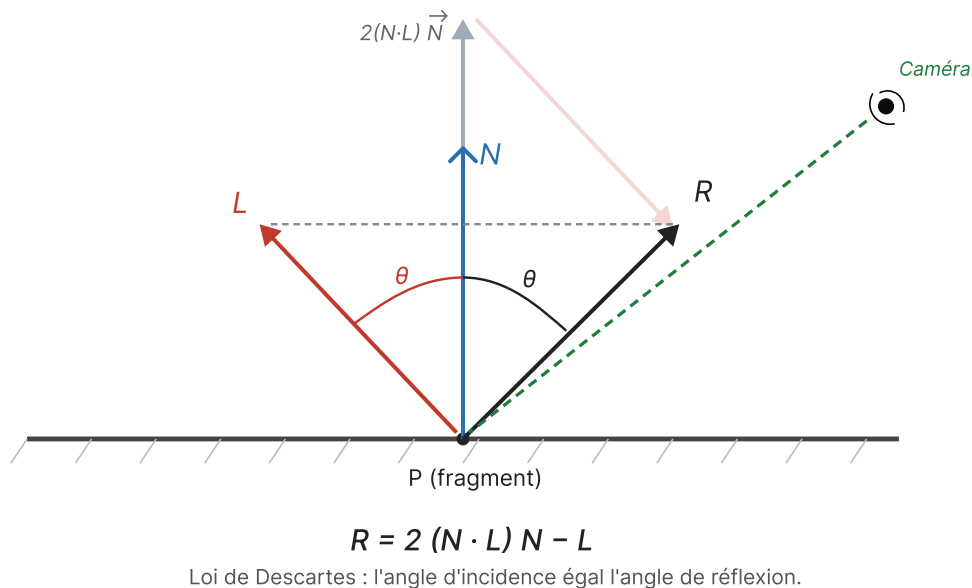


Fig. 15. – Le vecteur réfléchi $\vec{R} = 2(\vec{N} \cdot \vec{L})\vec{N} - \vec{L}$: symétrique de $\vec{L}$ par rapport à la normale $\vec{N}$. L'angle d'incidence θ égal l'angle de réflexion.

9. Blinn-Phong : la variante optimisée

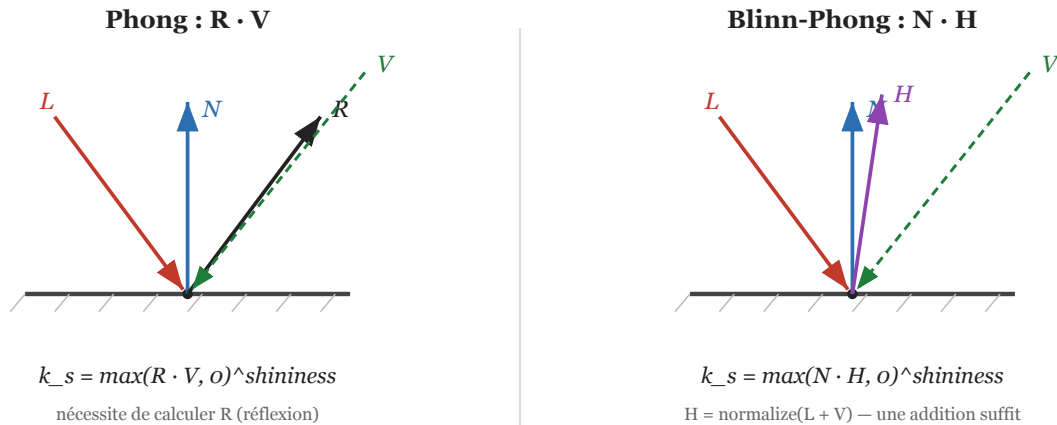
Le vecteur half-angle $\vec{H}$

Le modèle Blinn-Phong remplace le vecteur réfléchi $\vec{R}$ par le vecteur **half-angle** $\vec{H}$:

$$\vec{H} = \text{normalize}(\vec{L} + \vec{V})$$

$$k_s = \max(\vec{N} \cdot \vec{H}, 0)^{\text{shininess}}$$

Avantage : $\vec{H}$ est plus simple à calculer que $\vec{R}$ (pas besoin de réfléchir le vecteur — une addition + une normalisation suffit). C'est le modèle utilisé par défaut dans la plupart des moteurs temps réel, y compris le StandardMaterial de Babylon.js (voir Fig. 16).



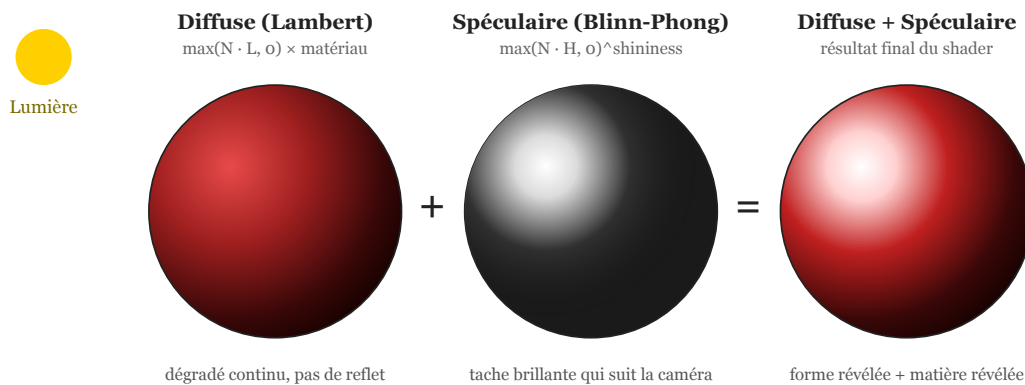
Blinn-Phong remplace R (coûteux) par $H = \text{normalize}(L + V)$. Visuellement équivalent, plus rapide.

Fig. 16. – Comparaison Phong vs Blinn-Phong. À gauche, on calcule le vecteur réfléchi $\vec{R}$ et on le compare à $\vec{V}$. À droite, on calcule le vecteur half-angle $\vec{H} = \text{normalize}(\vec{L} + \vec{V})$ et on le compare à $\vec{N}$.

💡 Pourquoi Blinn-Phong a remplacé Phong

Le produit scalaire $\vec{N} \cdot \vec{H}$ donne un angle **deux fois plus petit** que $\vec{R} \cdot \vec{V}$. Pour retrouver le même reflet visuel, on multiplie généralement shininess par 4 quand on passe de Phong à Blinn-Phong. Le résultat est visuellement très proche, mais le calcul est plus rapide — un gain précieux quand on rend des millions de fragments par frame.

10. Le shader complet : diffus + spéculaire



La composante diffuse révèle la couleur et la forme ; la spéculaire révèle la brillance et la matière.

Fig. 17. – La composante diffuse (Lambert) révèle la couleur et la forme ; la composante spéculaire (Blinn-Phong) révèle la brillance et la matière. La somme donne le rendu final.

Fragment shader Phong (Lambert + spéculaire).

```
in vec3 vNormal;
in vec3 vLightDir;
in vec3 vViewDir;      // Direction de la caméra vers le fragment

uniform vec3 uLightColor;
uniform vec3 uMaterialColor;
uniform vec3 uSpecularColor; // Couleur du reflet (souvent blanc)
uniform float uShininess;    // Exposant de brillance

out vec4 fragColor;

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

    // Produit scalaire brut (non clampé) – sert pour diffuse ET pour le masque
    float NdotL = dot(N, L);

    // --- Composante diffuse (Lambert) ---
    float diffuse = max(NdotL, 0.0);

    // --- Composante spéculaire (Phong) ---
    vec3 R = reflect(-L, N); // R = 2*(N.L)*N - L
    // step(0.0, NdotL) vaut 1 si NdotL >= 0, sinon 0 → masque sans branchement
    float specular = pow(max(dot(R, V), 0.0), uShininess)
        * step(0.0, NdotL);

    // --- Combinaison ---
    vec3 color = uLightColor * uMaterialColor * diffuse
        + uLightColor * uSpecularColor * specular;
```

```
fragColor = vec4(color, 1.0);
}
```

❗ Pourquoi `step()` plutôt qu'un `if` ?

Sans masque, la spéculaire apparaîtrait sur la face **non éclairée** — un reflet fantôme. Mais un `if (diffuse <= 0.0)` est un **branchement sur une valeur varying** : comme `diffuse` diffère d'un fragment à l'autre, les threads d'un même warp divergent et s'exécutent en série, ce qui annule l'avantage du SIMD.

La fonction `step(edge, x)` retourne 0 si $x < \text{edge}$, 1 sinon — c'est une opération **mathématique pure**, exécutée en lockstep par tous les threads. Multiplier la spéculaire par `step(0.0, NdotL)` coupe donc le reflet sur les faces sombres **sans aucune divergence**. C'est l'idiome standard en GLSL pour tout masque conditionnel.

Variante Blinn-Phong (plus rapide).

```
// ... même setup que ci-dessus ...

void main() {
    vec3 N = normalize(vNormal);
    vec3 L = normalize(vLightDir);
    vec3 V = normalize(vViewDir);

    float NdotL = dot(N, L);

    // --- Diffuse (Lambert) ---
    float diffuse = max(NdotL, 0.0);

    // --- Spéculaire (Blinn-Phong) ---
    vec3 H = normalize(L + V); // Vecteur half-angle
    float specular = pow(max(dot(N, H), 0.0), uShininess)
        * step(0.0, NdotL); // Masque sans branchement

    vec3 color = uLightColor * uMaterialColor * diffuse
        + uLightColor * uSpecularColor * specular;

    fragColor = vec4(color, 1.0);
}
```

11. StandardMaterial : assembler les deux en Babylon.js

Deux approches d'éclairage

Babylon.js propose deux principaux types de matériaux pour interagir avec la lumière :

- **StandardMaterial** : Modèle empirique (Lambert + Blinn-Phong). Simple, rapide, mais peu réaliste. Propriétés : `diffuseColor`, `specularColor`, `specularPower`.
- **PBRMaterial** : Modèle physiquement correct (Physically Based Rendering). Plus coûteux, mais produit des résultats réalistes. Propriétés : `albedoColor`, `metallic`, `roughness`. (Vu en détail dans une session ultérieure.)

StandardMaterial : Lambert + Blinn-Phong en une ligne.

```
const mat = new BABYLON.StandardMaterial("mat", scene);

// Composante diffuse (Loi de Lambert)
mat.diffuseColor = new BABYLON.Color3(0.8, 0.2, 0.2); // Rouge

// Composante spéculaire (Blinn-Phong)
mat.specularColor = new BABYLON.Color3(1, 1, 1); // Reflet blanc
mat.specularPower = 32; // Shininess

// Quand vous réglez specularPower, vous contrôlez directement
// l'exposant uShininess du shader vu ci-dessus.
```

Concepts clés à retenir

💡 Les 5 points essentiels

1. **La normale est reine** : Sans normales correctes, pas d'éclairage correct. Le produit scalaire $\vec{N} \cdot \vec{L}$ est la formule fondamentale.
2. **Lambert = cosinus de l'angle** : L'intensité diffuse est $\max(\vec{N} \cdot \vec{L}, 0)$. Un simple produit scalaire de vecteurs normalisés donne le cosinus — c'est toute la magie.
3. **La couleur se multiplie** : $\text{color}_{\text{final}} = \text{color}_{\text{light}} \times \text{color}_{\text{material}} \times \max(\vec{N} \cdot \vec{L}, 0)$. Une lumière rouge sur un objet bleu donne du noir.
4. **Quatre lumières, quatre usages** : HemisphericLight (ambiance), DirectionalLight (soleil), PointLight (ampoule), SpotLight (projecteur).
5. **Spéculaire = reflet qui dépend de la caméra** : Contrairement au diffus, le spéculaire bouge quand on tourne autour de l'objet. Blinn-Phong ($\vec{N} \cdot \vec{H}$) est l'optimisation standard de Phong ($\vec{R} \cdot \vec{V}$).

💡 Question de réflexion pour la prochaine session

« Nous savons maintenant éclairer des objets. Mais que se passe-t-il quand un objet bloque la lumière d'un autre ? Comment le GPU sait-il qu'un fragment est **derrière** un autre vu depuis la lumière ? »

Réponse : C'est le **Shadow Mapping**, que nous verrons en Session 6. Le GPU rend la scène depuis le point de vue de la lumière pour calculer ce qui est visible et ce qui est dans l'ombre.

Session 6 : L'Éclairage Statique

🔔 Objectifs de la session

Comprendre pourquoi l'éclairage temps réel est trop coûteux pour les grandes scènes, et comment les moteurs modernes précalculent l'éclairage hors ligne (baking). Découvrir les lightmaps, l'Ambient Occlusion, les light probes, et les stratégies d'éclairage mixte. Les exemples pratiques s'appuient sur la documentation de Godot, mais les concepts sont universels — ils s'appliquent à Unity, Unreal Engine, ou tout moteur custom.

Partie 1 : Le Problème — L'Éclairage Temps Réel est Trop Cher

Rappel : le coût de l'éclairage dynamique

En Session 5, nous avons vu que chaque fragment calcule $\vec{N} \cdot \vec{L}$ pour chaque lumière. Pour une scène avec 1000 fragments et 10 lumières, c'est 10 000 produits scalaires — et une scène réelle a des **millions** de fragments. Ajouter les ombres (shadow maps), la spéculaire, et l'atténuation, et le budget de 2 ms explose rapidement.

💡 Le constat des moteurs professionnels

Dans un jeu réel, la majorité de l'éclairage est **statique** : les bâtiments, les murs, le sol ne bougent pas. La lumière du soleil ne change pas (ou rarement). Pourquoi recalculer le même éclairage 60 fois par seconde ? La réponse : on **ne le fait pas**. On le calcule **une fois** hors ligne, on stocke le résultat dans des textures, et on l'affiche gratuitement à l'exécution.

Partie 2 : Les Lightmaps — L'Éclairage Cuit au Four

Principe du Lightmapping

Un **lightmap** est une texture qui stocke l'éclairage précalculé pour chaque surface de la scène. Au lieu de calculer $\vec{N} \cdot \vec{L}$ en temps réel, le GPU se contente de **lire** la couleur dans la texture — une opération $O(1)$, quasiment gratuite.

Le processus se déroule en deux phases :

1. **Offline (baking)** : Un algorithme de ray tracing ou de radiance calcule l'éclairage de chaque surface et l'écrit dans une texture UV.
2. **Runtime (rendering)** : Le fragment shader multiplie la couleur du matériau par la couleur du lightmap. Aucun calcul de lumière.

1. UV2 — Le dépliage de lightmap

Pourquoi un deuxième canal UV ?

Les UV1 servent à appliquer la texture du matériau (diffuse, normal map). Mais une lightmap a besoin d'un **dépliage différent** : chaque surface doit avoir sa propre région dans la texture, sans chevauchement, et avec une densité d'échantillonnage uniforme. C'est le rôle des **UV2** — un second canal UV dédié au lightmap.

- **UV1** : Optimisé pour la répétition de textures (tiling), peut chevaucher.
- **UV2** : Optimisé pour l'éclairage, **aucun chevauchement**, densité uniforme.

💡 Dans Godot

Godot génère automatiquement les UV2 lors du baking si l'option « Generate UV2 » est activée sur le MeshInstance3D. On peut aussi fournir des UV2 personnalisés depuis Blender par exemple. Voir la documentation Godot : MeshInstance3D GI Mode.

2. Le shader de lightmap — si simple

Fragment shader avec lightmap (GLSL générique).

```
in vec2 vUV;           // UV1 pour la texture du matériau
in vec2 vUV2;          // UV2 pour le lightmap
uniform sampler2D uDiffuseMap;
uniform sampler2D uLightmap;

out vec4 fragColor;

void main() {
    vec3 albedo = texture(uDiffuseMap, vUV).rgb;
    vec3 bakedLight = texture(uLightmap, vUV2).rgb;
    // Une simple multiplication — c'est tout !
    fragColor = vec4(albedo * bakedLight, 1.0);
}
```

💬 Pourquoi c'est si rapide

Le shader ci-dessus fait **deux lectures de texture et une multiplication**. Aucun produit scalaire, aucune boucle sur les lumières, aucune shadow map. C'est le shader d'éclairage le plus rapide qui existe — et il produit un rendu de qualité **ray tracing** (ombres douces, rebonds de lumière, color bleeding). Le coût : tout est précalculé et immobile.

3. Limitations des lightmaps

⚠️ Ce que les lightmaps ne peuvent pas faire

- **Objets en mouvement** : Un personnage qui marche sur un sol éclairé par lightmap ne reçoit pas l'éclairage du lightmap — il faut un éclairage dynamique séparé.
- **Lumières dynamiques** : Si une lumière s'allume ou s'éteint, le lightmap ne change pas. Il faut rebaker.
- **Mémoire** : Les lightmaps sont des textures, souvent en HDR, qui consomment de la VRAM. Une grande scène peut nécessiter plusieurs lightmaps de 2048×2048.
- **Temps de baking** : Le précalcul peut prendre de quelques secondes à plusieurs heures selon la complexité de la scène et la qualité demandée.

4. Lightmaps dans Godot

Le système LightmapGI

Godot utilise le noeud `LightmapGI` pour précalculer l'éclairage. Le baking utilise une approche par voxel + ray tracing :

- **Bake Quality** : Low, Medium, High, Ultra — contrôle la densité des voxels et le nombre de rayons.
- **Directional** : Si activé, stocke aussi la direction de l'éclairage (normales de lightmap), ce qui permet aux normal maps d'affecter l'éclairage baked.
- **Interior** : Optimise pour les scènes d'intérieur (pas de lumière du ciel).

Le résultat est stocké dans un fichier `.exr` (HDR) et appliqué automatiquement aux meshes dont le `GI Mode` est réglé sur `Static`.

💡 Documentation Godot

- `LightmapGI` : `LightmapGI Class`
- Guide d'utilisation : `Global Illumination in Godot`

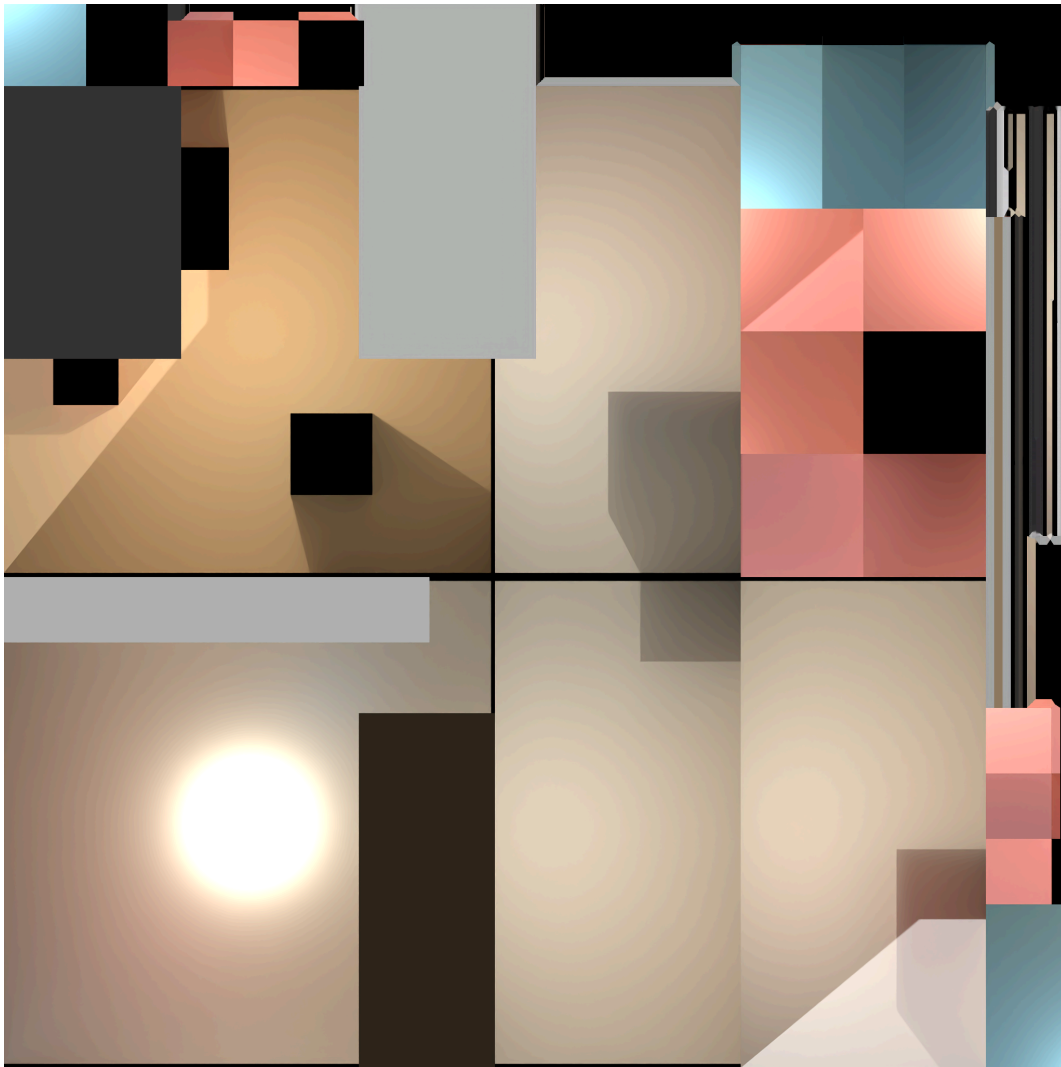


Fig. 18. – Exemple de lightmap baked dans Blender (Cycles, Combined bake, 2048×2048). Chaque îlot UV correspond à une surface de la scène. On y voit l'éclairage direct (taches claires), les ombres portées, et l'occlusion ambiante dans les coins. Cette texture est multipliée par l'albedo au runtime — aucune lumière temps réel nécessaire.

Partie 3 : L'Ambient Occlusion (AO)

Qu'est-ce que l'Ambient Occlusion ?

L'Ambient Occlusion simule l'assombrissement qui se produit dans les crevasses, les coins et les zones où la géométrie bloque la lumière ambiante. C'est un phénomène **global** : il ne dépend pas d'une lumière particulière, mais de la géométrie environnante.

$$AO(p) = \text{ratio de directions non bloquées vers l'hémisphère au-dessus de } p$$

Plus une surface est entourée d'autres surfaces, plus l'AO est faible (sombre). Les coins de murs, les plis de tissu, les articulations mécaniques sont des zones de fort AO.

5. AO Cuit (Baked AO) vs SSAO

Deux approches pour l'AO

- **Baked AO** : Précalculé hors ligne et stocké dans une texture (comme un lightmap). Qualité parfaite, coût runtime nul. Mais statique — ne s'adapte pas aux objets en mouvement.
- **SSAO (Screen Space Ambient Occlusion)** : Calculé en temps réel dans l'espace écran en utilisant le depth buffer. Approximation, mais dynamique — réagit aux objets en mouvement. Coût GPU non négligeable.

💡 Dans Godot

- **Baked AO** : Intégré automatiquement dans le LightmapGI — l'occlusion ambiante est calculée pendant le baking et stockée dans le lightmap.
- **SSAO** : Activé via Environment > SSAO dans les WorldEnvironment. Voir Environment SSAO.

6. Pourquoi l'AO change tout visuellement



Fig. 19. – Exemple d'Ambient Occlusion baked dans Blender (Cycles, Bake Type: Ambient Occlusion).

Les zones claires reçoivent la lumière ambiante, les zones sombres sont occluses par la géométrie voisine. Remarquez comment les coins, le contact des boîtes avec le sol, et les crevasses sont assombris — c'est ce qui donne le sens du contact et de la profondeur.

💡 L'effet Wadler

Sans AO, une scène 3D paraît « plate » — les coins sont aussi clairs que le centre des murs, les objets semblent flotter. Avec AO, chaque crevasse s'assombrit légèrement, ce qui donne une perception immédiate de **profondeur** et de **contact**. C'est l'un des effets les plus rentables en termes de qualité perçue par rapport au coût de calcul.

Partie 4 : Les Light Probes — L'Éclairage pour les Objets Dynamiques

Le problème des objets en mouvement

Les lightmaps éclairent les surfaces **statiques**. Mais un personnage qui marche dans une scène baked n'a pas de lightmap — il est éclairé par des lumières temps réel, ce qui le détache visuellement de l'environnement. Comment lui donner l'éclairage de la scène sans tout recalculer ?

Light Probes

Les **light probes** sont des points placés dans la scène qui **échantillonnent** l'éclairage baked à leur position. Au runtime, un objet dynamique (personnage, véhicule) interpole l'éclairage entre les probes les plus proches. L'objet reçoit un éclairage qui correspond à son environnement — sans aucun calcul de lumière en temps réel.

- **Placement** : Les probes sont disposées manuellement ou automatiquement dans la scène, surtout autour des zones où des objets dynamiques passent.
- **Stockage** : Chaque probe stocke l'éclairage incoming sous forme d'harmoniques sphériques (SH) — une compression de l'éclairage en 9 coefficients par canal RGB.
- **Interpolation** : Le GPU interpole entre les 4 probes les plus proches (interpolation tétraédrique) pour obtenir l'éclairage à la position de l'objet.

💡 Dans Godot

Godot utilise un noeud `LightmapProbe` placé dans la scène. Les objets dynamiques dont le `GI Mode` est réglé sur `Dynamic` utilisent automatiquement les probes pour recevoir l'éclairage baked. Voir `LightmapProbe Class`.

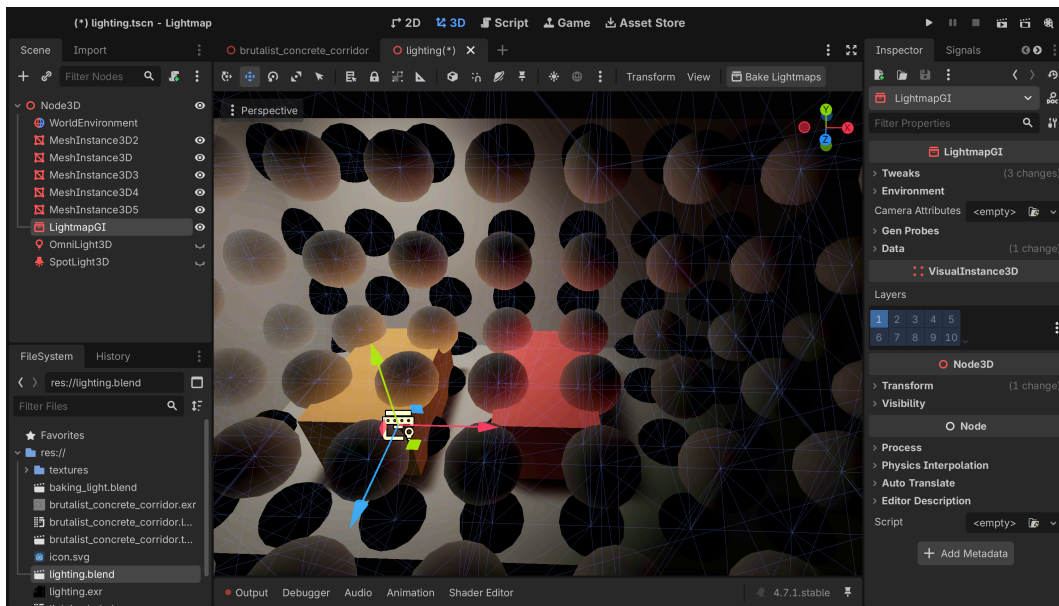


Fig. 20. – Exemple de placement de light probes dans une scène. Les sphères représentent les probes — chacune capture l'éclairage baked à sa position. Un objet dynamique qui se déplace interpole l'éclairage entre les probes les plus proches, ce qui lui donne un éclairage cohérent avec son environnement sans aucun calcul de lumière temps réel.

7. Spherical Harmonics — La compression magique

Pourquoi pas simplement stocker RGB ?

L'éclairage en un point dépend de la **direction** — une surface pointant vers le haut reçoit la lumière du ciel, une surface pointant vers le sol reçoit la lumière réfléchi du sol. Stocker l'éclairage pour toutes les directions nécessiterait un cube map par probe — beaucoup trop de mémoire.

Les **harmoniques sphériques** (SH) décomposent l'éclairage directionnel en une somme de fonctions de base. Avec seulement 9 coefficients par couleur (27 valeurs au total), on peut reconstruire un éclairage **directionnel** de basse fréquence — suffisant pour l'éclairage diffus d'un personnage. C'est une compression avec un ratio d'environ 100:1 par rapport à un cube map.

Utilisation des SH dans un shader (pseudo-code).

```
// 9 coefficients par canal (27 valeurs au total)
uniform vec3 shCoeffs[9];

// N = normale du fragment
vec3 evalSH(vec3 N) {
    // Basis functions d'ordre 0, 1, 2
    float sh[9];
    sh[0] = 0.282095; // Y_0^0
    sh[1] = 0.488603 * N.y; // Y_1^-1
    sh[2] = 0.488603 * N.z; // Y_1^0
    sh[3] = 0.488603 * N.x; // Y_1^1
    sh[4] = 1.092548 * N.x * N.y; // Y_2^-2
    sh[5] = 1.092548 * N.y * N.z; // Y_2^-1
    sh[6] = 0.315392 * (3.0 * N.z * N.z - 1.0); // Y_2^0
    sh[7] = 1.092548 * N.x * N.z; // Y_2^1
```

```
sh[8] = 0.546274 * (N.x*N.x - N.y*N.y); // Y_2^2

vec3 color = vec3(0.0);
for (int i = 0; i < 9; i++)
    color += shCoeffs[i] * sh[i];
return color;
}
```

Partie 5 : L'Éclairage Mixte (Mixed Lighting)

Le meilleur des deux mondes

L'éclairage **mixte** combine l'éclairage baked (statique) et dynamique (temps réel) :

- **Lumières statiques** (soleil, lampes fixes) : Baked dans les lightmaps.
- **Lumières dynamiques** (lampe torche du joueur, explosions) : Calculées en temps réel.
- **Objets statiques** (murs, sol) : Éclairés par lightmap.
- **Objets dynamiques** (personnages) : Éclairés par light probes + lumières temps réel.

C'est l'approche utilisée par la majorité des jeux commerciaux.

8. Les modes d'éclairage mixte dans Godot

GI Mode	Statique	Dynamique	Coût runtime
Disabled	Aucun	Tout temps réel	Élevé
Static	Lightmap + LightmapGI	LightmapProbe pour objets dyn.	Faible
Dynamic	SDFGI + LightmapProbe	SDFGI temps réel	Moyen

Tableau 1. – Les trois modes GI de Godot. Static est l'équivalent du baked — le plus utilisé en production. Dynamic utilise SDFGI (Signed Distance Field GI) pour l'illumination globale temps réel.

Documentation Godot

- GI Modes : MeshInstance3D GI Mode
- SDFGI : SDFGI in Environment
- Global Illumination : Global Illumination Guide

Partie 6 : Reflection Probes — Les Réflexions Baked

Le problème des réflexions sur objets baked

Un objet avec un matériau métallique ou brillant réfléchit son environnement. En temps réel, les cube maps dynamiques sont trop chers. Solution : précalculer des cube maps à des positions clés de la scène — les **reflection probes**.

Reflection Probes

Une **reflection probe** est un cube map de l'environnement rendu (baked) à une position donnée. Au runtime, un objet brillant utilise la probe la plus proche pour calculer sa réflexion. Comme les light probes, on interpole entre plusieurs probes pour les transitions.

- **Baked** : Précalculée hors ligne. Statique mais gratuite.
- **Realtime** : Mise à jour à l'exécution. Coûteuse mais dynamique.
- **Custom** : Cube map fourni manuellement (ex: HDR environment).

🔗 Dans Godot

Godot utilise le noeud `ReflectionProbe` avec la propriété `update_mode` réglée sur `Once` (baked) ou `Always` (temps réel). Voir `ReflectionProbe Class`.

Partie 7 : Le Pipeline Complet d'Éclairage Statique

Workflow type dans un moteur moderne

1. **Préparer la scène** : Placer les lumières statiques, les meshes statiques.
2. **Générer les UV2** : Dépliage automatique ou manuel pour les lightmaps.
3. **Placer les light probes** : Autour des zones de passage des objets dynamiques.
4. **Placer les reflection probes** : Dans les zones avec des matériaux brillants.
5. **Bake** : Lancer le précalcul (secondes à heures selon la scène).
6. **Runtime** : Les objets statiques utilisent les lightmaps, les objets dynamiques utilisent les light probes, les matériaux brillants utilisent les reflection probes.

Partie 8 : Le Texture Baking pour Personnages

Pas seulement les scènes !

Le baking ne sert pas qu'à précalculer l'éclairage des murs et du sol. Il est aussi **indispensable** pour les personnages et objets : on bake des détails d'un mesh **high-poly** vers un mesh **low-poly** pour conserver l'apparence sans le coût géométrique. C'est le workflow standard de tout personnage de jeu vidéo.

9. Le problème : high-poly vs low-poly

Le dilemme du modélisateur

Un personnage sculpté dans ZBrush ou Blender peut avoir 10 millions de polygones — beaucoup trop pour un moteur temps réel. On crée donc une version **low-poly** (5 000 à 50 000 triangles) qui sera utilisée au runtime. Mais la version low-poly perd tous les détails fins : cicatrices, écailles, coutures, rugosité.

Solution : On **bake** les détails de la high-poly vers des textures appliquées sur la low-poly. Le moteur lit ces textures au runtime — les détails réapparaissent sans le coût géométrique.

Étape	Mesh	Résultat
1. Sculpt	High-poly (10M+ tris)	Détails géométriques
2. Retopo	Low-poly (5K-50K tris)	Géométrie légère
3. Bake	High → Low	Textures de détails
4. Runtime	Low-poly + textures	Apparence high-poly

Tableau 2. – Le workflow high-poly → low-poly : on sculpte les détails, on crée une topologie légère, puis on bake les détails dans des textures.

10. Les textures que l'on bake

Les maps essentielles pour un personnage

- **Normal Map** : La plus importante. Encode les détails géométriques (crevasses, bosses, écailles) sous forme de normales perturbées. Le fragment shader les utilise pour simuler du relief sans géométrie supplémentaire. Vue en détail en Session 7.
- **Ambient Occlusion Map** : Assombrit les crevasses du personnage (aisselle, plis, sous les cheveux). Précalculée depuis la géométrie high-poly, elle ajoute du contact visuel même sans éclairage dynamique.
- **Curvature Map** : Détecte les zones convexes (bords, arêtes) et concaves (creux). Utilisée pour l'usure (poussière dans les creux, éclats sur les arêtes) ou le masquage de matériaux.
- **Cavity Map** : Variante d'AO plus locale — ne capte que les crevasses très fines. Souvent mélangée avec l'AO pour un résultat plus riche.
- **Diffuse/Albedo (baked)** : Parfois on bake la couleur depuis le high-poly (ex: polypainting de ZBrush) vers la low-poly. Utile quand le sculpt contient déjà les couleurs.
- **Position Map / ID Map** : Stocke la position ou un identifiant par région du mesh. Sert pour masquer des zones dans le shader (ex: peau vs métal sur un cyborg).

11. Le baking dans Blender

Le moteur Cycles pour le baking

Blender utilise le moteur **Cycles** pour baker des textures. Le processus :

1. **Préparer les UVs** de la low-poly (UV1, non-chevauchants).
2. **Créer une image texture** vide (ex: 2048×2048) dans l'UV Editor.
3. **Sélectionner** d'abord la high-poly, puis la low-poly (ordre important).
4. **Dans le Render Properties** : sélectionner **Cycles**, puis **Bake**.
5. **Bake Type** : choisir le type de map (Normal, Ambient Occlusion, Diffuse, etc.).
6. **Cage** : Blender projette les rayons depuis la low-poly vers la high-poly. Le **cage** définit la distance maximale de projection — trop petit, des détails sont manqués ; trop grand, des artefacts apparaissent.

Bake d'une Normal Map dans Blender (workflow).

1. Importer high-poly et low-poly dans la même scène
2. Sélectionner high-poly → Shift+click low-poly
3. Render Properties > Engine: Cycles
4. Bake > Bake Type: Normal
5. Influence > Space: Tangent (standard pour jeux)
6. Margin: 16px (évite les seams aux bords d'UV)
7. Click "Bake" → l'image se remplit
8. Sauvegarder l'image (PNG ou TGA)

💡 Le margin (dilatation)

Les UV islands ne sont jamais parfaitement alignés — il y a des espaces entre eux. Sans **margin**, des pixels noirs apparaissent aux coutures. Le margin (ex: 16px) **dilate** la couleur des islands vers l'extérieur pour remplir ces espaces. C'est un réglage critique pour éviter les seams visibles sur le personnage.

12. Utiliser les textures baked dans un moteur

Assemblage dans le matériau

Les textures baked sont combinées dans le matériau du personnage :

- **Normal Map** → branchée sur le Normal du matériau (via un Normal Map node).
- **AO Map** → multipliée avec l'albedo ou ajoutée comme masque d'assombrissement.
- **Curvature Map** → utilisée comme masque pour l'usure, la poussière, ou les effets de bord.

Au runtime, le personnage low-poly + ses textures baked donne l'illusion d'un mesh high-poly — pour une fraction du coût GPU.

💡 Dans Godot

- **Normal Map** : propriété `normal_texture` du `StandardMaterial3D`.
- **AO Map** : propriété `ao_texture` du `StandardMaterial3D`.
- **Roughness/Metallic** : peuvent aussi être baked et branchés sur `roughness_texture` et `metallic_texture`.
- Voir `StandardMaterial3D` Documentation.

Concepts clés à retenir

💡 Les 7 points essentiels

1. **Lightmap = éclairage précalculé dans une texture** : Le GPU lit au lieu de calculer. Qualité ray tracing pour le prix d'une texture lookup.
2. **UV2 = canal UV dédié au lightmap** : Pas de chevauchement, densité uniforme. Les UV1 restent pour les textures de matériau.
3. **Light Probes = éclairage pour les objets dynamiques** : Échantillonnent l'éclairage baked à des points clés, interpolé via harmoniques sphériques. Les personnages ne sont plus détachés de la scène.

4. **Ambient Occlusion = assombrissement des crevasses** : Baked (parfait, statique) ou SSAO (dynamique, approximatif). L'effet le plus rentable en qualité perçue.
5. **Éclairage mixte = baked + temps réel** : Les lumières statiques sont baked, les lumières dynamiques (joueur, explosions) restent temps réel. C'est ce que font les jeux commerciaux.
6. **Reflection probes = réflexions baked** : Cube maps précalculés pour les matériaux brillants. Comme les light probes mais pour les réflexions.
7. **Texture baking pour personnages** : On bake les détails du high-poly (normal map, AO, courbure) vers la low-poly. Le personnage léger a l'apparence du sculpt détaillé — c'est le workflow standard de tout jeu commercial.

🔗 Question de réflexion pour la prochaine session

« Nous avons vu comment **baker** des normal maps et des AO maps pour les personnages. Mais comment ces textures sont-elles **appliquées** sur une surface ? Comment répéter une texture de brique sur un mur infini ? Comment éviter le flou quand on regarde une texture de loin ? Et quels formats de texture utilise-t-on en production ? »

Réponse : C'est la **texturisation de base**, que nous verrons en Session 7 — UV1, tiling, mipmaps, formats de compression, et sampler filtering.

Travail Pratique : Baked Lighting en Three.js

🔗 Objectif du TP

Mettre en pratique les concepts de lightmapping et d'environnement map. Vous allez :

1. Générer une scène dans Blender avec un script Python
2. Baker l'éclairage dans Blender (Cycles)
3. Exporter la scène en .glb
4. Appliquer le lightmap et un environment map en Three.js

Étape 1 : Générer la scène dans Blender

Script Python fourni

Un script `generate_bake_scene.py` est fourni. Il crée :

- Une pièce (sol, 3 murs, plafond)
- 3 boîtes de tailles différentes
- 2 lumières (soleil + ampoule)
- Les UV2 dépliés automatiquement (prêts pour le baking)

Procédure :

1. Ouvrez Blender > onglet **Scripting**
2. Collez le contenu de `generate_bake_scene.py`
3. Exécutez le script (bouton **Run Script**)
4. La scène apparaît avec les objets, lumières et UV2

Étape 2 : Baker le lightmap dans Blender

Baking avec Cycles

1. Dans **Render Properties** : moteur **Cycles**, **GPU** si disponible
2. Créez une nouvelle image dans l'UV Editor (2048×2048, 32-bit Float si EXR)
3. Sélectionnez tous les meshes (Shift+A > Select All)
4. **Bake > Bake Type : Combined**
5. **Margin** : 16px
6. Cliquez **Bake** — l'image se remplit
7. Sauvegardez l'image : session06_lightmap.png (ou .exr pour HDR)

💡 Conseils

- **Combined** bake inclut diffus + ombres + AO en une seule texture
- Si le résultat est trop sombre, augmentez l'énergie des lumières
- Vérifiez que les UV2 sont bien actifs avant de baker

Étape 3 : Exporter en .glb

Export glTF

1. **File > Export > glTF 2.0 (.glb)**
2. Options : cochez **UVs**, décochez **Compression** pour simplifier
3. Nommez le fichier session06_bake_scene.glb
4. Placez-le dans le même dossier que les fichiers Three.js

Étape 4 : Appliquer le lightmap en Three.js

Le fichier de départ

Un fichier session06_baked_lighting.js est fourni avec :

- Le chargement de la scène .glb via GLTFLoader
- Une GUI pour comparer lightmap vs lumières temps réel
- Des **TODO ÉTUDIANT** marqués dans le code

Votre mission :

1. Charger la texture de lightmap (session06_lightmap.png)
2. L'assigner à material.lightMap sur chaque mesh
3. Régler lightMapIntensity
4. (Bonus) Charger un environment map HDR avec RGBELoader
5. (Bonus) Comparer les FPS : lightmap vs lumières temps réel

⚠️ Pièges fréquents

- **Color space** : Le lightmap doit être en sRGBColorSpace si bake en sRGB, ou linéaire si EXR
- **UV2** : Three.js utilise le deuxième canal UV automatiquement pour lightMap — vérifiez que le .glb contient bien les UV2

- **glTF export** : Assurez-vous que les UV2 sont exportés (option **UVs** cochée)
- **Intensité** : Si la scène est trop sombre/claire, ajustez `lightMapIntensity` (0 à 3)

💡 Résultat attendu

À la fin du TP, vous devriez avoir une scène 3D éclairée **sans aucune lumière temps réel** — tout vient du lightmap. La GUI permet de :

- Activer/désactiver le lightmap
- Activer/désactiver l'environnement map
- Activer/désactiver les lumières temps réel pour comparer

La différence de FPS entre lightmap et lumières temps réel devrait être significative — c'est tout l'intérêt du baking !

Session 7 : Textures — Les Fondamentaux

💡 Objectifs de la session

Comprendre ce qu'est une texture, comment elle est appliquée sur une géométrie via les UVs, comment le GPU la filtre et l'échantillonne, et découvrir le **set de textures PBR** qui définit l'apparence d'un matériau moderne. La session suit le pipeline réel : **créer et préparer les textures dans Blender**, puis **les intégrer dans un moteur** (Babylon.js / Node Material Editor).

💡 Rappel : les UVs vus en Session 1

En Session 1, nous avons défini les **UVs** — des coordonnées $(u, v) \in [0, 1]^2$ qui relient chaque sommet à un pixel d'une texture 2D. Cette session approfondit **comment** cette application fonctionne, ce qui peut mal se passer, et comment les moteurs modernes utilisent non pas une mais **plusieurs** textures pour définir un matériau.

Partie 1 : Qu'est-ce qu'une Texture ?

Définition

Au départ, une texture est un fichier image (PNG, JPG, etc.) qui contient des informations visuelles. On va ensuite charger cette image dans la mémoire du GPU (VRAM) pour qu'elle puisse être utilisée par le fragment shader via un **sampler**. Le shader donne au sampler des coordonnées UV, le sampler retourne la couleur du **texel** correspondant — le pixel de la texture.

💡 Texels vs Pixels

Un **texel** est un pixel de la texture. Un **pixel** est un pixel de l'écran. Ce ne sont pas les mêmes ! Une texture 2048×2048 a 4 millions de texels, mais un fragment shader peut en échantillonner beaucoup plus ou beaucoup moins selon la distance à la caméra. C'est ce déséquilibre qui rend le **filtrage** nécessaire (Partie 3).

Le sampler

En GLSL, on déclare une texture comme un uniform `sampler2D` et on lit une couleur avec `texture(sampler, uv)` :

```
uniform sampler2D uAlbedoMap;    // la texture
in vec2 vUV;                    // UV interpolée par le rasterizer

void main() {
    vec3 albedo = texture(uAlbedoMap, vUV).rgb;
    fragColor = vec4(albedo, 1.0);
}
```

Le sampler est l'interface entre le shader et la VRAM — il gère le filtrage, le wrapping et les mipmaps automatiquement (voir Parties 3 et 4).

Partie 2 : UV Mapping — Le Dépliage dans Blender

Le problème du dépliage

Une texture 2D est plate. Un mesh 3D ne l'est pas. Le **UV mapping** consiste à « déplier » la surface 3D sur le plan 2D de la texture — comme déplier une boîte en carton pour la poser à plat. Chaque sommet reçoit une coordonnée (u, v) qui pointe dans la texture.

1. Seams, islands et stretching

Vocabulaire du dépliage

- **Seam** (couture) : une arête où l'on « coupe » le mesh pour pouvoir le déplier. Comme couper un papier cadeau.
- **Island** (îlot) : un groupe de faces connectées, dépliées ensemble. Un mesh déplié produit plusieurs îlots dans la texture.
- **Stretching** : quand une face 3D est projetée sur une zone 2D trop petite ou disproportionnée — la texture apparaît étirée ou compressée.
- **Packing** : disposition des îlots dans la texture pour minimiser l'espace gaspillé. Les îlots ne doivent pas se chevaucher (sauf si volontaire).

🔗 Démo Blender — dépliage d'un cube

Dans Blender : sélectionner le mesh → Mode Édition → U → **Smart UV Project** ou **Unwrap**. Observer dans l'UV Editor :

- Les seams automatiques créent des îlots.
- Chaque îlot est un groupe de faces dépliées.
- Si on déplace un îlot hors de la zone $[0, 1]^2$, la texture ne s'applique plus correctement (voir Partie 4 — wrapping).

2. Interpolation des UVs par le rasterizer

Comment les UVs arrivent au fragment shader

Les UVs sont des **attributs par sommet** (définis en Session 1). Le vertex shader les passe au fragment shader via un varying (ou out/in). Le rasterizer **interpole** ces UVs à travers chaque triangle — chaque fragment reçoit une UV interpolée, pas l'UV d'un sommet.

$$UV_{\text{fragment}} = \alpha \cdot UV_A + \beta \cdot UV_B + \gamma \cdot UV_C$$

où α, β, γ sont les coordonnées barycentriques du fragment dans le triangle. C'est pourquoi une texture appliquée sur un grand triangle apparaît **continue** — chaque pixel lit un texel différent, interpolé à partir des trois sommets.

Partie 3 : Le Filtrage

! Le problème central

Quand une texture est affichée à l'écran, il y a deux cas extrêmes :

- **Minification** : la texture est plus grande que l'écran (un mur de briques vu de loin — plusieurs texels par pixel).
- **Magnification** : la texture est plus petite que l'écran (un mur vu de très près — plusieurs pixels par texel).

Le **filtrage** détermine comment le sampler choisit la couleur du texel dans ces deux cas. Un mauvais filtrage produit des artefacts visibles : crénelage (aliasing), flou, ou scintillement.

3. Magnification — Nearest vs Bilinear

Nearest neighbor

Le sampler prend le texel le plus proche des UVs. Résultat : blocs nets, look « pixel art ». Rapide mais crénelé.

```
// Équivalent conceptuel
vec2 texelCoord = uv * textureSize;
ivec2 nearest = ivec2(floor(texelCoord + 0.5));
vec3 color = texelFetch(sampler, nearest, 0).rgb;
```

Bilinear filtering

Le sampler prend les 4 texels les plus proches et interpole linéairement entre eux. Résultat : transitions douces, pas de blocs. C'est le filtrage par défaut.

$$\text{color} = \text{lerp}(\text{lerp}(t_{00}, t_{10}, f_x), \text{lerp}(t_{01}, t_{11}, f_x), f_y)$$

où f_x, f_y sont les fractions de position dans la cellule de texels.

4. Minification et Mipmaps

Le problème de la minification

Quand un mur de briques est vu de loin, chaque pixel couvre **des dizaines** de texels. Si le sampler ne lit qu'un seul texel (nearest ou bilinear), il **rate** la plupart des briques — on obtient du **moiré** (scintillement, motifs parasites). C'est l'aliasing de texture.

Mipmaps — la solution

Le GPU génère automatiquement une **pyramide** de versions de la texture de plus en plus petites : 2048→1024→512→256→128→64→32→16→8→4→2→1. Chaque niveau est un quart de la résolution du précédent (moitié en largeur et hauteur).

Le sampler choisit le niveau de mipmap approprié selon la distance à la caméra : de loin, il lit un texel d'une petite mipmap (qui représente déjà la moyenne de plusieurs texels). Plus de moiré.

- **Trilinear filtering** : bilinear sur les deux niveaux de mipmap les plus proches, puis interpolation entre les deux. Transitions douces entre les niveaux.
- **Coût mémoire** : les mipmaps ajoutent $\sim 33\%$ de mémoire (somme d'une série géométrique : $1 + \frac{1}{4} + \frac{1}{16} + \dots \approx \frac{4}{3}$).

💡 Dans Babylon.js / Godot

Les mipmaps sont activés par défaut sur les textures 2D. On peut les désactiver pour des textures d'interface (UI) ou des textures pixel art où l'on **veut** l'effet bloc.

```
// Babylon.js
const tex = new BABYLON.Texture("textures/bricks.jpg", scene);
tex.wrapU = BABYLON.Texture.WRAP_ADDRESSMODE; // voir Partie 4
// Les mipmaps sont générés automatiquement.
```

Partie 4 : Le Wrapping

Que se passe-t-il hors de [0,1] ?

Les UVs peuvent dépasser l'intervalle [0, 1] — par exemple, un sol qui répète une texture de carrelage sur 10×10 mètres avec des UVs allant de 0 à 10. Le mode de **wrapping** détermine ce que fait le sampler :

- **Repeat** : la texture se répète (carrelage, tiling). Le mode le plus utilisé pour les sols, murs, etc.
- **Clamp** : les UVs hors [0, 1] sont écrêtées sur les bords. La texture s'étire sur les bords. Utilisé pour les textures uniques (un personnage, un logo).
- **Mirror** : la texture se reflète à chaque répétition. Utile pour cacher les seams de tiling.

Wrapping en Babylon.js.

```
const floorTex = new BABYLON.Texture("textures/tiles.jpg", scene);
floorTex.wrapU = BABYLON.Texture.WRAP_ADDRESSMODE; // repeat
floorTex.wrapV = BABYLON.Texture.WRAP_ADDRESSMODE;
floorTex.uScale = 10; // répéter 10 fois en U
floorTex.vScale = 10; // répéter 10 fois en V

// Pour une texture unique (personnage) :
const skinTex = new BABYLON.Texture("textures/skin.jpg", scene);
skinTex.wrapU = BABYLON.Texture.CLAMP_ADDRESSMODE;
skinTex.wrapV = BABYLON.Texture.CLAMP_ADDRESSMODE;
```

Partie 5 : Les Formats de Texture

Deux grandes catégories

- **Formats non compressés GPU** : PNG, JPEG, TIFF. Le GPU doit les **décompresser** à l'envoi, puis stocke les pixels bruts en VRAM. Simple mais gourmand en mémoire : une texture 2048×2048 RGBA = $2048 \times 2048 \times 4 = 16$ « Mo ».
- **Formats compressés GPU** : DDS (DirectX), KTX (Khronos), avec des algorithmes de compression **bloc** comme BCn (Desktop), ETC (Mobile), ASTC (Mobile moderne). Le GPU lit les

blocs compressés **directement** et les décompresse à la volée dans le shader. Ratio de compression $\sim 4 : 1$ à $\sim 8 : 1$ sans coût de décompression.

💡 Quand utiliser quoi ?

- **Prototypage / démo** : PNG ou JPEG — simple, universel.
- **Production desktop** : BCn (via DDS ou KTX). BC1 pour l'albedo (1 bit alpha), BC5 pour les normal maps (2 canaux, haute précision), BC7 pour les textures avec alpha.
- **Production mobile** : ASTC (qualité réglable, ratio excellent).
- **HDR / lightmaps** : EXR ou HDR (valeurs > 1.0 , vu en Session 6).

Partie 6 : Le Set de Textures PBR

💡 Une texture ne suffit pas

En Session 5, on a vu que PBRMaterial de Babylon.js utilise albedoColor, metallic, roughness — des valeurs **uniformes** (une couleur pour tout l'objet). Mais un vrai matériau n'est pas uniforme : une planche de bois a des fibres, des nœuds, des variations de brillance. C'est pourquoi on utilise **plusieurs textures** — un **set PBR** — pour contrôler chaque propriété **par texel**.

Les maps essentielles d'un set PBR

Map	Canaux	Rôle
Albedo / Base Color	RGB	La couleur de base du matériau, sans aucune information d'éclairage. C'est ce que l'objet aurait sous une lumière blanche uniforme.
Normal Map	RGB	Perturbe la normale de surface par texel pour simuler du relief sans géométrie. Détail en Partie 7.
Roughness	1 (grayscale)	Contrôle la rugosité de la surface : 0 = miroir parfait (spéculaire net), 1 = mate (spéculaire diffus). Varie par texel : une planche usée est plus rugueuse que sa vernie.
Metallic	1 (grayscale)	Indique si le texel est un métal (1.0) ou un diélectrique (0.0). Les métaux réfléchissent teintés par leur albedo ; les diélectriques réfléchissent blanc. Généralement binaire, parfois lissé.
Ambient Occlusion	1 (grayscale)	Assombrit les crevasses et zones occluses. Vu en Session 6 — souvent baked depuis la géométrie high-poly.
Height / Displacement	1 (grayscale)	Carte de hauteur pour le parallax mapping (Session 8) ou le déplacement réel (déplacement des sommets).

Tableau 3. – Un set PBR complet : 6 maps qui contrôlent chaque propriété du matériau indépendamment, par texel.

🔗 Démo Blender — le set de briques

Dans Blender, ouvrir Blender/assets/textures/StoneBricksSplitface001/. Le set contient :

- StoneBricksSplitface001_COL_2K.jpg — Albedo
- StoneBricksSplitface001_NRM_2K.jpg — Normal
- StoneBricksSplitface001_REFL_2K.jpg — Réflectivité (inverse de roughness)
- StoneBricksSplitface001_GLOSS_2K.jpg — Glossiness
- StoneBricksSplitface001_AO_2K.jpg — Ambient Occlusion
- StoneBricksSplitface001_DISP_2K.jpg — Displacement

Dans le **Shader Editor** de Blender, brancher chaque map dans le noeud **Principled BSDF** correspondant. Observer comment chaque map affecte le rendu indépendamment.

Partie 7 : Les Normal Maps en Détail

Le principe

Une normal map est une texture RGB où chaque texel stocke un **vecteur normal perturbé** (n_x, n_y, n_z) . Au lieu d'utiliser la normale géométrique de la surface (interpolée depuis les sommets), le fragment shader **lit** la normale dans la texture et l'utilise pour le calcul d'éclairage.

Résultat : les fibres d'une planche, les écailles d'un reptile, les briques d'un mur semblent en **relief** — sans ajouter un seul triangle. C'est l'illusion la plus rentable en infographie.

❗ Pourquoi les normal maps sont bleues ?

Les normales sont stockées dans l'**espace tangent** — un repère local à chaque surface, défini par la tangente, la binormale et la normale géométrique (vus en Session 1). Dans ce repère :

- n_x (rouge) : déviation gauche/droite
- n_y (vert) : déviation avant/arrière
- n_z (bleu) : profondeur (normale « vers l'extérieur »)

Une surface **plate** a une normale $(0, 0, 1)$ — donc $R = 0, G = 0, B = 1$ — **bleu**. Les normal maps sont donc dominées par des tons bleus. Une zone rouge penche à droite, une zone verte penche vers l'avant.

Normal mapping dans un fragment shader (GLSL).

```
uniform sampler2D uNormalMap;
in vec2 vUV;
in vec3 vNormal;      // normale géométrique (interpolée)
in vec3 vTangent;     // tangente (interpolée)
in vec3 vBitangent;   // binormale (interpolée)

void main() {
    // 1. Lire la normale perturbée dans la texture ( [0,1] -> [-1,1] )
    vec3 n = texture(uNormalMap, vUV).rgb * 2.0 - 1.0;

    // 2. Construire la matrice TBN (tangent -> world space)
    mat3 TBN = mat3(normalize(vTangent), normalize(vBitangent), normalize(vNormal));

    // 3. Transformer la normale de l'espace tangent vers l'espace monde
    vec3 worldNormal = normalize(TBN * n);

    // 4. Utiliser worldNormal pour l'éclairage (Lambert, etc.)
    float diff = max(dot(worldNormal, lightDir), 0.0);
    fragColor = vec4(albedo * diff, 1.0);
}
```

La matrice **TBN** (Tangent, Bitangent, Normal) convertit du repère local de la surface vers le repère monde — c'est le pont entre la normale stockée dans la texture et la normale utilisée pour l'éclairage.

Partie 8 : Bump, Normal et Displacement — Les Trois Approches du Relief

! Le problème commun

Une surface 3D a une géométrie **limitée** — un mur de briques modélisé avec 10 triangles est plat, même si les vraies briques ont des joints, des bosses, des éclats. On veut ajouter du relief **sans** ajouter de polygones. Il existe trois techniques, du moins coûteux au plus réaliste, qui font toutes la même chose — **simuler du relief** — mais avec des compromis différents.

1. Bump Map — l'ancêtre

Principe

Une **bump map** est une texture **grayscale** (niveaux de gris) où chaque texel stocke une **hauteur** relative : blanc = haut, noir = bas. Le fragment shader calcule une normale perturbée **à partir de cette hauteur** en estimant les dérivées partielles :

$$n_x \approx \frac{h(u + \varepsilon, v) - h(u - \varepsilon, v)}{2\varepsilon}$$

$$n_y \approx \frac{h(u, v + \varepsilon) - h(u, v - \varepsilon)}{2\varepsilon}$$

$$n_z = 1$$

(normalisé ensuite)

En pratique, le shader lit 3 ou 4 texels voisins dans la bump map et calcule le **gradient** — la pente locale. Cette pente devient la normale perturbée.

💡 Bump vs Normal — quelle différence ?

Une **bump map** et une **normal map** produisent **exactement le même effet** : une normale perturbée dans le fragment shader. La différence est le **format** :

- **Bump map** : grayscale (1 canal), hauteur. Le shader calcule la normale à la volée.
- **Normal map** : RGB (3 canaux), normale pré-calculée. Le shader lit directement la normale.

Les normal maps ont **remplacé** les bump maps dans les pipelines modernes : la normale est pré-calculée (par Blender, Substance Painter, etc.) avec plus de précision qu'un gradient estimé dans le shader, et stockée dans 3 canaux au lieu d'1. Mais le résultat visuel est le même — l'une est juste plus précise que l'autre.

En résumé : une normal map est une bump map **pré-calculée et plus précise**. On n'utilise presque plus les bump maps aujourd'hui.

2. Displacement Map — le vrai relief

Principe

Une **displacement map** (ou **height map**) est aussi une texture grayscale de hauteur — **comme un bump map**. Mais au lieu de truquer la normale dans le **fragment shader**, on déplace **réellement les sommets** de la géométrie dans le **vertex shader** :

$$\text{position}_{\text{déplacée}} = \text{position}_{\text{originale}} + \text{normale} \cdot h(\text{UV}) \cdot k$$

où $h(\text{UV})$ est la hauteur lue dans la texture et k un facteur d'intensité. La surface est **physiquement** déformée — le relief est vrai, visible de profil, et la silhouette est correcte.

! La différence fondamentale : vertex vs fragment shader

- **Bump / Normal map** : agissent dans le **fragment shader**. La géométrie est intacte — seules les normales utilisées pour l'éclairage changent. La surface reste plate en réalité.
- **Displacement map** : agit dans le **vertex shader**. La géométrie est **réellement modifiée** — les sommets bougent. La surface est vraiment déformée.

C'est pourquoi le déplacement nécessite **beaucoup** de polygones : si un triangle est grand, le vertex shader ne peut le déplacer qu'à ses 3 sommets — le centre du triangle reste plat (interpolation linéaire). Pour un relief fin, il faut **subdiviser** le mesh en milliers de petits triangles.

Displacement dans un vertex shader (GLSL).

```
uniform sampler2D uHeightMap;
uniform float uDisplacementScale;
in vec3 position;    // position du sommet (object space)
in vec3 normal;      // normale du sommet
in vec2 uv;          // UV du sommet

void main() {
    // 1. Lire la hauteur dans la texture
    float h = texture(uHeightMap, uv).r;    // [0,1]

    // 2. Déplacer le sommet le long de sa normale
    vec3 displaced = position + normal * (h * uDisplacementScale);

    // 3. Transformer en clip space comme d'habitude
    gl_Position = projection * view * model * vec4(displaced, 1.0);
}
```

Le vertex shader lit la height map **par sommet** — pas par fragment. C'est beaucoup moins de lectures de texture (un mesh a des milliers de sommets, mais des millions de fragments), mais chaque sommet ne peut être déplacé qu'une fois.

⚠ Le problème de la résolution géométrique

Si le mesh a trop peu de polygones, le déplacement est **invisible** ou **imprécis** :

- Un plan de 2 triangles déplacés par une height map de briques ne montrera que 2 sommets déplacés — le reste est interpolé. Le relief est inexistant.

- Solution : **subdiviser** le mesh avant le déplacement. Les moteurs modernes font ça dynamiquement avec la **tessellation** (GPU) — le GPU subdivise chaque triangle en plus petits triangles à la volée, puis le vertex shader les déplace.

La tessellation est avancée et hors programme, mais le concept est important : **le déplacement n'a de sens que si la géométrie est assez dense.**

3. Comparaison des trois techniques

Critère	Bump Map	Normal Map	Displacement Map
Shader	Fragment	Fragment	Vertex
Format	Grayscale (1 ch.)	RGB (3 ch.)	Grayscale (1 ch.)
Géométrie modifiée ?	Non	Non	Oui
Silhouette correcte ?	Non	Non	Oui
Coût GPU	Faible	Faible	Élevé (subdivision)“
Polygones requis	Quelconque	Quelconque	Beaucoup
Précision du relief	Moyenne (gradient estimé)	Haute (normale pré-calculée)	Parfaite (vrai relief)
Usage aujourd'hui	Obsolète	Standard	Cinématique / tessellation’

Tableau 4. – Les trois techniques de relief. La normal map est le standard temps réel — le meilleur compromis qualité/coût. Le déplacement est réservé aux cas où la silhouette doit être correcte (cinématique, rendu offline) ou avec la tessellation GPU.

💡 Dans Blender — les trois dans le Shader Editor

Dans le **Shader Editor** de Blender, le noeud **Principled BSDF** propose trois entrées :

- **Normal** : brancher une **Image Texture** + **Normal Map** node (normal map standard).
- **Bump** : brancher une **Image Texture** + **Bump** node (bump map — Blender convertit la hauteur en normale).
- **Displacement** : brancher une **Image Texture** + **Displacement** node. Nécessite que le mesh soit **subdivisé** (modifier **Subdivision Surface**) et que le matériel soit en mode **Displacement and Bump** (pas **Bump only**).

Observer : avec **Displacement**, la silhouette du mesh change réellement — les bords sont dentelés. Avec **Normal** seul, les bords restent droits.

💡 Et le parallax mapping ? (Session 8)

Il existe une **quatrième** technique : le **parallax mapping**, vu en Session 8. C'est un compromis entre normal map et déplacement : la géométrie reste plate, mais le fragment shader **décale les**

UVs selon l'angle de vue pour simuler la parallaxe du relief. Moins précis que le déplacement, mais beaucoup moins coûteux — et meilleur que la normal map seule pour la perception de profondeur.

Partie 9 : Export depuis Blender

Le format GLB — textures embarquées

Pour transférer un modèle texturé de Blender vers un moteur web (Babylon.js, Three.js), on exporte en **GLB** (GLTF binaire) — un format qui embarque la géométrie, les UVs, les textures et les matériaux dans un seul fichier.

Dans Blender : **File** → **Export** → **glTF 2.0 (.glb)**. Les options importantes :

- **Format** : glb (binaire, un fichier) ou gltf + textures séparées.
- **Materials** : cocher **Export materials** pour embarquer les textures PBR.
- **UVs** : coché par défaut — sans UVs, les textures ne s'appliquent pas.

🔗 Vérifier l'export

Avant d'importer dans le moteur, vérifier que le GLB contient bien les textures :

- Ouvrir le GLB dans un viewer (ex: gltf-viewer) — les textures PBR doivent s'afficher.
- Si le modèle apparaît gris, les textures n'ont pas été embarquées — revoir les options d'export.

Partie 10 : TP — Set PBR Complet dans Babylon.js

🔗 Objectif du TP

Importer un modèle texturé (exporté de Blender ou téléchargé) dans Babylon.js, et reproduire le branchement du set PBR dans le **Node Material Editor** — comme on l'a vu dans le Shader Editor de Blender, mais côté moteur.

Mise en place

- Télécharger un set PBR sur ambientCG (gratuit, CCo) — par exemple **StoneBricks** ou **WoodPlanks**.

Vous pouvez aussi utiliser les assets fournis dans le cours.

- Ouvrir le **Node Material Editor** de Babylon.js : nme.babylonjs.com.
- Créer un material avec un **PBR Metallic Roughness** block.
- Vous pouvez regarder un setup ici: [PBR setup example](#)

Missions

Mission 1 — Albedo

Ajouter un **Texture** block, charger la map `_COL_2K.jpg`, la brancher sur l'entrée `albedo` du PBR block. Observer la couleur de base.

Mission 2 — Normal Map

Ajouter un second **Texture** block pour `_NRM_2K.jpg`, la brancher sur `normal`. Observer le relief apparent — les briques semblent en 3D. Tourner la caméra de profil pour voir l'illusion se briser.

Mission 3 — Roughness

Brancher la map `_GLOSS_2K.jpg` (ou `_ROUGH_2K.jpg` selon le set) sur `roughness`. Observer : les zones vernies sont brillantes (spéculaire net), les zones poreuses sont mates.

Mission 4 — Ambient Occlusion

Brancher `_AO_2K.jpg` sur l'entrée `ambient occlusion` (ou la multiplier avec l'albedo dans un **Multiply** block). Observer comment les crevasses s'assombrissent.

Mission 5 — Comparer

Désactiver chaque map une par une et observer la dégradation. Sans normal map : la surface paraît plate. Sans roughness : tout est uniformément brillant. Sans AO : les crevasses manquent de profondeur.

Session 8 : Textures Avancées

💡 Objectifs de la session

Aller au-delà du chargement d'une image : générer des textures **dans le shader** (procédurales), rendre la scène **dans une texture** (render targets), simuler la profondeur avec des **parallax maps**, et se passer d'UVs avec le **triplanar mapping**. La session suit le même principe que la Session 7 : **créer dans Blender, intégrer dans Babylon.js / Node Material Editor**.

💡 Rappel Session 7

Une texture est une image sur le GPU, échantillonnée par le fragment shader via des UVs. On a vu le set PBR (albedo, normal, roughness, metallic, AO) et le filtrage (mipmaps, bilinear). Cette session explore ce qu'on peut faire **de plus** — quand une image fichier ne suffit pas, ou quand on n'a pas d'UVs.

Partie 1 : Textures Procédurales

Principe

Une texture procédurale est **calculée** dans le shader à partir des UVs (ou de la position) — pas lue dans un fichier image. Le shader devient la texture. Avantages :

- **Pas de mémoire** : zéro VRAM, quelle que soit la « résolution ».
- **Résolution infinie** : pas de texel, pas de flou de près.
- **Paramétrable** : on change les paramètres du bruit, pas besoin de refaire une texture.

Inconvénient :

- **Coût shader** : chaque fragment exécute le calcul — un bruit complexe peut coûter plus qu'une lecture de texture.

1. Le bruit — la brique de base

💡 Pourquoi pas juste `random()` ?

Un générateur aléatoire pur (`rand()`) produit du **bruit blanc** : chaque pixel est indépendant de ses voisins. Visuellement, c'est de la neige TV — inutilisable pour une texture. Ce qu'on veut, c'est du bruit **cohérent** : des valeurs qui varient **progressivement** d'un pixel à l'autre, avec du détail à plusieurs échelles. C'est ce que font les fonctions de bruit procédural.

Le principe commun

Toutes les fonctions de bruit procédural partagent la même structure :

1. Une **grille** de points aléatoires (déterministes — même input → même output).
2. Une **interpolation** entre les points de la grille pour produire une valeur continue.

La différence entre les types de bruit est : **qu'est-ce qu'on interpole** et **comment**.

2. La fonction hash — le cœur du bruit

Hash déterministe

Un hash est une fonction qui prend un input (entier, vecteur) et retourne une valeur pseudo-aléatoire dans $[0, 1]$. **Déterministe** : le même input donne toujours le même output. C'est l'équivalent d'une table de nombres aléatoires pré-calculés, mais sans stockage.

```
// Hash 2D → 1 valeur [0,1]
float hash21(vec2 p) {
    vec3 p3 = fract(vec3(p.xy * 0.1031));
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.x + p3.y) * p3.z);
}

// Hash 2D → 2 valeurs [0,1] (pour les gradients)
vec2 hash22(vec2 p) {
    vec3 p3 = fract(vec3(p.xy * vec3(0.1031, 0.1030, 0.0973)));
    p3 += dot(p3, p3.yzx + 33.33);
    return fract((p3.xx + p3.yz) * p3.zy);
}
```

Les constantes magiques (0.1031, 33.33) sont empiriques — elles garantissent une bonne distribution pseudo-aléatoire. On n'invente pas les siennes ; on utilise des hash connus et testés.

🧐 Quel hash pour quel usage ?

- **Value noise** : n'importe quel hash (IQ, sin) suffit — on n'a besoin que d'une valeur déterministe.
- **Perlin / gradient noise** : le hash doit **décorréliser** les cellules voisines, sinon on voit la grille. Les hashes simples (IQ, sin) ne le font pas assez bien — on utilise une **table de permutation** (mod289 + permute), voir section 4.

3. Value Noise — le plus simple

Principe

On place une valeur aléatoire $[0, 1]$ à chaque point d'une grille entière. Pour un pixel entre les points, on interpole (bilinear + smoothstep) entre les 4 valeurs voisines.

$$\text{value}(p) = \text{interp}(\#(\text{floor}(p)), \#(\text{floor}(p) + (1, 0)), \#(\text{floor}(p) + (0, 1)), \#(\text{floor}(p) + (1, 1)))$$

Value noise en GLSL.

```
float valueNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    // Smoothstep : adoucit les transitions (sinon bilinear = dur)
    f = f * f * (3.0 - 2.0 * f);

    float a = hash21(i);
    float b = hash21(i + vec2(1.0, 0.0));
    float c = hash21(i + vec2(0.0, 1.0));
```

```

float d = hash21(i + vec2(1.0, 1.0));

return mix(mix(a, b, f.x), mix(c, d, f.x), f.y);
}
    
```

⚠ Limites du value noise

Le value noise est **bloqueux** : on voit les cellules de la grille, surtout à basse fréquence. Les transitions sont en C^0 (continues mais pas dérivables) — la pente change brusquement aux bordures de cellules. Acceptable pour des effets simples, mais pas pour du réalisme.

4. Perlin Noise (gradient noise) — le standard

Principe

Inventé par **Ken Perlin** en 1985 (Oscar technique 1997). Au lieu d'interpoler des **valeurs**, on interpole des **gradients** — des vecteurs de direction aléatoires à chaque point de la grille. Pour un pixel, on calcule le produit scalaire entre le gradient et le vecteur du point de grille vers le pixel.

$$\text{perlin}(p) = \text{interp}(\text{dot}(g_A, p - A), \text{dot}(g_B, p - B), \text{dot}(g_C, p - C), \text{dot}(g_D, p - D))$$

où g_A, g_B, g_C, g_D sont les gradients aléatoires aux 4 coins de la cellule.

💬 Pourquoi Perlin > Value

Le Perlin noise est **C^1 -continu** : la dérivée est continue, donc pas de « cassure » visible. Les transitions sont douces et naturelles. C'est pourquoi c'est le bruit standard en infographie — tous les moteurs (Blender, Unity, Unreal, Substance) l'utilisent par défaut.

Perlin noise en GLSL (version pédagogique).

```

float perlinNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);
    // Quintic smoothstep : C2-continu (encore plus doux que C1)
    f = f * f * f * f * (f * (f * 6.0 - 15.0) + 10.0);

    // Gradients aléatoires aux 4 coins – ATTENTION : il faut
    // (1) remapper [0,1] → [-1,1]   (2) normaliser en vecteur unitaire
    // Sinon : tous les gradients pointent vers le même quadrant → artefacts.
    vec2 ga = normalize(hash22(i) * 2.0 - 1.0);
    vec2 gb = normalize(hash22(i + vec2(1.0, 0.0)) * 2.0 - 1.0);
    vec2 gc = normalize(hash22(i + vec2(0.0, 1.0)) * 2.0 - 1.0);
    vec2 gd = normalize(hash22(i + vec2(1.0, 1.0)) * 2.0 - 1.0);

    // Produits scalaires : gradient · vecteur vers le pixel
    float va = dot(ga, f - vec2(0.0, 0.0));
    float vb = dot(gb, f - vec2(1.0, 0.0));
    float vc = dot(gc, f - vec2(0.0, 1.0));
    float vd = dot(gd, f - vec2(1.0, 1.0));
    
```

```

    return mix(mix(va, vb, f.x), mix(vc, vd, f.x), f.y) * 0.5 + 0.5;
}

```

Le $* 0.5 + 0.5$ à la fin remappe le résultat de $[-1, 1]$ vers $[0, 1]$ (les produits scalaires peuvent être négatifs).

⚠ Cette version pédagogique a un défaut

Même avec des gradients normalisés, un hash simple (sin, IQ) produit des **lignes de grille** visibles à basse fréquence : les cellules voisines ont des gradients trop corrélés. C'est suffisant pour comprendre le principe, mais pas pour un rendu de production.

💡 La solution : table de permutation

Les implémentations réelles (Blender, Unity, la démo [session08_noise.html](#)) utilisent une **table de permutation** — un hash basé sur $\text{mod}289 + \text{permute}$ qui décorrèle vraiment les cellules voisines. C'est l'implémentation de **Stefan Gustavson / Ashima Arts** (2005), devenue le standard GLSL. Le code est plus long mais le résultat est un vrai bruit de type nuage, sans artefacts. Voir [examples/session08_noise.js](#) pour le code complet.

5. Simplex Noise — l'évolution

Principe

Inventé par **Ken Perlin** en 2001 pour corriger les défauts du Perlin original. Au lieu d'une grille **carrée** (4 coins en 2D), on utilise une grille **triangulaire** (3 coins en 2D). Avantages :

- **Moins d'artefacts directionnels** : le Perlin a des « lignes » visibles le long des axes de la grille. Le Simplex n'en a pas.
- **Plus rapide en N dimensions** : le Perlin nécessite 2^N coins ; le Simplex n'en nécessite que $N + 1$. En 3D : 8 coins (Perlin) vs 4 (Simplex). En 4D : 16 vs 5.
- **Meilleure isotropie** : le bruit a la même apparence dans toutes les directions.

💡 Quand utiliser Simplex vs Perlin ?

- **Perlin** : plus simple à comprendre et implémenter. Suffisant pour la plupart des effets 2D.
- **Simplex** : préférable en 3D et 4D (volumetric, textures solides), ou quand on veut éviter les artefacts directionnels. L'implémentation est plus complexe (skewing de la grille triangulaire).

6. Worley Noise (cellular) — les cellules

Principe

Inventé par **Steven Worley** en 1996. Au lieu d'interpoler, on calcule la **distance** au point aléatoire le plus proche dans la grille. Résultat : un motif de **cellules** (type Voronoi) — chaque cellule contient un point, et la valeur est la distance au point de sa cellule.

$$\text{worley}(p) = \min_{\text{voisins}} \text{dist}(p, \text{point}_{\text{aléatoire}})$$

Worley noise en GLSL.

```
float worleyNoise(vec2 p) {
    vec2 i = floor(p);
    vec2 f = fract(p);

    float minDist = 1.0;

    // Chercher dans les 9 cellules voisines (3x3)
    for (int y = -1; y <= 1; y++) {
        for (int x = -1; x <= 1; x++) {
            vec2 neighbor = vec2(float(x), float(y));
            vec2 point = hash22(i + neighbor);
            vec2 diff = neighbor + point - f;
            float dist = length(diff);
            minDist = min(minDist, dist);
        }
    }
    return minDist;
}
```

💡 Usage du Worley

Le Worley est idéal pour les motifs **cellulaires** : écailles de reptile, bulles, cellules organiques, pierre fracturée, écume. On peut aussi utiliser la distance au **deuxième** point le plus proche (F2-F1) pour produire des bordures nettes entre cellules.

7. FBM (Fractal Brownian Motion) — le détail à toutes les échelles

💡 Le problème du bruit simple

Un bruit seul (value, Perlin, simplex) a une **fréquence unique** — il est lisse ou détaillé, mais pas les deux. La nature a du détail à **toutes les échelles** : une montagne a des pics, des ravines, des rochers, des graviers, du sable. Une seule couche de bruit ne peut pas reproduire ça.

Principe du FBM

On superpose plusieurs **octaves** de bruit à des fréquences croissantes et amplitudes décroissantes :

$$\text{fbm}(p) = \sum_{i=0}^{N-1} a_i \cdot \text{noise}(f_i \cdot p)$$

avec :

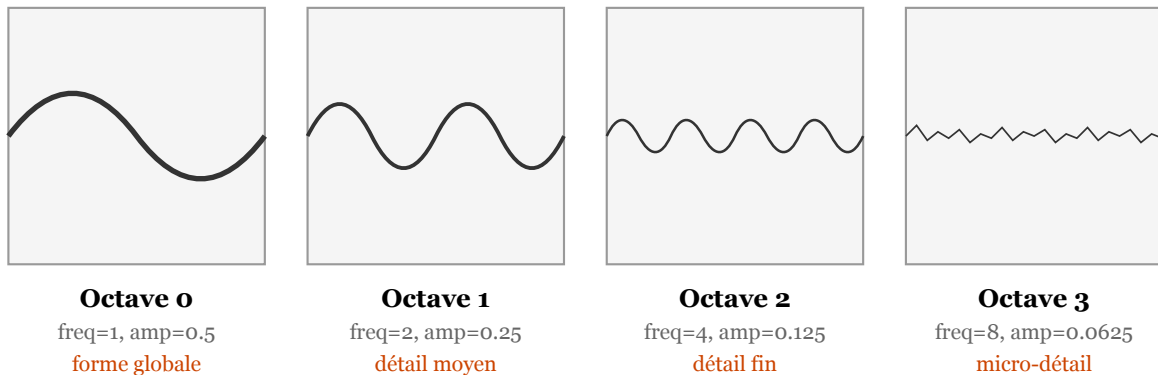
- f_i = lacunaritéⁱ (fréquence — typiquement lacunarité = 2)
- a_i = persistanceⁱ (amplitude — typiquement persistance = 0.5)

Chaque octave ajoute du détail **plus fin** mais avec **moins d'amplitude**. Le résultat a du détail à toutes les échelles — comme la nature.

FBM en GLSL.

```
float fbm(vec2 p) {
    float value = 0.0;
    float amplitude = 0.5;
    float frequency = 1.0;

    for (int i = 0; i < 5; i++) {
        value += amplitude * perlinNoise(p * frequency);
        frequency *= 2.0;    // lacunarité : double la fréquence
        amplitude *= 0.5;    // persistance : divise l'amplitude par 2
    }
    return value;
}
```



Σ = FBM : détail à toutes les échelles

Fig. 21. – Les 4 octaves du FBM. Chaque octave double la fréquence (détail plus fin) et divise l'amplitude par 2 (contribution plus faible). La somme produit du détail à toutes les échelles.

Les deux paramètres clés

- **Lacunarité** (L) : facteur multiplicatif de la fréquence entre octaves. $L = 2$ (défaut) : chaque octave a le double de détail. $L > 2$: détail plus dense, plus « chaotique ». $L < 2$: détail plus espacé, plus « doux ».
- **Persistance** (P) : facteur multiplicatif de l'amplitude entre octaves. $P = 0.5$ (défaut) : chaque octave contribue moitié moins. $P > 0.5$: les hautes fréquences sont plus visibles (terrain rocheux). $P < 0.5$: les hautes fréquences sont atténuées (terrain doux).

8. Comparaison visuelle

Bruit	Continuité	Coût	Usage
Value	C^0 (bloqueux)	Faible	Prototypage, effets simples
Perlin	C^1 (doux)	Moyen	Standard : terrain, nuages, marbre
Simplex	C^1 (isotrope)	Moyen	3D/4D, pas d'artefacts directionnels
Worley	Discontinu	Moyen	Cellules : écailles, bulles, pierre
FBM	Dépend du bruit de base	$\times N$ octaves	Terrain réaliste, textures naturelles

Tableau 5. – Les 5 types de bruit. Le FBM n'est pas un bruit indépendant — c'est une **combinaison** d'octaves d'un autre bruit (typiquement Perlin).

Démo interactive

Ouvrir `examples/session08_noise.html` : une galerie interactive qui montre les 5 types de bruit sur un plan, avec une GUI pour :

- **Type de bruit** : Value, Perlin, Simplex, Worley, FBM.
- **Fréquence** : densité du bruit (zoom in/out).
- **Contraste / Luminosité** : post-traitement du résultat.
- **Animer** : déplace le bruit dans le temps.
- **Mode terrain** : applique le bruit comme height map avec coloration (eau → plaine → montagne → neige).

9. Applications procédurales

Le principe commun

Toutes ces textures combinent un bruit avec une **fonction de forme** (sinus, seuil, distance) pour produire un motif reconnaissable. Le bruit apporte le **chaos naturel**, la fonction de forme apporte la **structure**.

La démo `examples/session08_textures.html` montre ces 5 textures en temps réel avec une GUI et un bouton d'export PNG.

Marbre

Principe

Des veines sinusoïdales perturbées par du FBM. Le FBM casse la régularité du sinus pour donner des veines naturelles et irrégulières.

Marbre en GLSL.

```
vec3 marble(vec2 uv) {
    float n = fbm(uv * 2.0);
    // Veines : sinus absolu pour des lignes nettes
    float vein = abs(sin((uv.x + n * 1.5) * 3.14159));
    // Couleur : blanc → gris-bleu foncé
    vec3 light = vec3(0.95, 0.95, 0.92);
    vec3 dark  = vec3(0.30, 0.35, 0.45);
    return mix(dark, light, vein);
}
```

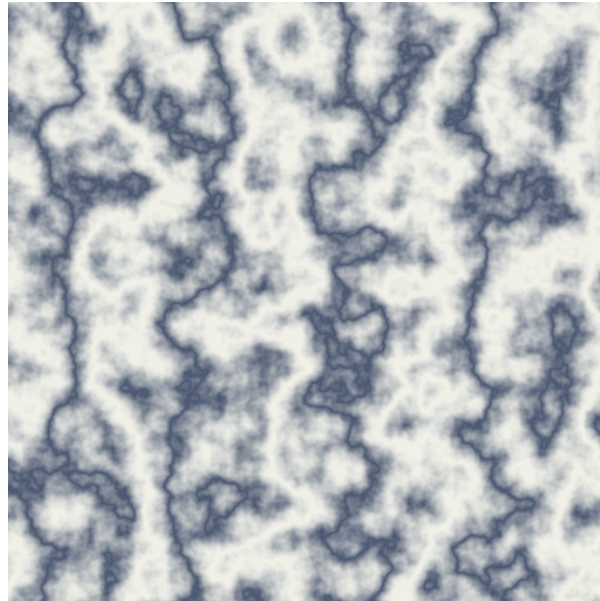


Fig. 22. – Texture de marbre procédurale — FBM + sinus absolu. Les veines sont irrégulières car le FBM perturbe la phase du sinus.

Bois

Principe

Des anneaux concentriques déformés par du bruit. Le $\text{abs}(\sin(\dots))$ crée des anneaux, le bruit les déforme pour qu'ils ne soient pas parfaits.

Bois en GLSL.

```
vec3 wood(vec2 uv) {
    float n = perlinNoise(uv * 3.0);
    // Anneaux : sinus absolu (effet de rings)
    float rings = abs(sin((uv.x * 8.0 + n * 5.0) * 3.14159));
    // Couleur : bois clair → bois foncé
    vec3 light = vec3(0.80, 0.60, 0.35);
    vec3 dark  = vec3(0.40, 0.25, 0.12);
    return mix(dark, light, rings);
}
```

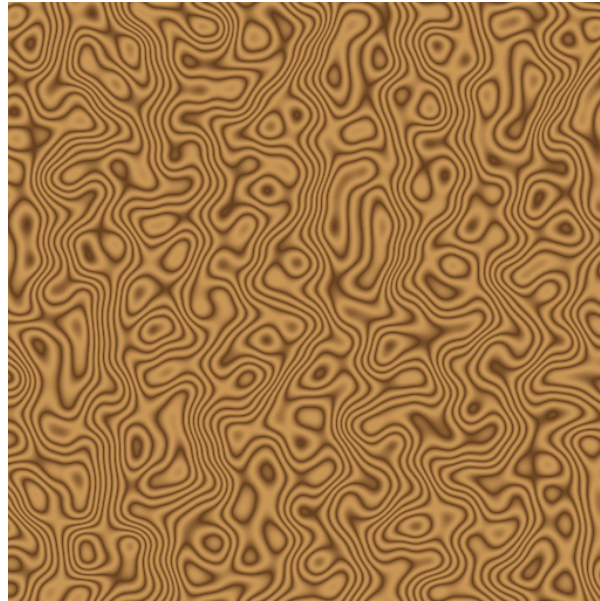


Fig. 23. – Texture de bois procédurale — Perlin + sinus absolu. Les anneaux sont déformés par le bruit, imitant les irrégularités du bois réel.

Dissolve

Principe

Un bruit seuillé — `step(threshold, noise(uv))` — pour faire fondre un objet. Animer le `threshold` de 0 à 1 fait disparaître l'objet progressivement. On ajoute souvent un **halo lumineux** au seuil pour un effet de « brûlure ».

Dissolve en GLSL.

```
vec3 dissolve(vec2 uv) {
    float n = fbm(uv * 3.0);
    float mask = step(uThreshold, n);
    // Bordure lumineuse au seuil (effet "burn")
    float edge = smoothstep(uThreshold - 0.05, uThreshold, n)
        - smoothstep(uThreshold, uThreshold + 0.05, n);
    vec3 base = vec3(0.1, 0.1, 0.15);
    vec3 solid = vec3(0.9, 0.9, 0.95);
    vec3 color = mix(base, solid, mask);
    color += edge * vec3(1.0, 0.5, 0.1); // halo orange
    return color;
}
```

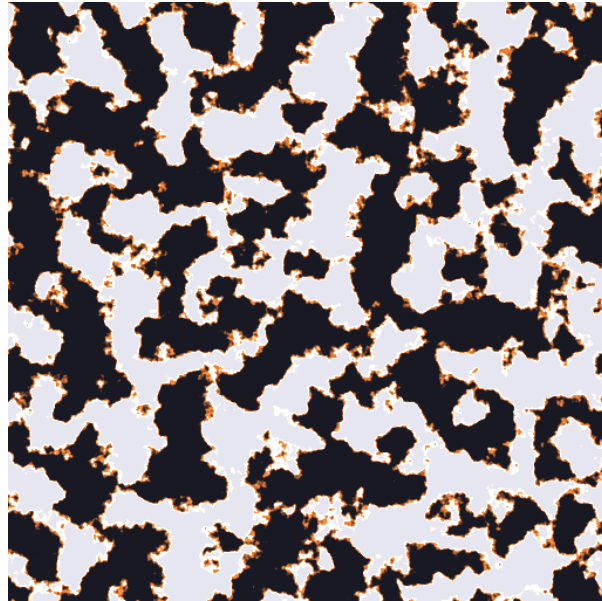


Fig. 24. – Effet dissolve — FBM seuillé avec halo orange au bord. Animer le seuil fait fondre l'objet progressivement, utile pour les transitions et la mort d'ennemis.

Écailles (Worley)

Principe

Le Worley noise avec un seuil crée des cellules séparées par des bordures — parfait pour les écailles de reptile, les cellules organiques, ou la pierre fracturée.

Écailles en GLSL.

```
vec3 scales(vec2 uv) {
    float w = worleyNoise(uv * 4.0);
    // Bordures : inverse du worley, seuillé
    float border = 1.0 - smoothstep(0.0, 0.15, w);
    // Couleur de base : dégradé selon la distance
    vec3 base = mix(vec3(0.2, 0.5, 0.6), vec3(0.4, 0.7, 0.8), w);
    // Bordures sombres
    return mix(base, vec3(0.05, 0.1, 0.15), border);
}
```

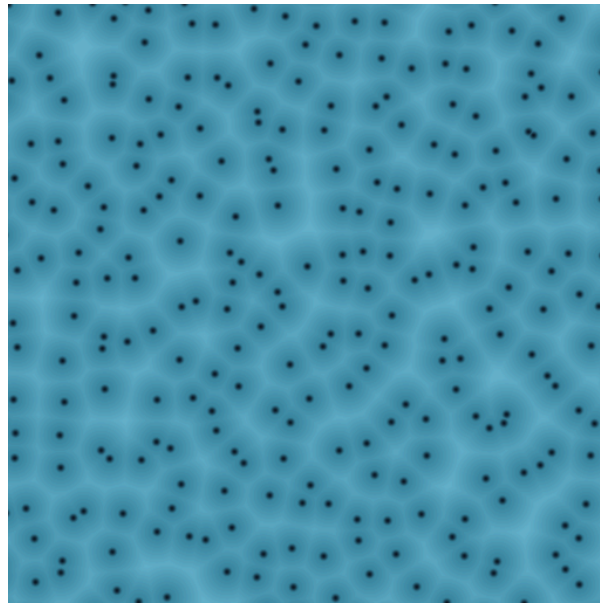


Fig. 25. – Texture d’écailles — Worley noise avec bordures sombres. Chaque cellule correspond à un point de la grille Voronoi. Idéal pour les reptiles, poissons, ou surfaces cellulaires.

Terrain

Principe

Le FBM multi-octaves utilisé comme height map, avec une coloration par altitude. Plus d’octaves = plus de détail (rochers, graviers). C’est la base de tous les terrains procéduraux — de Minecraft à No Man’s Sky.

Terrain en GLSL.

```
vec3 terrain(vec2 uv) {
    float h = fbm(uv * 2.0);
    if (h < 0.45) return vec3(0.15, 0.25, 0.5); // eau profonde
    else if (h < 0.5) return vec3(0.20, 0.35, 0.6); // eau peu profonde
    else if (h < 0.55) return vec3(0.85, 0.80, 0.6); // sable
    else if (h < 0.65) return vec3(0.30, 0.50, 0.2); // plaine
    else if (h < 0.75) return vec3(0.25, 0.40, 0.15); // forêt
    else if (h < 0.85) return vec3(0.40, 0.35, 0.25); // montagne
    else return vec3(0.95, 0.95, 0.98); // neige
}
```

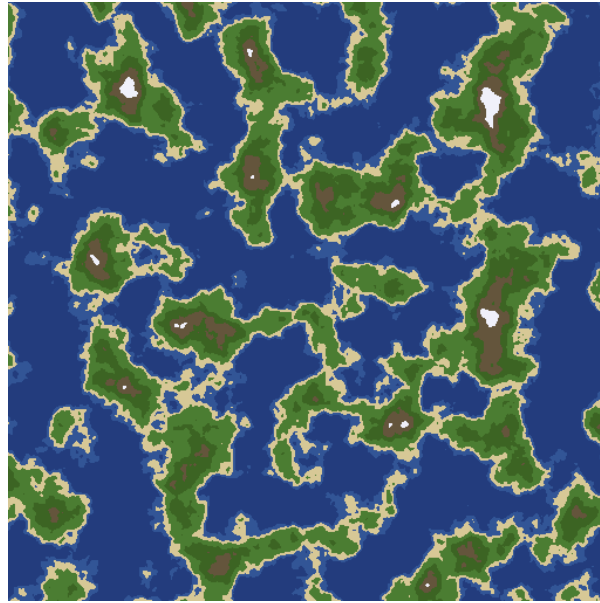


Fig. 26. – Texture de terrain procédurale — FBM avec coloration par altitude. Les seuils définissent les biomes : eau, sable, plaine, forêt, montagne, neige.

Autres effets

Effets avancés

- **Normal map procédurale** : on calcule les dérivées partielles du bruit ($\partial/\partial x$, $\partial/\partial y$) pour obtenir une normale perturbée — un relief procédural sans normal map pré-calculée.
- **Dplacement procédural** : le FBM déplace les sommets dans le vertex shader (voir le « Mode terrain » de la démo `session08_noise.html`).
- **Perturbation de domaine** : au lieu de perturber le **résultat** du bruit, on perturbe les **coordonnées** d'entrée : `noise(uv + noise(uv * 2) * 0.3)`. Crée des swirls et des déformations plus organiques.

Partie 2 : Render Targets

Rendre dans une texture au lieu de l'écran

Un **render target** est une texture dans laquelle le GPU rend la scène au lieu de l'afficher. Une fois la scène rendue, cette texture peut être :

- **Échantillonnée** par un autre shader (post-processing, réflexions).
- **Sauvée** dans un fichier (screenshots, lightmap baking).
- **Réutilisée** comme texture d'un autre objet (miroir, écran de surveillance).

! Le pipeline en deux passes

1. **Passé 1** : rendre la scène (ou une partie) dans le render target.
2. **Passé 2** : rendre la scène finale à l'écran, en échantillonnant le render target là où on en a besoin.

C'est la base de **tous** les effets avancés : post-processing, shadow maps, réflexions, flou de profondeur.

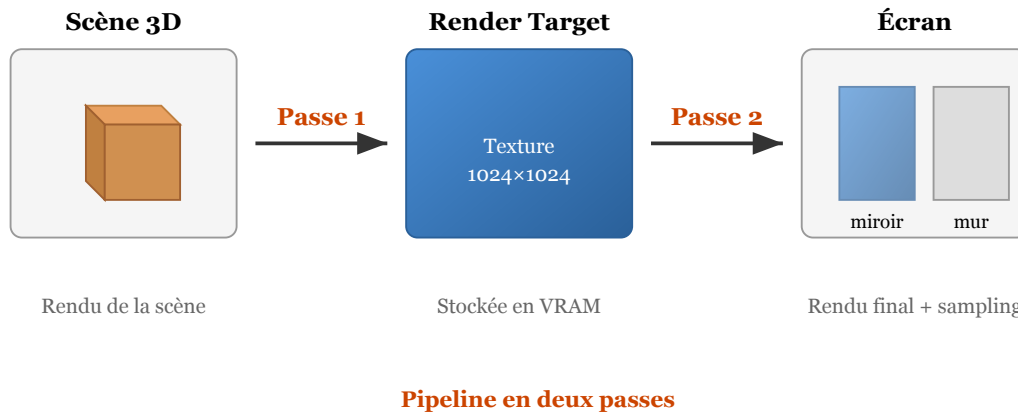


Fig. 27. – Pipeline en deux passes. Passe 1 : la scène est rendue dans une texture (render target) en VRAM. Passe 2 : la scène finale est rendue à l'écran en échantillonnant le render target (ici, pour un miroir).

Render target dans Babylon.js.

```
// 1. Créer un render target de 1024x1024
const rt = new BABYLON.RenderTargetTexture("mirrorRT", 1024, scene);
rt.renderList.push(...scene.meshes); // quels meshes rendre

// 2. L'assigner comme texture d'un matériau (ex: un miroir)
mirrorMat.diffuseTexture = rt;

// 3. Le rendu se fait automatiquement avant la frame principale
```

💡 Cas d'usage

- **Shadow maps** (vu en Session 5) : la scène est rendue du point de vue de la lumière dans un render target de profondeur.
- **Post-processing** (vu en Session 7 originale) : la scène est rendue dans un render target, puis un shader plein écran lit cette texture et applique un effet (bloom, flou, correction de couleur).
- **Réflexions temps réel** : rendre la scène depuis un miroir dans un render target, puis l'appliquer sur la surface du miroir.
- **Portal / écran** : rendre une vue dans un render target et l'afficher sur un mesh (écran de surveillance, tableau magique).

Exemple : miroir d'eau avec vagues

💡 Le cas d'usage classique

Un miroir d'eau combine **render target** et **textures procédurales** :

1. **Passe 1** : la scène est rendue depuis une **caméra de réflexion** (miroir de la caméra principale par rapport au plan d'eau) dans un render target.
2. **Passe 2** : la surface de l'eau échantillonne ce render target avec un **projective texturing**, et deux textures de vague animées en sens opposé déforment les UVs pour simuler les ondulations.

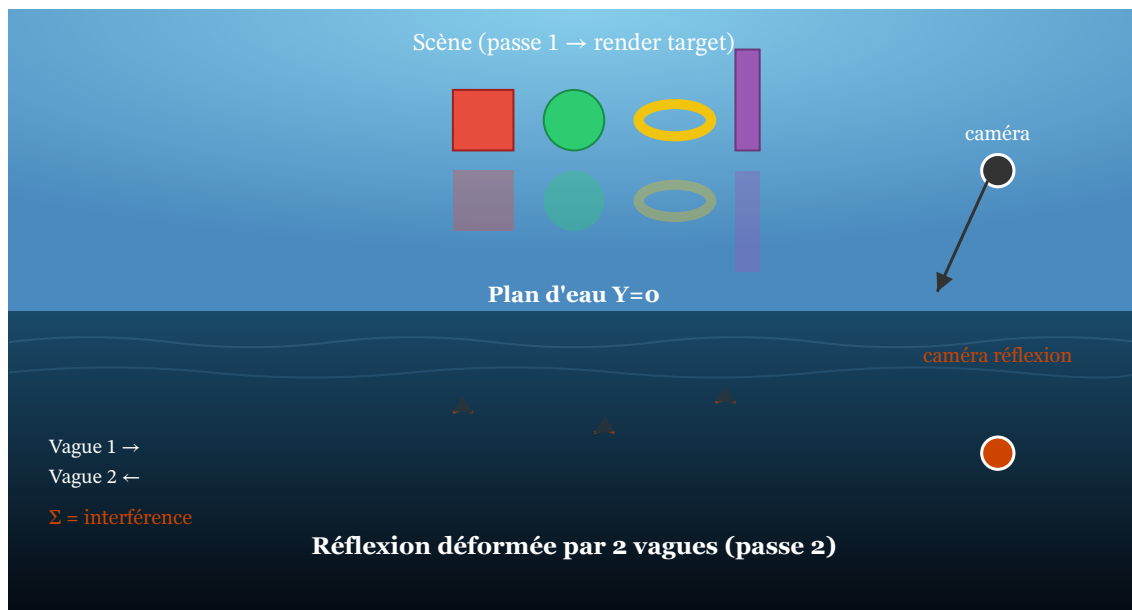


Fig. 28. – Miroir d'eau en deux passes. Passe 1 : la scène est rendue depuis la caméra de réflexion (miroir en Y) dans un render target. Passe 2 : la surface de l'eau échantillonne ce render target par projective texturing, avec deux vagues en sens opposé qui déforment les UVs — l'interférence crée un motif d'ondulation naturel.

Les deux textures de vague

L'effet d'ondulation utilise deux couches de bruit (FBM) qui défilent en **sens opposés** :

- **Vague 1** : `fbm(pos.xz * scale + vec2(time * speed, 0.0))` — défile vers $+x$.
- **Vague 2** : `fbm(pos.xz * scale * 1.3 - vec2(time * speed * 0.7, time * speed * 0.5))` — défile vers $-x$ et $-z$.

La **somme** des deux crée un motif d'**interférence** — comme deux vagues qui se croisent sur l'eau. Une seule couche donnerait un mouvement trop régulier ; deux couches en sens opposés donnent un motif chaotique naturel.

Projective texturing en GLSL.

```
// Transformer la position world-space en UVs du render target
vec4 projected = uReflectionVP * vec4(vWorldPos, 1.0);
vec3 projCoord = projected.xyz / projected.w; // NDC [-1, 1]
vec2 reflectUv = projCoord.xy * 0.5 + 0.5; // remap [0, 1]

// Distortion par les vagues (normale perturbée)
reflectUv += normal.xz * uDistortion;
reflectUv = clamp(reflectUv, 0.0, 1.0);

vec3 reflection = texture(uReflection, reflectUv).rgb;
```

`uReflectionVP` est la matrice view-projection de la **caméra de réflexion** — pas la caméra principale. C'est ce qui fait correspondre chaque fragment de l'eau au bon texel du render target.

💡 Démo interactive

Ouvrir `exemples/session08_water.html` : un miroir d'eau temps réel avec objets colorés qui se reflètent. La GUI contrôle la vitesse, fréquence et amplitude des vagues, la distortion de la réflexion, et la puissance du Fresnel. Observer :

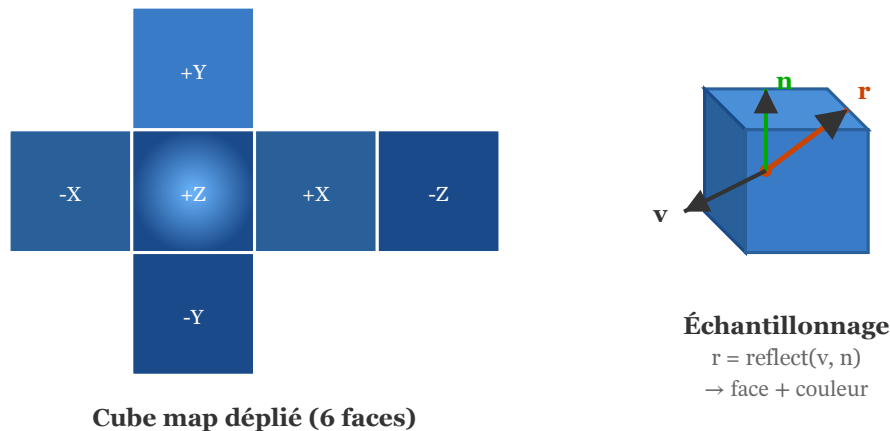
- De **biais** : la réflexion domine (Fresnel élevé).
- De **haut** : on voit la couleur de l'eau (Fresnel faible).
- Les **crêtes** des vagues sont plus claires, les **creux** plus sombres.
- Le **soleil** crée des glints spéculaires sur les crêtes.

Partie 3 : Cube Maps et Environment Maps

Le cube map

Un **cube map** est une texture 3D composée de 6 faces d'un cube (comme une boîte dépliée). On l'échantillonne avec une **direction 3D** $\vec{r}$ (pas des UVs 2D) — le sampler trouve la face correspondante et le texel dans cette face.

C'est la structure idéale pour représenter l'environnement **autour** d'un point : la direction $\vec{r}$ « regarde » dans une direction et obtient la couleur de l'environnement dans cette direction.



Cube map : 6 faces échantillonnées par direction 3D

Fig. 29. – Un cube map est composé de 6 faces (dépliées en croix). L'échantillonnage se fait avec un vecteur de direction 3D $\vec{r}$ — le sampler trouve la face et le texel correspondant. Pour les réflexions : $\vec{r} = \text{reflect}(\vec{v}, \vec{n})$.

Skybox et réflexions PBR

- **Skybox** : un cube map placé autour de la scène, affiché en arrière-plan. Donne l'illusion d'un environnement lointain (ciel, montagnes, étoiles).

- **Réflexions PBR** : le matériau PBR (Session 5) réfléchit l'environnement. Pour un métal poli, la réflexion vient du cube map — le fragment shader calcule le vecteur de réflexion $\vec{r} = \text{reflect}(\vec{v}, \vec{n})$ et l'échantillonne dans le cube map.
- **IBL (Image Based Lighting)** : au lieu de lumières discrètes, l'éclairage vient de **tout** l'environnement — le cube map est préfiltré (diffuse irradiance + specular radiance) pour fournir l'éclairage diffus et spéculaire en un seul lookup.

🔗 Créer un cube map dans Blender

Dans Blender : **World** → **Surface** → **Background** → **Color** → **Environment Texture**.

Charger un fichier HDR (ex: de HDRI Haven). Blender le convertit automatiquement en cube map pour l'éclairage et les réflexions.

Pour exporter un cube map depuis Blender : **Render** → **Bake Type** → **Cubemap**, ou utiliser un script Python pour rendre les 6 faces depuis un point donné.

Partie 4 : Texture Atlases et Arrays

Le problème des draw calls

Chaque matériau avec une texture différente = un **draw call** séparé. Un objet avec 10 matériaux = 10 draw calls. Pour réduire les draw calls, on **groupe** plusieurs textures en une seule.

4.A. Texture Atlas

Atlas

Un **atlas** est une grande texture qui contient plusieurs sous-textures côte à côte. Les UVs de chaque objet pointent dans la région de l'atlas qui contient sa texture. Un seul draw call pour tous les objets qui utilisent l'atlas.

- **Avantage** : un draw call pour N matériaux.
- **Inconvénient** : le wrapping (repeat) ne marche plus — les UVs doivent rester dans la région de la sous-texture, sinon on « déborde » sur la voisine.

4.B. Texture Array

Texture Array

Un **texture array** est une pile de textures de **même taille**, échantillonnées avec un index de couche en plus des UVs : `texture(samplerArray, vec3(uv, layer))`. Plus moderne que l'atlas, pas de problème de wrapping.

- **Avantage** : un draw call, wrapping préservé.
- **Inconvénient** : toutes les textures doivent avoir la même résolution et le même format.

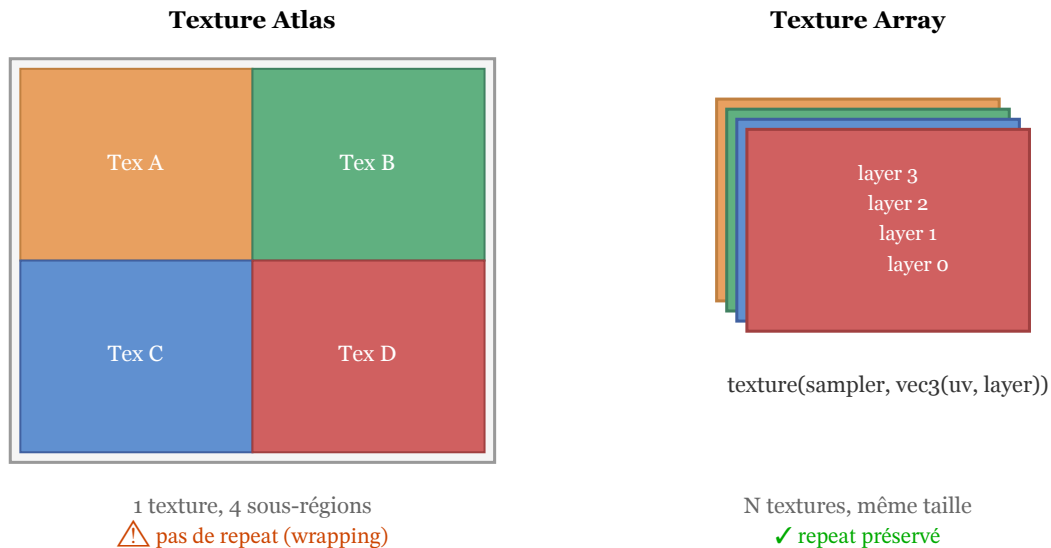


Fig. 30. – Texture atlas (gauche) : une grande texture découpée en sous-régions — pas de wrapping. Texture array (droite) : une pile de textures de même taille échantillonnées par index de couche — wrapping préservé.

4.C. Trim Sheets

! Le problème des éléments répétitifs

Dans un environnement (bâtiments, intérieurs, véhicules), beaucoup d'éléments sont des **bandes** répétitives : plinthes, moulures, bordures, gouttières, cadres. Faire une texture complète par élément = dizaines de draw calls et de la VRAM gaspillée. Le **trim sheet** regroupe toutes ces bandes dans **une seule texture**.

Principe du trim sheet

Un **trim sheet** est une texture (souvent **tall** — plus haute que large) qui contient plusieurs **bandes horizontales** empilées verticalement. Chaque bande est un matériau ou un élément d'architecture :

- plinthe basse, moulure, frise, tableau, corniche, etc.
- ou : bordure de toit, gouttière, cadre de fenêtre, rail.

Le mesh est **déplié** de façon à ce que chaque partie (plinthe, mur, moulure) utilise une **bande horizontale** spécifique de la texture. Les UVs horizontaux se **répètent** le long de la bande (le motif est tilable en X), tandis que les UVs verticaux sélectionnent la bonne bande.

$$UV_x \text{ in } \text{bande}_{\text{début}}..\text{bande}_{\text{fin}}, \quad UV_y \text{ in } [0, 1] \text{ répété}$$

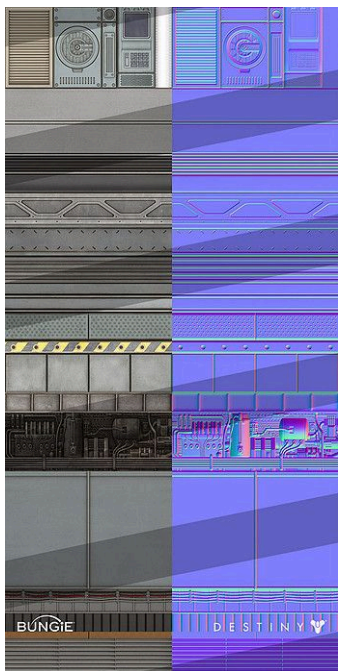


Fig. 31. – Exemple de trim sheet : plusieurs bandes de matériaux (bois, métal, peinture) empilées verticalement. Chaque bande est tilable horizontalement — on peut répéter le motif le long d’un mur sans voir de seam.

💡 **Avantages du trim sheet**

- **Un seul draw call** pour tout un bâtiment (murs, plinthes, moulures, cadres).
- **Wrapping horizontal préservé** — les bandes sont tilable en X, on peut répéter le motif autant de fois que nécessaire.
- **PBR complet** : le trim sheet contient souvent les **4 maps** (albedo, normal, roughness, AO) empilées de la même façon, synchronisées.
- **Workflow modulaire** : on construit des bibliothèques de trims réutilisables entre projets.

⚠️ **Contraintes**

- Le **dépliage** doit être précis : chaque partie du mesh doit aligner ses UVs sur la bonne bande. C’est un travail de modélisation rigoureux.
- Les bandes ont une **hauteur fixe** en pixels — si une plinthe fait 10cm dans le monde réel, sa bande fait par exemple 64px, et il faut que la résolution soit suffisante pour le détail.
- Le wrapping **vertical** ne marche pas (comme pour l’atlas) — on ne peut répéter qu’horizontalement.

Trim sheet vs Atlas vs Array

Critère	Atlas	Trim Sheet	Texture Array
Forme	Grille 2D	Bandes verticales	Pile de couches
Wrapping	Aucun	Horizontal seul	Complet

Critère	Atlas	Trim Sheet	Texture Array
Taille des sous-textures	Variable	Hauteur fixe	Identique
Usage typique	UI, sprites	Architecture, environnements	Terrain, matériaux

Le trim sheet est le **compromis** entre l'atlas (pas de wrapping) et l'array (tout est identique) : il préserve le wrapping dans **une direction** — celle qui compte pour les éléments linéaires.

Partie 5 : UV Avancés — Triplanar Mapping

! Le problème des UVs sur les formes complexes

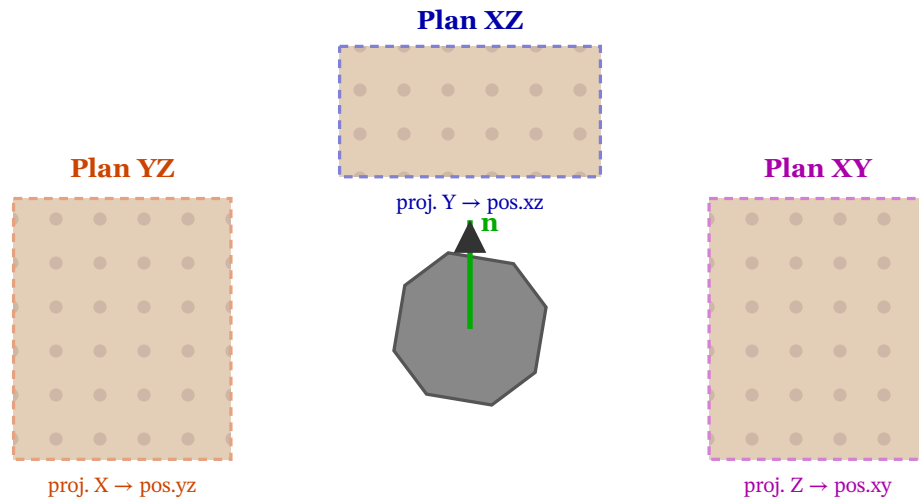
Les UVs classiques nécessitent un **dépliage** (Session 7) — un processus manuel, fastidieux, et qui produit des seams. Pour les formes **naturelles** (terrain, rochers, organique), le dépliage est souvent impraticable. Le **triplanar mapping** résout ce problème en **supprimant les UVs**.

Principe du triplanar mapping

Au lieu d'utiliser un canal UV, on **projette** la texture sur 3 plans orthogonaux (X, Y, Z) et on **mélange** les trois projections selon la normale de la surface :

- Une face orientée vers $+Y$ utilise surtout la projection sur le plan XZ (vue de dessus).
- Une face orientée vers $+X$ utilise surtout la projection sur le plan YZ (vue de côté).
- Le mélange est pondéré par les composantes de la normale : $w_i = |n_i|^p$ (exposant p pour durcir la transition).

$$\text{color} = \frac{w_x \cdot \text{proj}_x + w_y \cdot \text{proj}_y + w_z \cdot \text{proj}_z}{w_x + w_y + w_z}$$



$$w = |n|^p \rightarrow \text{color} = (w_x \cdot \text{proj}_x + w_y \cdot \text{proj}_y + w_z \cdot \text{proj}_z) / (w_x + w_y + w_z)$$

La normale détermine quelle projection domine

Fig. 32. – Triplanar mapping : la texture est projetée sur 3 plans orthogonaux (YZ, XZ, XY). Le poids de chaque projection est déterminé par la normale de la surface $w_i = |n_i|^p$. Une face orientée vers le haut utilise la projection XZ ; une face latérale utilise YZ ou XY.

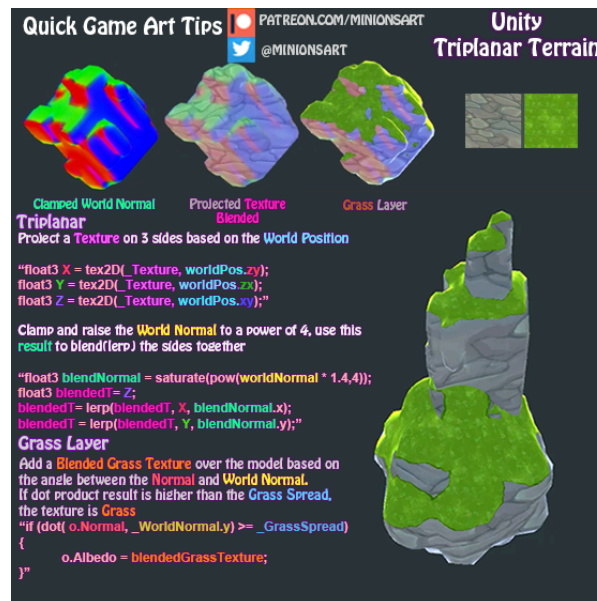


Fig. 33. – Exemple de triplanar mapping sur une forme complexe : la même texture est projetée depuis 3 directions, et les transitions sont fondues selon la normale — pas de seam visible, pas de dépliage nécessaire.

Triplanar mapping en GLSL (simplifié).

```
uniform sampler2D uTexture;
in vec3 vWorldPos;
in vec3 vNormal;

vec3 triplanar(sampler2D tex, vec3 pos, vec3 normal, float scale) {
```

```
// 3 projections avec les coordonnées de position
vec3 cx = texture(tex, pos.yz * scale).rgb; // plan YZ
vec3 cy = texture(tex, pos.xz * scale).rgb; // plan XZ
vec3 cz = texture(tex, pos.xy * scale).rgb; // plan XY

// Poids selon la normale (power pour durcir les transitions)
vec3 w = pow(abs(normal), vec3(8.0));
w /= (w.x + w.y + w.z);

return w.x * cx + w.y * cy + w.z * cz;
}
```

💡 Dans le Node Material Editor

Babylon.js NME propose un block **Triplanar** qui fait ce calcul automatiquement. On branche une texture, la position world-space et la normale — le block sort la couleur mélangée. Parfait pour les terrains, rochers, et toute géométrie sans UVs.

Partie 6 : Parallax Mapping

Le problème des normal maps

Les normal maps (Session 7) simulent du relief en **truquant l'éclairage**. Mais la surface reste plate — si on se déplace latéralement, les détails ne bougent **pas** avec la parallaxe. L'illusion se brise dès qu'on bouge la caméra.

Parallax mapping — décaler les UVs

Le **parallax mapping** utilise une **height map** (grayscale) pour décaler les UVs selon l'angle de vue. L'idée : si on regarde la surface en biais, les zones « hautes » devraient apparaître décalées par rapport aux zones « basses » — comme si on regardait un relief réel en biais.

$$UV_{\text{décalé}} = UV + \text{height}(UV) \cdot (\vec{v}_{\text{tangent}} \cdot k)$$

où $\vec{v}_{\text{tangent}}$ est le vecteur de vue dans l'espace tangent et k est un facteur d'intensité.

Parallax Occlusion Mapping (POM)

Le **Parallax Occlusion Mapping** est la version avancée : au lieu d'un seul décalage, on **ray-marche** dans la height map — on avance par petits pas le long du vecteur de vue, on compare la hauteur à chaque pas, et on s'arrête quand on « touche » la surface. Résultat : les zones hautes **occluent** les zones basses — un vrai relief apparent, avec des silhouettes déformées.

Le coût : plusieurs échantillons de texture par fragment (8 à 32 typiquement). Plus précis que le parallax simple, mais plus cher.

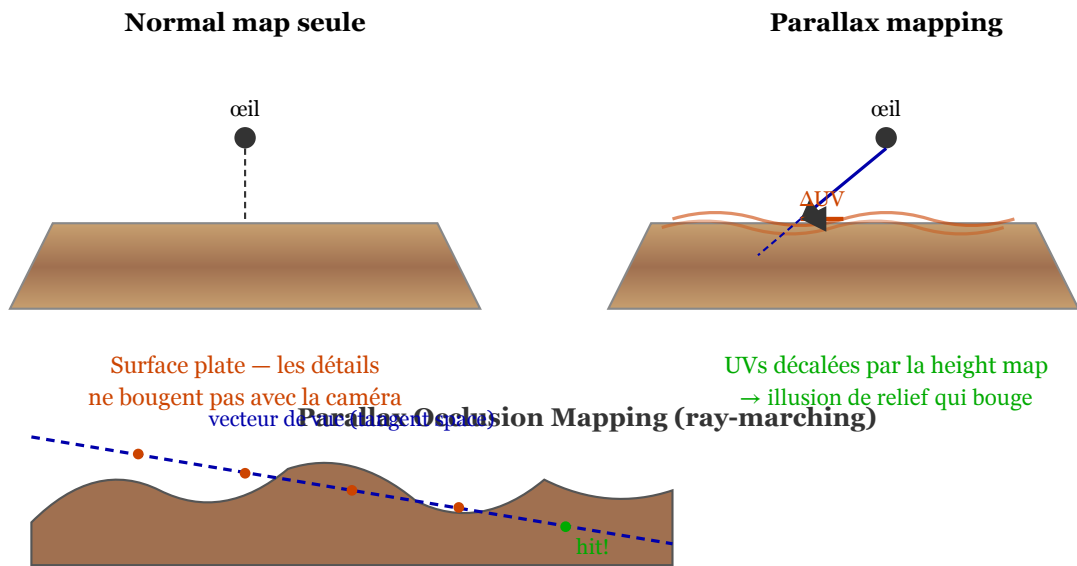


Fig. 34. – Normal map seule (gauche) : la surface reste plate, les détails ne bougent pas avec la caméra. Parallax mapping (droite) : les UVs sont décalées par la height map selon l'angle de vue — le relief apparent se déplace. En bas : le POM ray-marche dans la height map jusqu'à toucher la surface (point vert).

Parallax mapping simple en GLSL.

```
uniform sampler2D uHeightMap;
in vec2 vUV;
in vec3 mViewDirTangent; // vecteur de vue dans l'espace tangent

vec2 parallaxUV(vec2 uv, vec3 viewDir) {
    float height = texture(uHeightMap, uv).r; // [0,1]
    vec2 offset = viewDir.xy * height * 0.1; // 0.1 = intensité
    return uv - offset;
}

void main() {
    vec2 uv = parallaxUV(vUV, mViewDirTangent);
    vec3 albedo = texture(uAlbedoMap, uv).rgb; // utiliser les UVs décalées
    // ...
}
```

💡 Dans le Node Material Editor

Babylon.js NME propose un block **Parallax Occlusion Mapping**. On branche la height map, la normal map, et le vecteur de vue — le block sort les UVs décalées à utiliser pour toutes les autres textures du set PBR.

⚠ Limites du parallax

- La silhouette reste plate — le parallax **truque** la surface, pas les bords. Vu de profil très rasant, l'illusion se brise.
- Le POM coûte plusieurs échantillons par fragment — à réserver pour les surfaces proches de la caméra.
- Pour un vrai relief de silhouette, il faut du **displacement mapping** (déplacement des sommets) — beaucoup plus coûteux.

Partie 7 : Detail Textures et Blending

Le problème de la résolution

Une texture de sol 2048×2048 couvrant 10×10 mètres a une densité de ~ 200 texels/mètre. De près, chaque texel couvre ~ 5 « mm » — c'est flou. Augmenter la résolution à 8K coûte 16× plus de VRAM. Solution : **superposer** une texture de détail.

Detail texture

On superpose deux textures :

- **Base** : la texture unique du matériau (2048×2048), avec ses couleurs et ses variations.
- **Détail** : une petite texture tiling (512×512) répétée 10×, en **grayscale** ou **normal only**, qui ajoute du grain fin (pores, rayures, micro-relief).

Le mélange se fait par **multiplication** (pour l'albedo) ou **addition** (pour les normales) :

$$\text{albedo}_{\text{final}} = \text{albedo}_{\text{base}} \cdot (0.5 + \text{détail} \cdot 0.5)$$

Le facteur 0.5 centre le détail autour de 1.0 pour ne pas assombrir/éclaircir globalement.

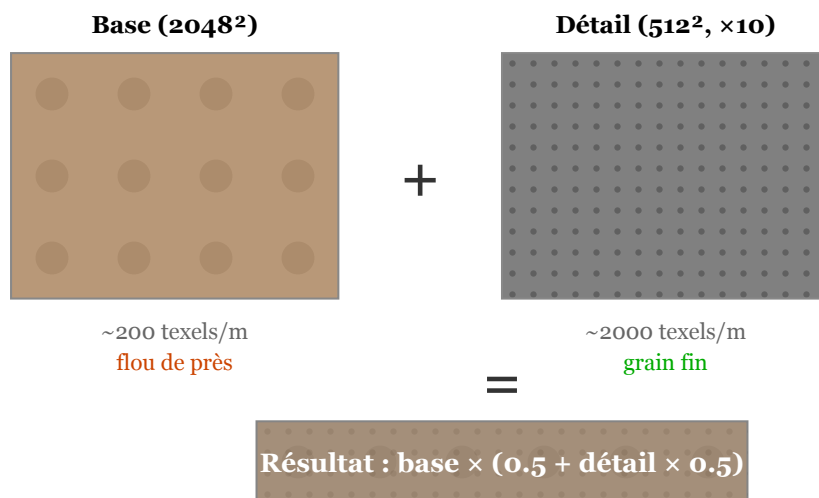


Fig. 35. – Detail texture : la base (2048², 200 texels/m) est floue de près. On superpose une petite texture de détail (512², répétée 10×) qui ajoute du grain fin (2000 texels/m). Le mélange par multiplication préserve les couleurs de base tout en ajoutant du micro-relief.

💡 Dans Blender — mix nodes

Dans le Shader Editor de Blender, utiliser un **Mix RGB** node en mode **Multiply** pour mélanger la base et le détail. Le **Fac** contrôle l'intensité du détail. Pour les normales, utiliser un **Normal Map** node avec la base, puis un **Mix** node en mode **Add** avec le détail normal.

Partie 8 : Pratique

💡 Objectif pratique

Mettre en place un tri-planar mapping avec une texture de détail.

Bonus commun

- **Triplanar** : remplacer les UVs classiques par un block **Triplanar** sur un terrain ou un rocher. Observer l'absence de seams.
- **Texture blending** : mélanger deux sets PBR (pierre + mousse) avec un masque procédural (bruit seillé). C'est la technique utilisée pour les terrains avec plusieurs matériaux (herbe, terre, rocher).

Session 9 : Baking de Textures avec Blender

🔔 Objectifs de la session

Partir d'un modèle généré par IA — diffuse acceptable, mais UVs non exploitables — et le rendre **propre** pour un moteur temps réel. On va voir comment **baker** (précalculer et transférer) une texture d'un UV vers un autre, refaire un dépliage soigné (marquage de seams, optimisation de la taille des îlots), puis ajouter du détail géométrique haute résolution et en **baker une normal map**. La session se termine par une **démonstration en direct** dans Blender, suivie d'une pratique d'éclairage statique dans Unity ou Godot (lightmaps + light probes pour un objet dynamique).

🔔 Rappel : UVs, PBR et lightmaps

- **Session 7** : les UVs relient chaque sommet à un texel ; le dépliage produit des îlots séparés par des seams ; le set PBR (albedo, normal, roughness, metallic, AO) définit un matériau moderne.
- **Session 8** : les render targets permettent de rendre la scène dans une texture — c'est exactement le mécanisme qu'utilise le baking.
- **Session 6** : les lightmaps sont des textures d'éclairage précalculé, échantillonnées via un canal UV2 dédié ; les light probes fournissent l'éclairage baked aux objets dynamiques.

Partie 1 : Le point de départ — un modèle issu de l'IA

Le contexte

Les générateurs de modèles 3D par IA (Meshy, Tripo, Rodin, Luma, etc.) produisent désormais un mesh + un set de textures PBR en quelques secondes. Le résultat est souvent **visuellement correct** en diffusion (albedo), mais souffre de deux défauts majeurs pour un usage en moteur :

- **UVs non optimisées** : dépliage automatique sans logique d'îlots, étirement, chevauchements, résolution gaspillée.
- **Détail géométrique absent** : le mesh est low-poly, les détails visuels sont « peints » dans la diffuse — pas de vrai relief, pas de normal map cohérente.

❗ Pourquoi ce n'est pas utilisable tel quel

- **En moteur**, des UVs étirées donnent une texture déformée et des mipmaps qui clignotent.
- **Sans normal map**, l'objet paraît plat sous un éclairage dynamique — la diffuse « fait semblant » d'avoir du relief, mais la lumière ne réagit pas.
- **Le baking** est le pont entre ces deux états : on reconstruit un matériau propre à partir du matériau IA.

Partie 2 : Le Baking — Principe Général

Qu'est-ce que baker ?

Baker une texture, c'est **précalculer** le résultat d'un rendu et l'écrire dans un fichier image, au lieu de le calculer en temps réel. Concrètement, Blender rend la scène (ou une partie) dans un **render target** (Session 8), puis sauvegarde ce render target comme texture.

Deux grandes familles de baking :

- **Baking de transfert** : recalculer une texture existante pour qu'elle corresponde à un **nouvel ensemble d'UVs**. Le rendu est le même, seul le dépliage change.
- **Baking de détail** : capturer le détail d'un mesh **haute résolution** vers la texture d'un mesh **basse résolution** (normal map, déplacement, AO).

! Le mécanisme commun — Cycles + Render Target

Dans Blender, le baking utilise le moteur **Cycles** (ray tracing, CPU/GPU). Pour chaque texel de la texture cible :

1. Le renderer trouve la face du mesh correspondante via les UVs cibles.
2. Il calcule la valeur demandée (couleur diffuse, normale, AO, etc.) à cet endroit.
3. Il écrit le résultat dans le pixel de la texture.

C'est exactement un **render target** (Session 8) — sauf qu'au lieu de rendre la vue d'une caméra, on rend **depuis les UVs** du mesh.

Bake Type	Sortie	Usage	Quand l'utiliser
Diffuse	Couleur RGB	Albedo propre	Transférer une diffuse vers de nouveaux UVs
Normal	Normales RGB	Relief apparent	Détail high-poly → low-poly
AO	Grayscale	Occlusion ambiante	Coin sombres, contact
Roughness / Metallic	Grayscale	PBR	Transférer le set PBR
Combined	RGB + éclairage	Lightmap	Précalculer l'éclairage d'une scène

Tableau 6. – Les principaux **Bake Type** de Blender. On en utilise trois aujourd'hui : **Diffuse** (transfert), **Normal** (détail), et indirectement **Combined** pour les lightmaps de la pratique (Session 6).

Partie 3 : Baking de Diffuse — Transférer vers de nouveaux UVs

Le problème

Le modèle IA a une diffuse **correcte**, mais ses UVs sont mauvaises. Si on refait le dépliage, la diffuse ne correspond plus — chaque texel de l'ancienne texture pointait vers une face avec les anciennes UVs. Il faut donc **recopier** la diffuse dans une nouvelle texture organisée selon les **nouveaux UVs**.

1. Le workflow en deux meshes

! L'astuce du double mesh

Le baking de transfert repose sur **deux meshes** :

- **Source** : le mesh d'origine avec ses UVs IA et sa diffuse. On le cache ou le désactive pour le rendu final.
- **Cible** : le même mesh (dupliqué), avec les **nouveaux** UVs et un matériau **vide** (Image Texture node branché sur le **Color** du Principled BSDF, avec une image vide créée).

Les deux meshes sont **superposés** (même position, même forme). Blender, pour chaque texel de la cible, va chercher la couleur correspondante sur la source par **ray casting** ou par correspondance d'UV.

2. Configuration dans Blender

Étapes du bake de diffuse.

1. **Sélectionner la source** (mesh IA + diffuse), puis **Shift-clic** la cible (le dernier sélectionné est l'objet actif = cible).
2. Ouvrir le **Shader Editor** sur la cible : ajouter un node **Image Texture**, créer une nouvelle image (ex: 2048×2048, 32-bit float si on veut garder de la plage), la brancher sur le **Base Color** du Principled BSDF. Garder ce node **sélectionné** — c'est lui qui reçoit le bake.
3. Dans **Render Properties** → **Bake** :
 - **Bake Type** : Diffuse.
 - **Influence** : décocher **Direct** et **Indirect** pour ne garder que **Color** (on veut l'albedo, pas l'éclairage).
 - **Margin** : 4-8 px (dilate la couleur dans les espaces entre îlots pour éviter des seams noirs au mipmapping).
4. Cliquer **Bake**. La nouvelle image se remplit, on l'enregistre (Image → Save As).

⚠ Pièges fréquents

- **Selected to Active** : c'est l'option qui dit « bake de la source sélectionnée vers la cible active ». Si elle est cochée par erreur (ou inversement), le bake échoue ou produit du noir. Vérifier le sens : **source sélectionnée en premier, cible active en dernier**.
- **Cage / Extrusion** : si les deux meshes ne sont pas parfaitement superposés, des rayons peuvent rater la source. L'option **Cage** ou un **Extrusion** positif (ex: 0.05) donne une marge de tolérance.
- **Margin** : sans margin, les mipmaps affichent des bordures noires entre îlots en moteur — toujours mettre au moins 4 px.

Partie 4 : Refaire un UV Propre

L'objectif d'un bon dépliage

Un dépliage propre maximise la **qualité visuelle en moteur** pour un coût de texture donné :

- **Pas d'étirement** : une face 3D carrée doit occuper une zone 2D carrée.
- **Îlots logiques** : regrouper les faces par région du modèle (une île par bras, par jambe, par face de boîte) plutôt qu'en mille morceaux.

- **Packing compact** : minimiser l'espace vide — la texture est un bien précieux (VRAM).
- **Taille des îlots proportionnelle à leur visibilité** : une face qu'on voit souvent doit être plus grande dans la texture qu'une face cachée dessous.

1. Marquer les seams

! Le marquage manuel — la base du bon dépliage

Le **Smart UV Project** de Blender (Session 7) fait un dépliage automatique, mais il produit beaucoup de petites îlots et des seams visibles. Pour un modèle destiné à un jeu, on **marque les seams à la main** en suivant des règles :

- **Coudre sur les arêtes cachées** : placer les seams là où l'œil ne les verra pas (dessous, dos, intérieur des bras, dessous des pieds). Un seam sur le devant du visage est catastrophique.
- **Suivre les lignes naturelles** : aisselles, ceinture, lignes de vêtements, bordures de pièces mécaniques. Le seam suit une couture réelle.
- **Minimiser le nombre d'îlots** sans créer d'étirement : une île par région logique, pas une île par face.

)

Workflow de marquage.

1. Mode Édition, sélection d'arêtes (2).
 2. Sélectionner une arête, Ctrl-E → Mark Seam (ou Right-click → Mark Seam selon la keymap).
- **Astuce** : sélectionner une **boucle d'arêtes** avec Alt-clic puis Mark Seam coupe tout un tour en une action.
1. Une fois les seams posées, **tout sélectionner** (A), U → Unwrap. Les îlots apparaissent dans l'UV Editor.
 2. Vérifier l'étirement avec l'overlay **Stretch** (UV Editor → View → Display Stretch) : les zones rouges sont étirées, il faut replacer un seam.

2. Optimiser la taille des îlots

Pourquoi la taille des îlots compte

La **densité de texels** d'une île = (aire de l'île dans la texture) / (aire 3D de la surface). Plus elle est élevée, plus la texture est nette sur cette région. Si toutes les îlots ont la même densité, la texture est **uniformément nette** — c'est l'idéal. Si une îlot occupe 50% de la texture pour une face cachée dessous, on gaspille de la VRAM.

💡 Règles pratiques

- **Régions visibles en premier plan** (visage, poignée, plaque avant) : îlots **grandes**.
- **Régions rarement vues** (dessous, dos, intérieur) : îlots **petites**.
- **Uniformiser la densité** : dans l'UV Editor, **Select** → **All**, puis **UV** → **Average Island Scale** (ou Ctrl-A en mode UV) égalise la densité entre îlots.

- **Packing** : UV → **Pack Islands** (Ctrl-P) réarrange les îlots sans chevauchement. L'option **Margin** (ex: 0.01) laisse un espace pour le bake margin.
- **Aligner** : pour les îlots rectangulaires (faces de boîte, panneaux), les aligner sur les axes pour un packing dense.

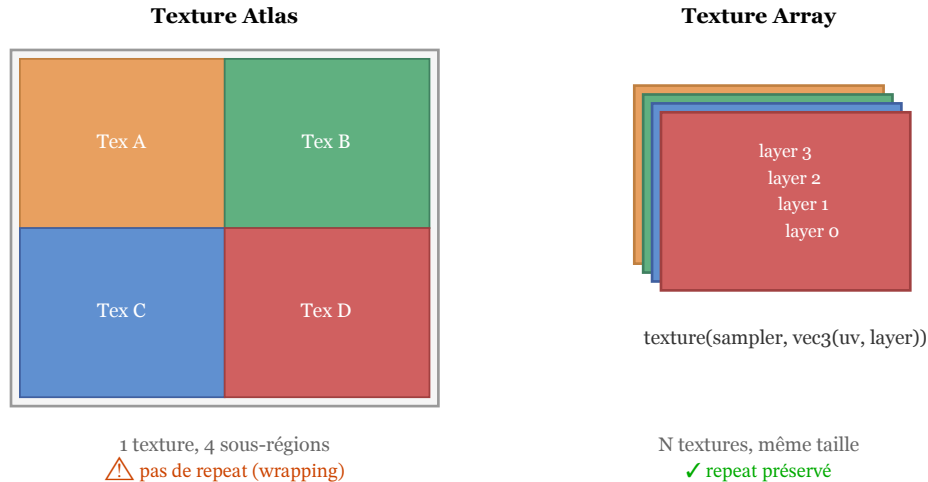


Fig. 36. – Bon vs mauvais packing. À gauche : îlots mal réparties, étirement, espace gaspillé. À droite : îlots logiques, densité uniforme, packing compact. La même texture 2048×2048 donne une qualité visuelle très différente selon le dépliage.

3. Itérer bake → dépliage → bake

⚠ Le baking n'est pas une étape finale

Une fois la diffuse bakée vers les nouveaux UVs, on peut découvrir que certains îlots sont trop petits (détail flou) ou mal orientés (motif qui « tourne » bizarrement). Le workflow est **itératif** :

1. Marquer les seams, déplier, packer.
2. Baker la diffuse.
3. Inspecter le résultat en moteur (ou en viewport Blender avec la nouvelle texture).
4. Ajuster les seams / la taille des îlots, rebaker.

Ne pas hésiter à **rebaker** — c'est une opération rapide (quelques secondes pour une diffuse 2K), et le gain de qualité est énorme.

Partie 5 : Baking de Normal Map – Détail High-Poly → Low-Poly

Le principe

Une **normal map** (Session 7) stocke des normales perturbées dans un RGB — elle **truque l'éclairage** pour faire croire qu'il y a du relief. Pour obtenir une normal map **cohérente avec la géométrie réelle**, on sculpte un mesh **haute résolution** (millions de polygones) et on bake ses normales vers la texture d'un mesh **basse résolution** (quelques milliers de polygones).

Le low-poly est utilisé en moteur (rapide à rendre), mais il **affiche** le détail du high-poly grâce à la normal map.

1. Sculpter le détail

💡 Workflow de sculpt

1. **Dupliquer** le mesh cible (low-poly) et le renommer high.
2. Passer en **Sculpt Mode**, ajouter des **Multiresolution Modifier** ou utiliser le **Dyntopo** pour sculpter des détails : rayures, boulons, écailles, stries, dommages.
3. On n'a pas besoin d'être photoréaliste — le détail sera **baked**, donc visible uniquement via la normal map. L'important est que les formes soient **lisibles** à l'échelle du modèle.
4. Conserver le low-poly **inchangé** : c'est lui qui sera exporté en moteur.

)

2. Baker la normal map

Étapes du bake de normal.

1. **Sélectionner le high-poly**, puis **Shift-clic** le low-poly (actif = cible).
2. Dans le Shader Editor du low-poly : ajouter un **Image Texture** node, créer une nouvelle image (2048×2048, 16-bit, **Non-Color** color space — important, pas de sRGB), le brancher sur le **Normal Map** node lui-même branché sur le **Normal** du Principled BSDF. Garder le node sélectionné.
3. Dans **Render Properties** → **Bake** :
 - **Bake Type** : Normal.
 - **Space** : Tangent (le standard pour les moteurs — Unity, Godot, Unreal l'attendent par défaut). Object est rare.
 - **Selected to Active** : **coché** (on bake du high vers le low).
 - **Extrusion** : 0.05 (marge pour le ray casting).
 - **Cage** : optionnel, utile si la forme est concave.
 - **Margin** : 4 px.
4. Cliquer **Bake**, sauvegarder l'image.

)

❗ Espaces tangents — cohérence moteur

Une normal map **tangent space** dépend de la **base tangente** du mesh (vecteurs tangent, bitangent, normale par sommet). Si le low-poly est modifié après le bake (ou exporté avec une autre option de normales), la normal map « clique » ou s'inverse. Règle d'or :

- **Baker après** avoir finalisé le low-poly (normales, smoothing, export).
- **Exporter** le low-poly avec les **normales calculées** (FBX/glTF avec +X forward, +Y up selon le moteur).
- En moteur, s'assurer que le matériau attend une normal map **tangent space** (défaut dans Unity/Godot).

⚠ Erreurs classiques de normal bake

- **Couleurs arc-en-ciel** : le ray casting a raté le high-poly par endroits → augmenter **Extrusion** ou utiliser un **Cage**.

- **Normales inversées** (bleu au lieu de rouge/vert) : mauvais **Space** ou low-poly dont les normales sont retournées (Ctrl-N → Recalculate Outside).
- **Seams visibles** : le low-poly a des **sharp edges** aux seams. Soit lisser les normales (Shade Smooth + **Auto Smooth**), soit baker avec un **Edge Split** désactivé.
- **sRGB au lieu de Non-Color** : la normal map est interprétée comme couleur → éclairage cassé. Vérifier le **Color Space** dans l'Image node et à l'import en moteur.

3. Compléter le set PBR

💡 Baker aussi l'AO et le roughness

Une fois la normal map bakée, on peut aussi baker :

- **AO** (Bake Type → Ambient Occlusion, Selected to Active) : capture l'occlusion du high-poly dans les creux — un AO baked est plus précis qu'un SSAO temps réel.
- **Roughness / Metallic** : si le high-poly a des matériaux assignés par face, on peut baker ces canaux aussi, ou les peindre manuellement dans l'Image Editor.

On se retrouve avec un **set PBR complet** (albedo + normal + AO + roughness + metallic) prêt pour un matériau Unity/Godot.

)

Partie 6 : Démonstration en Direct

💡 Déroulement de la démo

La démo se fait en **live** dans Blender, sur un modèle IA fourni (ex: un objet simple généré via Meshy ou Tripo — un bouclier, une caisse, un petit prop). Les étapes suivent exactement les Parties 3 à 5.

Plan de la démo (45-60 min)

Phase 1 — Diagnostic (5 min)

- Ouvrir le modèle IA, montrer la diffuse et les UVs dans l'UV Editor.
- Pointer les défauts : étirement, îlots en mille morceaux, chevauchements, seams au milieu du visage.

Phase 2 — Baking de transfert de diffuse (10 min)

- Dupliquer le mesh en source (UVs IA) et cible (UVs qu'on va refaire).
- Refaire le dépliage de la cible : marquer les seams à la main, Unwrap, Average Island Scale, Pack Islands.
- Configurer le bake (Diffuse, Color only, Selected to Active, Margin 6), baker, sauver la nouvelle diffuse.
- Comparer l'ancienne et la nouvelle texture dans l'UV Editor.

Phase 3 — Sculpt high-poly (15 min)

- Dupliquer la cible en high, passer en Sculpt Mode.
- Ajouter des détails : rayures, boulons, écailles, dommages — selon le prop.
- Insister sur le fait que le low-poly reste **inchangé**.

Phase 4 — Baking de normal map (10 min)

- Configurer le bake (Normal, Tangent, Selected to Active, Extrusion 0.05, Margin 4).
- Baker, sauver en Non-Color.
- Brancher la normal map dans le Principled BSDF via un **Normal Map** node, observer le relief en viewport (mode Material Preview ou Rendered).

Phase 5 — Bonus : AO baked (5 min)

- Baker l'Ambient Occlusion du high vers le low, la multiplier avec la diffuse dans le shader.

Phase 6 — Export et test en moteur (5-10 min)

- Exporter le low-poly en glTF/FBX + les textures (albedo, normal, AO).
- Ouvrir rapidement dans Godot ou Unity, assigner le matériau Standard, vérifier que la normal map réagit à la lumière.

⚠ Points à insister pendant la démo

- **L'ordre des sélections** source/cible — c'est l'erreur n°1 des étudiants.
- **Le Color Space** Non-Color de la normal map — sinon tout casse en moteur.
- **Le margin** — sans lui, des seams noirs apparaissent au mipmapping.
- **L'itération** — un premier bake est rarement parfait, on ajuste et on rebake.

Partie 7 : Pratique — Éclairage Statique en Moteur

🎯 Objectif de la pratique

Mettre en application les concepts de la **Session 6** (lightmaps, light probes) : construire une petite scène statique, **baker son éclairage**, et éclairer un **objet dynamique** avec des **light probes**. Au choix : **Unity** ou **Godot**.

💬 Ce qu'on réutilise de la Session 6

- **Lightmap** : texture d'éclairage précalculée, échantillonnée via UV2 (Partie 2 de la Session 6).
- **Light probes** : échantillonnent l'éclairage baked à des positions, interpolées pour les objets dynamiques (Partie 4 de la Session 6).
- **Stratégie mixte** : statique en lightmap, dynamique en probes + lumières temps réel (Partie 5 de la Session 6).

Voir les figures et de la Session 6 pour rappel visuel.

)

Cahier des charges

La scène à construire

Une **pièce intérieure simple** (un sol, 4 murs, un plafond) avec :

- **Géométrie statique** : les murs, le sol, le plafond, et quelques props posés (caisses, chaise, table). Tout est marqué **static** / **bake mode : static**.

- **Une lumière directionnelle ou ponctuelle** par une fenêtre ou ouverture — source principale.
- **Une lumière secondaire** plus douce (lampe d’ambiance, ou sky light par la fenêtre).
- **Un objet dynamique** : un petit personnage (capsule, cube, ou le modèle IA de la démo) qui se **déplace** dans la pièce au runtime (script simple de patrol ou contrôle clavier).

)

Étapes — Godot

Workflow Godot (LightmapGI + LightmapProbe).

1. Créer la scène, placer les meshes statiques et la lumière.
2. Marquer les meshes statiques : **Geometry** → **GI Mode** → **Static**.
3. Ajouter un noeud **LightmapGI**, régler la **Bake Quality** sur **Medium** (pour la vitesse en classe).
4. Placer quelques **LightmapProbe** autour de la zone où l’objet dynamique se déplace.
5. Marquer l’objet dynamique : **GI Mode** → **Dynamic**.
6. **Bake** (bouton **Bake Lightmaps** du LightmapGI). Attendre la fin du calcul.
7. Lancer la scène, déplacer l’objet dynamique, observer :
 - Les murs et le sol sont éclairés par la lightmap (aucun calcul runtime).
 - L’objet dynamique reçoit l’éclairage des probes — il s’assombrit dans les coins, s’éclaire près de la fenêtre, sans aucune lumière temps réel.
8. **Référence** : LightmapGI et LightmapProbe.

)

Étapes — Unity

Workflow Unity (Progressive Lightmapper + Light Probes).

1. Créer la scène, placer les meshes et la lumière.
2. Marquer les meshes statiques : cocher **Static** dans l’inspecteur (ou **Lightmap Static** dans le menu déroulant).
3. **Window** → **Rendering** → **Lighting**, onglet **Baked Lightmaps**.
4. **Lighting Mode** : **Baked Indirect** ou **Subtractive** (selon le résultat voulu). **Realtime Shadow Color** pour les ombres dynamiques en mode Subtractive.
5. Placer un **Light Probe Group** (GameObject → Light → Light Probe Group), étendre les probes autour de la zone de déplacement.
6. Cliquer **Generate Lighting** (en bas du panneau Lighting). Attendre la fin du bake.
7. L’objet dynamique reçoit automatiquement l’éclairage des probes (pas de réglage supplémentaire si le Renderer est standard).
8. Lancer, déplacer l’objet, observer la cohérence d’éclairage entre statique et dynamique.
9. **Référence** : Unity Lightmapping et Unity Light Probes.

)

Critères de réussite

Ce qui est évalué

- **Lightmap visible** : les murs et le sol montrent un éclairage baked (taches claires, ombres portées, AO dans les coins) — pas de calcul temps réel pour la géométrie statique.

- **Cohérence probes** : l'objet dynamique change d'éclairage selon sa position (assombrit dans les coins, éclairé près de la fenêtre) **sans lumière temps réel** — preuve que les probes fonctionnent.
- **Pas de fuite** : pas de « lumière qui traverse un mur » (signe d'une probe mal placée ou d'un mesh non statique).
- **Performance** : avec les lumières temps réel désactivées, la scène reste correctement éclairée — preuve que l'éclairage est bien baked.

)

⚠ Pièges fréquents en pratique

- **Mesh non marqué static** : la lightmap ne s'applique pas, l'objet reste noir ou éclairé en temps réel. Vérifier le **GI Mode** (Godot) ou **Static** (Unity) sur **tous** les meshes statiques.
- **Probes trop loin** : si l'objet dynamique sort de la zone couverte par les probes, l'éclairage devient plat ou erratique. Couvrir généreusement la zone de déplacement.
- **UV2 manquants** : Godot génère les UV2 automatiquement ; Unity aussi. Mais si un mesh importé a des UV2 explicites cassés, le bake échoue. Vérifier l'import.
- **Bake quality trop haute** : en classe, rester sur **Medium** — un bake **Ultra** sur une scène complexe peut prendre des dizaines de minutes.

)

💡 Bonus

- **Ajouter une reflection probe** (Session 6, Partie 6) près d'un objet métallique pour qu'il réfléchisse l'environnement baked.
- **Comparer** la même scène en **fully real-time** (SDFGI en Godot, Realtime GI en Unity) vs **baked** — mesurer la différence de FPS et de qualité.
- **Rebaker** après avoir déplacé une lumière : c'est l'inconvénient principal du baking, le vivre pour le retenir.

)

Session 10 : Ombres en Temps Réel — le Shadow Mapping

🔗 Objectifs de la session

Comprendre comment le GPU calcule les ombres portées en temps réel. La **Session 5** a posé la question — « comment le GPU sait-il qu'un fragment est derrière un autre vu depuis la lumière ? (Dans l'ombre) » — et a promis la réponse en Session 6. Le baking (Session 6) l'a esquivée en précalculant les ombres statiques dans la lightmap. Aujourd'hui on y revient : le **Shadow Mapping** est LA technique universelle du temps réel, utilisée par Three.js, Unity, Unreal, Godot, et jusqu'aux moteurs custom. On va voir le principe, les artefacts (acne, peter panning), les filtres d'adoucissement (PCF, PCSS), les variantes pour grandes scènes (CSM) et pour lumières ponctuelles (cube shadow maps), puis l'implémentation pratique en Three.js avec une démo interactive, et un aperçu des réglages dans Godot et Unity.

🔗 Rappel : ce qu'on sait déjà

- **Session 5** : l'éclairage direct calcule $\vec{N} \cdot \vec{L}$ par fragment. Une surface dos à la lumière reçoit 0 — mais ce n'est pas une ombre, c'est juste l'absence d'éclairage direct. Une **ombre portée** apparaît quand un objet **bloque** la lumière d'un autre.
- **Session 6** : les lightmaps précalculent les ombres dans une texture (qualité ray tracing, coût nul), mais uniquement pour la géométrie **statique**, un moteur de rendu a calculé les ombres au préalable. Un personnage qui marche n'a pas d'ombre baked — il va nous falloir trouver des solutions pour le temps réel.
- **Session 8** : On a abordé les **render target** qui sont des textures dans lesquelles le GPU rend au lieu d'afficher. Le shadow mapping est réutilise cette idée.

Partie 1 : Pourquoi les Ombres ?

Le rôle perceptif des ombres

Une scène sans ombres paraît **plate et flottante** : on ne sait pas si un objet touche le sol, ni à quelle distance il se trouve. Les ombres apportent trois informations critiques :

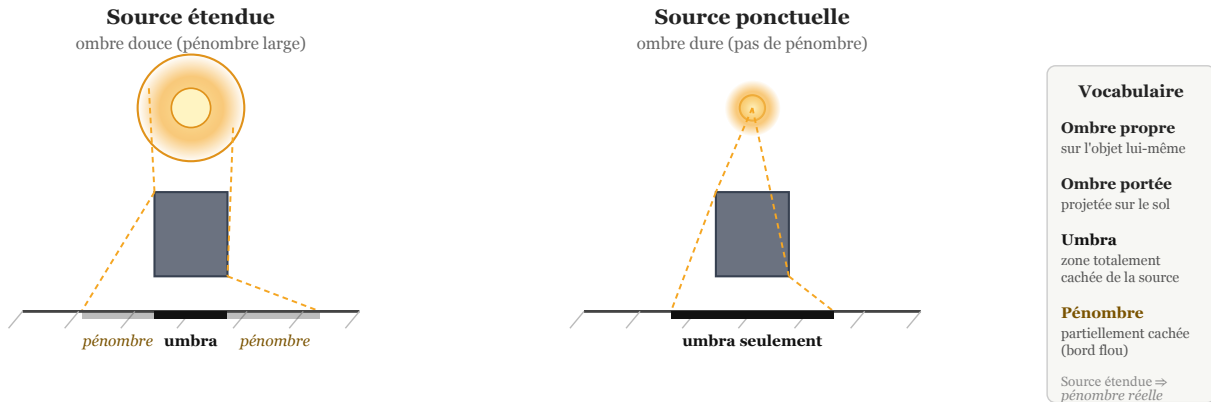
- **Contact** : une ombre attachée à la base d'un objet confirme qu'il est posé. Sans elle, l'objet semble flotter.
- **Profondeur et position** : la longueur et la direction de l'ombre révèlent la position de l'objet dans l'espace et la hauteur de la source.
- **Échelle** : une ombre longue indique une lumière rasante (lever/coucher), une ombre courte indique une lumière au zénith.

💡 Ombre propre vs ombre portée

- **Ombre propre** (self-shadow) : les parties de l'objet non éclairées — déjà gérées par $\max(\vec{N} \cdot \vec{L}, 0)$ (Session 5). C'est le côté sombre de la sphère.

- **Ombre portée** (cast shadow) : l'ombre que l'objet **projette** sur d'autres surfaces. C'est celle-ci qui demande le shadow mapping — le GPU ne peut pas la déduire de la seule normale du receveur.

1. Anatomie d'une ombre



Anatomie d'une ombre : umbra (cœur plein) et pénombre (bord adouci). Une source ponctuelle idéale ne produit que de l'umbra.

Fig. 37. – Anatomie d'une ombre. L'**umbra** est la zone totalement cachée de la source ; la **pénombre** est la zone partiellement cachée, qui produit le bord flou. Une source ponctuelle idéale (à droite) ne produit que de l'umbra — d'où l'aspect « tranché » des ombres temps réel non filtrées.

Umbra et pénombre

- **Umbra** : région d'où la source est **entièrement** masquée par le bloqueur. Aucune lumière n'arrive — c'est le cœur plein de l'ombre.
- **Pénombre** : région d'où la source n'est que **partiellement** masquée. Une partie des rayons arrive — c'est le bord adouci.
- **Source étendue** : un soleil ou une ampoule a une **taille** (un disque, pas un point). C'est ce qui crée la pénombre. Plus la source est grande (ou proche), plus la pénombre est large.

🔗 Conséquence pour le temps réel

Le shadow mapping de base modélise une **source ponctuelle** — il ne produit que de l'umbra (bord net). Pour retrouver la pénombre d'une source étendue, il faut **filtrer** (Partie 5 : PCF, PCSS). C'est la principale différence entre une ombre « jeu vidéo ancienne génération » (nette, tranchée) et une ombre moderne (douce, réaliste).

Partie 2 : Le Shadow Mapping — Principe

L'idée centrale

« Ce qui est visible depuis la lumière est éclairé ; ce qui est masqué vu depuis la lumière est dans l'ombre. »

Le GPU ne « sait » pas qu'un fragment est derrière un objet. Mais il sait **mesurer la profondeur**. Si on rend la scène **depuis la lumière** et qu'on garde la profondeur de chaque pixel, on obtient une

carte de « ce que la lumière voit ». Ensuite, pendant le rendu caméra, pour chaque fragment on regarde sa profondeur **vue depuis la lumière** : si elle est **plus grande** que la valeur stockée, c'est qu'un autre objet est plus proche de la lumière à cet endroit — donc le fragment est dans l'ombre.

1. Les deux passes

! Le rendu en deux passes

1. **Passe 1 — Shadow map** : on place la caméra à la position de la lumière, on oriente vers la scène, on rend **uniquement la profondeur** (pas la couleur) dans une texture — la **shadow map**. C'est un render target (Session 8) au format depth.
2. **Passe 2 — Rendu final** : on rend la scène depuis la caméra du jeu. Pour chaque fragment, on calcule sa position dans l'espace de la lumière, on échantillonne la shadow map à cet endroit, et on compare les profondeurs.

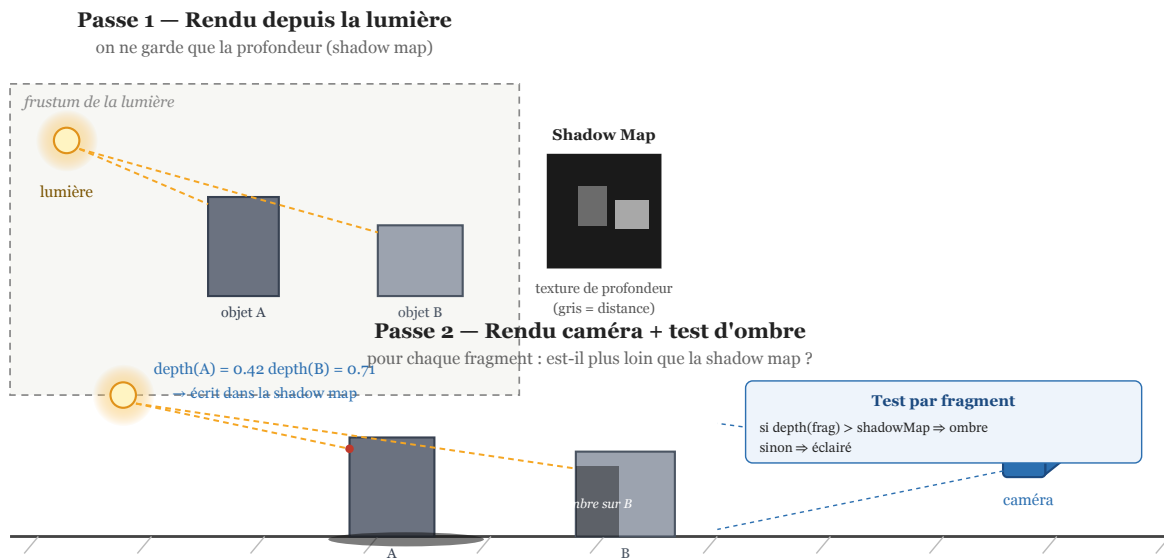


Fig. 38. – Le shadow mapping en deux passes. Passe 1 : la scène est rendue depuis la lumière, on ne garde que la profondeur (gris = distance) dans la shadow map. Passe 2 : depuis la caméra, chaque fragment est re-projeté dans l'espace lumière ; si sa profondeur dépasse celle stockée dans la shadow map, il est dans l'ombre.

2. La shadow map — une texture de profondeur

Format et contenu

La shadow map est tout simplement le **depth buffer** (Session 3) d'un rendu fait depuis la lumière — sauvegardé dans une texture échantillonnable. Chaque texel contient la profondeur du fragment le plus proche de la lumière à cet endroit.

- **Résolution** : typiquement 1024^2 à 4096^2 . Plus grand = ombres plus nettes, mais plus de VRAM et de coût de rendu.
- **Format** : depth texture (souvent 16 ou 24 bits). 24 bits est le standard — 16 bits provoque de l'acne (Partie 4) par manque de précision.

- **Clear** : initialisée à 1.0 (profondeur maximale) avant la passe 1, comme tout depth buffer.

3. Les coordonnées d'ombre — projeter dans l'espace lumière

! Le principe

La shadow map est dans l'espace de la lumière. Le fragment est dans l'espace monde. Pour comparer, on reprojette p_{world} dans l'espace de la lumière — c'est le même pipeline que celui qui place un vertex à l'écran (Session 2), mais avec la **caméra de la lumière** :

$$p_{\text{world}} \xrightarrow{V_L} p_{\text{vue L}} \xrightarrow{P_L} p_{\text{clip L}} \xrightarrow{\div w} p_{\text{NDC L}} \xrightarrow{\text{remap}} (s_x, s_y, s_z)$$

On obtient les **shadow coordinates** (s_x, s_y, s_z) , toutes dans $[0, 1]$:

- (s_x, s_y) : **quel pixel de la shadow map regarder** (UV).
- s_z : **à quelle profondeur est le fragment vu depuis la lumière**.

Le test : si la shadow map à (s_x, s_y) contient une profondeur **plus petite** que $s_z \rightarrow$ un autre objet est plus proche de la lumière $\rightarrow$ le fragment est dans l'ombre.

💡 On compare des profondeurs normalisées, pas des distances

s_z est le z NDC de la lumière (après perspective divide), pas la distance brute $|\text{lumière} - \text{fragment}|$.
La shadow map stocke la même grandeur. Il faut comparer dans le **même espace**.

💡 En pratique

Tout se précalcule en une matrice : $S = \text{remap} \times P_L \times V_L$. Le shader fait $s = S \times p_{\text{world}}$, puis la perspective divide : $(s_x/s_w, s_y/s_w, s_z/s_w)$. Pour le soleil (orthographique), $w = 1$ donc la divide ne change rien — mais le même shader gère tous les types de lumière.

S est recalculée quand la lumière bouge ou que sa projection change (ex. CSM qui suit la caméra).
Pour un soleil statique, on la calcule une seule fois.

Partie 3 : Le Shader d'Ombre

Fragment shader — test d'ombre basique.

```
uniform sampler2D uShadowMap; // la depth texture de la passe 1
uniform mat4     uShadowMatrix; // S = remap * P_light * V_light
in vec3 vWorldPos;

float shadowTest(vec3 worldPos) {
    // 1. Coordonnées d'ombre du fragment
    vec4 s = uShadowMatrix * vec4(worldPos, 1.0);
    s.xyz /= s.w; // perspective divide
    // s.xyz est maintenant dans [0,1] si le fragment est dans le frustum de la lumière

    // 2. Hors du frustum lumière ? Pas d'ombre (la lumière ne "voit" pas cet endroit)
    if (s.x < 0.0 || s.x > 1.0 || s.y < 0.0 || s.y > 1.0 || s.z > 1.0)
        return 1.0; // éclairé
```

```
// 3. Comparaison des profondeurs
float storedDepth = texture(uShadowMap, s.xy).r;
float currentDepth = s.z;

// 4. Test : si le fragment est plus loin que ce que la lumière voit → ombre
return (currentDepth > storedDepth) ? 0.0 : 1.0; // 0 = ombré, 1 = éclairé
}
```

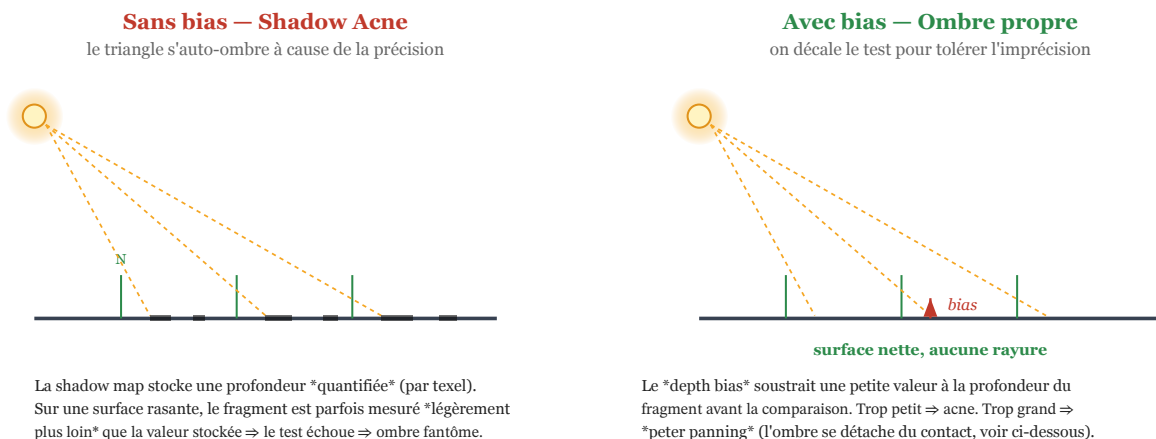
Le résultat (0 ou 1) **multiplie** la contribution de la lumière dans le shader d'éclairage (Session 5) : $\text{color} = \text{diffuse} * (\text{NdotL} * \text{shadow} + \text{ambient})$. Sans cette multiplication, l'ombre n'existe pas — le shadow mapping n'est qu'un **masque**.

Le coût du shadow mapping

- **Passe 1** : un rendu complet de la scène (sans couleur, sans shader complexe — juste la profondeur). Coût GPU proportionnel au nombre de triangles et à la résolution de la shadow map.
- **Passe 2** : un échantillonnage de texture supplémentaire + une comparaison par fragment et par lumière qui projette une ombre.
- **Total** : ajouter une lumière avec ombre coûte **environ un rendu de scène supplémentaire**. C'est pourquoi on limite drastiquement le nombre de lumières avec ombres (1 à 4 en général, contre des dizaines sans ombre).

Partie 4 : Les Artefacts — Acne et Peter Panning

1. Le shadow acne



Le shadow acne : des rayures sombres apparaissent sur les surfaces éclairées en biais. Le bias corrige ce défaut mais peut créer le peter panning.

Fig. 39. — Le shadow acne : des rayures sombres apparaissent sur les surfaces éclairées en biais. La cause est la **quantification** de la shadow map — chaque texel couvre une zone de la surface, et la profondeur stockée est une valeur moyenne. Un fragment légèrement en arrière de cette valeur échoue le test et devient « ombré » par lui-même.

La cause exacte

La shadow map a une **résolution finie** : un texel couvre plusieurs fragments de la surface. La profondeur stockée est celle du fragment le plus proche **au centre** du texel. Pour un fragment voisin sur une surface en biais, la profondeur réelle diffère légèrement — parfois elle est **juste au-dessus** de la valeur stockée, et le test `currentDepth > storedDepth` réussit à tort. Le fragment s'auto-ombrage. C'est d'autant pire que la surface est **rasante** par rapport à la lumière (la profondeur varie vite sur la surface).

La solution : le depth bias

On **décalle** la profondeur du fragment avant la comparaison :

```
float bias = 0.005;
return (currentDepth - bias > storedDepth) ? 0.0 : 1.0;
```

Le bias compense l'imprécision de la shadow map. Trop petit → l'acne persiste. Trop grand → l'ombre se **détache** du contact (peter panning, ci-dessous).

Il existe deux formes de bias :

- **Constant bias** : une valeur fixe soustraite à `currentDepth`. Simple mais inadapté (la bonne valeur dépend de l'angle de la surface).
- **Slope-scaled bias** : le bias augmente avec la pente de la surface par rapport à la lumière ($\frac{1}{\max(\vec{N} \cdot \vec{L}, \epsilon)}$). Les surfaces rasantes (où l'acne est pire) reçoivent un plus grand bias. C'est le réglage standard des moteurs.

2. Le peter panning

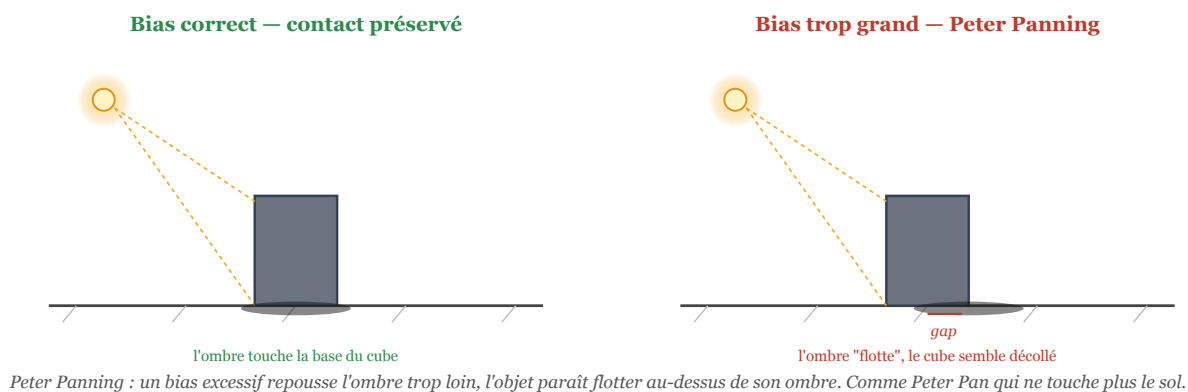


Fig. 40. – Peter Panning : un bias trop grand repousse l'ombre trop loin de l'objet, qui semble flotter au-dessus de son ombre — comme Peter Pan qui ne touche plus le sol. Le contact visuel entre l'objet et le sol est perdu.

Le compromis bias

Le bias est un **compromis** permanent :

- **Trop petit** → acne (rayures sur les surfaces éclairées).

- **Trop grand** → peter panning (l'ombre se détache, l'objet flotte).

En pratique on ajuste à la main jusqu'à ce que l'acne disparaisse sans que le peter panning soit visible. Les moteurs exposent aussi un **normal offset bias** (décaler la position du fragment le long de sa normale) qui corrige l'acne sans autant déplacer l'ombre — souvent préféré au bias constant.

🔧 Réglage pratique

1. Commencer avec un **slope-scaled bias** modéré (ex: 0.5–2.0 selon l'échelle de la scène).
2. Si acne visible, augmenter progressivement.
3. Si l'ombre se décolle du contact (peter panning), ajouter du **normal offset bias** plutôt que d'augmenter encore le depth bias.
4. Vérifier à **plusieurs distances** : le bon bias dépend de la résolution de la shadow map, qui change avec les cascades (Partie 7).

Partie 5 : Adoucir les Ombres — PCF

💬 Le problème des ombres nettes

Le test de base est **binaire** (éclairé ou ombré). Le bord de l'ombre est donc tranché à 1 pixel — or une vraie ombre a une **pénombre** (bord flou, voir Fig. 37). Une ombre parfaitement nette paraît artificielle, « jeu vidéo ancienne ». Pour la réalisme, il faut **filtrer** le test.

1. Percentage-Closer Filtering (PCF)

Principe

Au lieu d'un seul test au centre du texel, on effectue **plusieurs** tests dans un voisinage et on en fait la **moyenne**. Le résultat n'est plus 0 ou 1, mais une valeur dans $[0, 1]$ représentant la **fraction** du voisinage qui est ombrée. Au cœur de l'ombre → 1.0 (tout ombré). Au bord → 0.3, 0.5, 0.7 (transition douce). Loin de l'ombre → 0.0.

)

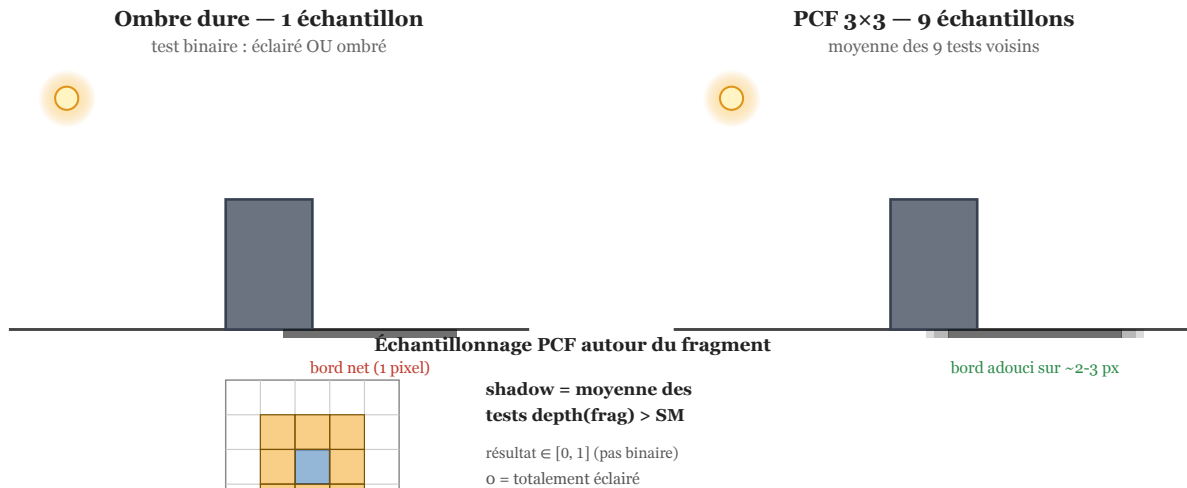


Fig. 41. – PCF : au lieu d'un test binaire (à gauche, bord net), on échantillonne la shadow map sur une grille $N \times N$ autour du fragment (à droite, ici $3 \times 3 = 9$ échantillons) et on moyenne les résultats. Le bord de l'ombre devient un dégradé. Plus N est grand, plus le bord est doux — mais plus le coût est élevé (N^2 échantillons par fragment).

PCF 3×3 en GLSL.

```
float pcf3x3(sampler2D shadowMap, vec3 s, vec2 texelSize) {
    float shadow = 0.0;
    // texelSize = vec2(1.0 / shadowMapWidth, 1.0 / shadowMapHeight)
    for (int y = -1; y <= 1; y++) {
        for (int x = -1; x <= 1; x++) {
            vec2 offset = vec2(float(x), float(y)) * texelSize;
            float stored = texture(shadowMap, s.xy + offset).r;
            shadow += (s.z - bias > stored) ? 1.0 : 0.0;
        }
    }
    return shadow / 9.0; // moyenne des 9 tests
}
```

$\text{texelSize}(\frac{1}{\text{résolution}})$ permet de se décaler d'**exactement un texel** — sinon le filtre ne fait rien. Pour un PCF 5×5, on boucle de -2 à +2 (25 échantillons), etc.

💡 Coût et compromis

- **PCF 1×1** : 1 échantillon — ombre nette (pas de filtre).
- **PCF 3×3** : 9 échantillons — bord légèrement adouci, coût modéré. Standard « minimum » acceptable.
- **PCF 5×5** : 25 échantillons — bord bien doux, coût notable.
- **PCF 9×9** : 81 échantillons — bord très doux, coût élevé (rare en temps réel).

En pratique on utilise **poisson disk sampling** (16 échantillons répartis en cercle plutôt qu'en grille) pour un meilleur rendu à coût égal, ou le **PCF hardware** (la carte graphique fait le filtrage bilinéaire des depth samples gratuitement — donne un PCF 2×2 « gratuit »).

⚠ PCF ne fait pas une vraie pénombre

PCF adoucit le bord, mais la **largeur** du bord est **constante** partout — elle ne dépend pas de la distance au bloqueur. Or dans la réalité, un objet près de son ombre a un bord **net**, un objet loin a un bord **très flou** (voir Fig. 37). PCF ignore cette physique. C'est ce que corrige PCSS (Partie 6).

Partie 6 : PCSS — Ombres Douces Physiquement Plausibles

Percentage-Closer Soft Shadows

PCSS (Fernando et al., 2005) étend PCF pour produire une **pénombre variable** : le bord de l'ombre est d'autant plus flou que le bloqueur est **loin** du receveur. Cela imite la géométrie réelle d'une source étendue (voir Fig. 37).

)

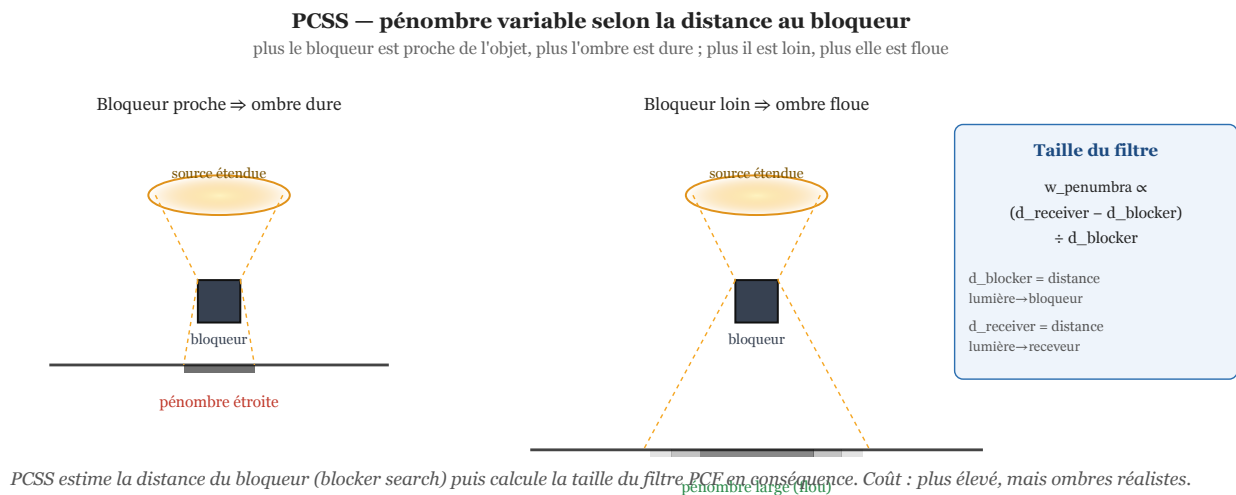


Fig. 42. – PCSS : la taille du filtre PCF dépend de la distance entre le bloqueur et le receveur. À gauche, le bloqueur est proche du sol → pénombre étroite (ombre dure). À droite, le bloqueur est loin du sol → pénombre large (ombre floue). C'est le comportement physique d'une source étendue.

💡 Les trois étapes de PCSS

1. **Blocker search** : échantillonner la shadow map sur une région (estimation de la **distance moyenne du bloqueur** d_{blocker}). On ne garde que les échantillons qui sont **effectivement** des bloqueurs (profondeur stockée < profondeur du receveur).
2. **Penumbra estimation** : calculer la taille du filtre w_{penumbra} proportionnelle à $(d_{\text{receiver}} - d_{\text{blocker}}) / d_{\text{blocker}}$ (formule de similarité de triangles). Plus le receveur est loin derrière le bloqueur, plus le filtre est large.
3. **PCF** : effectuer un PCF avec la taille w_{penumbra} calculée — un filtre **variable** par fragment.

Détail : le blocker search

Le problème : où est le bloqueur ?

PCSS veut faire l'ombre **plus floue** quand le bloqueur est loin du receveur, et **plus nette** quand il est proche. Pour ça, il faut savoir à quelle distance est le bloqueur. Mais le shader ne connaît que la position du fragment — il ne sait pas où est l'objet qui le bloque. La seule trace du bloqueur, c'est sa profondeur stockée dans la shadow map.

La solution : regarder la shadow map autour du fragment

Pour le fragment courant, on échantillonne la shadow map sur une **petite région** autour de ses coordonnées UV — par exemple une grille 5×5 (25 texels). Pour chaque texel :

- On lit la profondeur stockée d_{stored} .
- On la compare à la profondeur du fragment (le receveur) d_{receiver} .
- Si $d_{\text{stored}} < d_{\text{receiver}}$: quelque chose est **plus proche de la lumière** que le fragment → c'est un **bloqueur**. On garde cet échantillon.
- Si $d_{\text{stored}} \geq d_{\text{receiver}}$: rien ne bloque à cet endroit → on **jette** cet échantillon.

On fait la moyenne des profondeurs des échantillons **gardés** → c'est d_{blocker} , la distance moyenne du bloqueur par rapport à la lumière.

Concret : un fragment sur le sol à 20 m de la lumière. Le fragment est sur le sol, à $d_{\text{receiver}} = 20$ m de la lumière. Un cube flotte au-dessus, à 15 m de la lumière. On échantillonne 9 texels autour de la position du fragment dans la shadow map :

Texel	Prof. stockée	< 20 (receveur) ?	Bloqueur ?	Prof. gardée
1	15	oui (15 < 20)	oui	15
2	15	oui	oui	15
3	20	non (20 = 20)	non	—
4	15	oui	oui	15
5	15	oui	oui	15
6	20	non	non	—
7	15	oui	oui	15
8	15	oui	oui	15
9	20	non	non	—

6 texels sur 9 sont des bloqueurs, tous à profondeur 15. La moyenne des profondeurs gardées : $d_{\text{blocker}} = 15$ m.

Le bloqueur est à 15 m, le receveur à 20 m → le bloqueur est à 5 m au-dessus du sol. Cette distance alimente l'étape 2 (penumbra estimation) : plus l'écart est grand, plus l'ombre sera floue.

❓ Pourquoi échantillonner une région et pas un seul texel ?

Un seul texel ne donnerait qu'un point de vue. Mais le bloqueur (un cube, un personnage) couvre **plusieurs** texels dans la shadow map. Échantillonner une région capture la **moyenne** des

profondeurs du bloqueur, ce qui donne une estimation plus stable. Au bord de l'ombre — où certains texels voient le bloqueur et d'autres non — la moyenne donne une valeur de **transition**, ce qui produit un fondu progressif plutôt qu'un saut brutal.

💡 Coût et usage

PCSS est nettement plus cher que PCF (le blocker search ajoute une passe d'échantillonnage). On l'utilise pour les **ombres héros** (le personnage principal, les objets proches) plutôt que sur toutes les ombres de la scène. Unreal Engine et les moteurs modernes l'offrent en option « Soft Shadows ».

Partie 7 : Cascaded Shadow Maps (CSM)

💡 CSM, PSSM — même chose

Cette technique porte deux noms selon les moteurs :

- **Cascaded Shadow Maps (CSM)** : le nom utilisé par Microsoft (DirectX), Unity, Unreal, Three.js.
- **Parallel-Split Shadow Maps (PSSM)** : le nom de l'article académique original (Fan Zhang et al., 2006), utilisé par Godot (SHADOW_PARALLEL_2_SPLITS, SHADOW_PARALLEL_4_SPLITS).

C'est exactement la même technique — le frustum est découpé en tranches, chacune avec sa propre shadow map. Dans la suite on utilise « CSM » (le terme le plus répandu dans l'industrie), mais quand vous verrez « PSSM » ou « Parallel-Splits » dans Godot, c'est la même chose.

💡 Le problème des grandes scènes

Une shadow map unique a une **résolution fixe** pour toute la scène. Pour une lumière directionnelle (soleil) qui couvre un grand terrain, 2048^2 étalés sur $100\text{ m} \times 100\text{ m}$ donnent 20 texels par mètre — les ombres proches de la caméra sont **pixelisées** (crénelées), tandis que les ombres lointaines (qu'on ne voit pas) gaspillent de la résolution. On ne peut pas juste augmenter la résolution : 16K textures ne tiennent pas en VRAM.

)

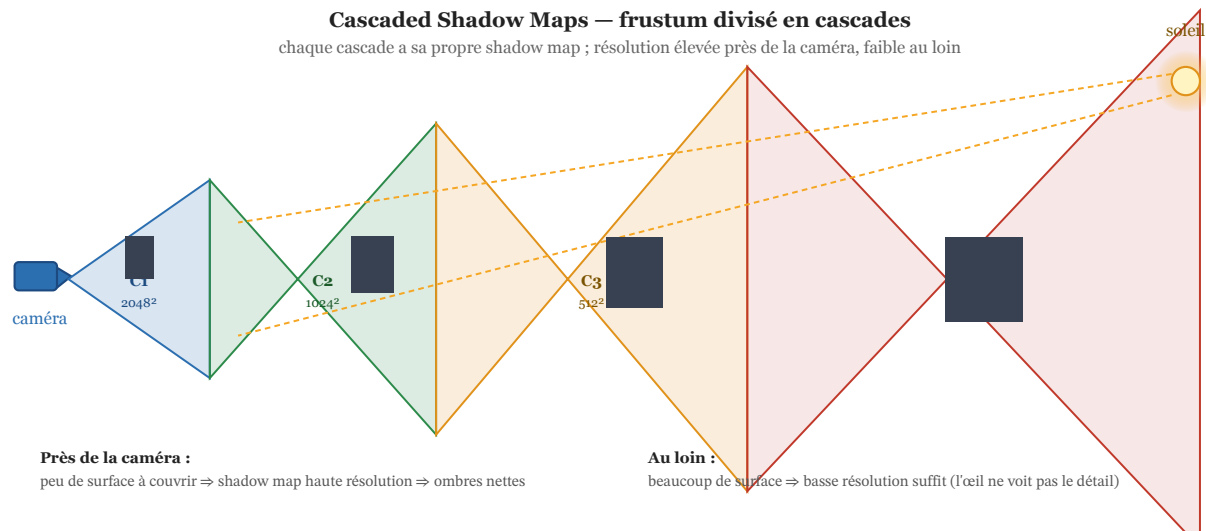


Fig. 43. — Cascaded Shadow Maps : le frustum de la caméra est divisé en **cascades** (ici 4). Chaque cascade a sa propre shadow map — haute résolution pour la cascade proche (C1, 2048²), basse résolution pour la cascade lointaine (C4, 256²). L'œil voit des ombres nettes près de lui, là où ça compte ; les ombres lointaines sont floues mais personne ne le remarque.

Principe

On découpe le frustum de la caméra en **tranches** de profondeur (les cascades), typiquement 4. Chaque cascade obtient sa propre shadow map, calculée pour **cette tranche uniquement** :

- **Cascade 1** (proche) : couvre 0–10 m, shadow map 2048² → haute densité de texels, ombres nettes.
- **Cascade 2** : 10–30 m, 1024².
- **Cascade 3** : 30–80 m, 512².
- **Cascade 4** (lointaine) : 80–200 m, 256² → basse densité, mais l'œil ne voit pas le détail.

Pendant le rendu, on choisit la cascade selon la profondeur du fragment (sa distance à la caméra). Les cascades se **chevauchent** légèrement avec un fondu pour éviter une coupure visible.

🔧 Réglages clés

- **Nombre de cascades** : 1 (pas de CSM), 2, 4 (standard), ou 8 (cinématique). Plus de cascades = plus de rendus (coût), mais meilleure qualité.
- **Split scheme** : comment découper le frustum. **Logarithmique** (plus de cascades près de la caméra) ou **practical split** (compromis logarithmique + linéaire). Le practical split est le standard.
- **Stable cascades** : on fixe la shadow map à la taille du **texel** pour éviter que l'ombre « tremble » quand la caméra bouge (sinon les texels glissent et l'ombre scintille).

⚠ Coût du CSM

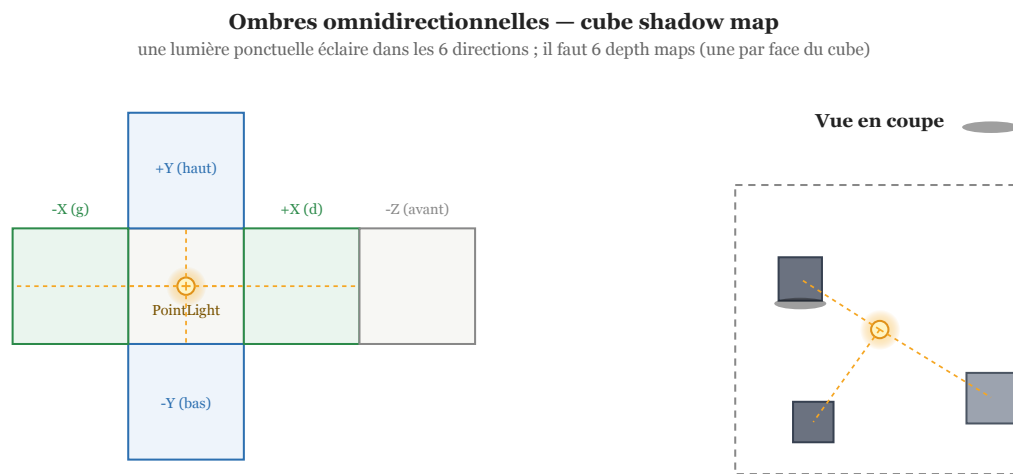
Avec 4 cascades, la passe 1 rend la scène **4 fois** (une par cascade) — c'est $4 \times$ le coût d'une shadow map unique. C'est le prix des ombres de qualité pour les scènes extérieures. En intérieur (petite scène), une shadow map unique suffit et CSM est inutile.

Partie 8 : Ombres pour Lumières Ponctuelles — Cube Shadow Maps

💡 Le problème des PointLight

Une lumière directionnelle a une **direction unique** — une shadow map 2D suffit. Mais un **PointLight** éclaire dans **toutes les directions** (Session 5). Pour capturer « ce que la lumière voit » dans toutes les directions, il faut 6 shadow maps — une par face d'un cube centré sur la lumière. C'est l'équivalent d'un **cube map** (Session 6) mais en profondeur.

)



Pour un PointLight, on rend la scène 6 fois (une par face du cube) depuis la position de la lumière. Coût : 6x plus cher qu'une shadow map directionnelle. La lumière "voit" dans les 6 directions ; chaque face du cube capture la profondeur d'une moitié.

Fig. 44. – Ombres omnidirectionnelles : un PointLight nécessite 6 depth maps (une par face du cube). À gauche, le cube déplié avec les 6 faces. À droite, vue en coupe : la lumière « voit » dans les 6 directions et capture la profondeur de tout ce qui l'entoure.

Implémentation

Deux approches :

- **6 passes** : on rend la scène 6 fois, une par face, avec 6 view matrices ($\pm X$, $\pm Y$, $\pm Z$) et la même projection (90° FOV). Simple mais coûteux (6 rendus de scène).
- **Geometry shader / layered rendering** : une seule passe, le geometry shader duplique les triangles vers les 6 faces d'une **cube depth texture** (WebGL2 / WebGPU). Plus efficace mais moins supporté.

Pendant la passe 2, le fragment calcule la **direction** de la lumière vers lui, échantillonne le cube map avec cette direction, et compare la profondeur (ici la **distance** $|l - p|$, pas la profondeur Z).

Échantillonnage d'une cube shadow map.

```
uniform samplerCube uPointShadowMap; // cube depth texture
uniform vec3 uLightPos;
in vec3 vWorldPos;

float pointShadowTest(vec3 worldPos) {
    vec3 toFrag = worldPos - uLightPos; // direction lumière → fragment
    float dist = length(toFrag); // distance réelle
    float stored = texture(uPointShadowMap, toFrag).r; // profondeur stockée
    // NB : on stocke souvent la *distance* (pas le Z) dans une cube map
    return (dist - bias > stored) ? 0.0 : 1.0;
}
```

La comparaison utilise la **distance** (longueur du vecteur) plutôt que la profondeur Z, car chaque face du cube a sa propre orientation — il n'y a pas de Z commun.

⚠ Coût et limitation

Un PointLight avec ombres coûte **6×** une lumière directionnelle avec ombres. C'est pourquoi on limite sévèrement le nombre de PointLight projetant des ombres (souvent 1 seule par scène, parfois 0). Les **SpotLight** sont préférés : ils n'ont qu'une direction, donc une seule shadow map 2D — même coût qu'une lumière directionnelle.

Partie 9 : Implémentation en Three.js

Le modèle de Three.js

Three.js (utilisé depuis la Session 8) gère les ombres via trois propriétés :

- **Render** : `renderer.shadowMap.enabled = true` active le shadow mapping globalement. `renderer.shadowMap.type` choisit le filtre par défaut (`BasicShadowMap`, `PCFShadowMap`, `PCFSoftShadowMap`, `VSMShadowMap`).
- **Lumière** : `light.castShadow = true` active la passe 1 pour cette lumière. `light.shadow.mapSize` règle la résolution, `light.shadow.bias` et `light.shadow.normalBias` corrigent l'acne.
- **Mesh** : `mesh.castShadow = true` fait projeter l'objet (passe 1), `mesh.receiveShadow = true` fait recevoir l'ombre (passe 2). Un mesh peut être les deux.

Contrairement à Babylon.js (qui a un ShadowGenerator séparé), Three.js intègre les ombres **directement** dans les lumières et les matériaux — pas d'objet supplémentaire à créer.

)

Ombres de base en Three.js.

```
// 1. Renderer – active les shadow maps globalement
renderer.shadowMap.enabled = true;
renderer.shadowMap.type = THREE.PCFSoftShadowMap; // filtre PCF doux par défaut

// 2. Une lumière directionnelle (le soleil)
const sun = new THREE.DirectionalLight(0xffff4e0, 2.0);
sun.position.set(10, 15, 8);
sun.castShadow = true; // active la passe 1
sun.shadow.mapSize.set(2048, 2048); // résolution de la shadow map
```

```
sun.shadow.camera.left = -15; // frustum de l'ombre (orthographique)
sun.shadow.camera.right = 15;
sun.shadow.camera.top = 15;
sun.shadow.camera.bottom = -15;
sun.shadow.camera.far = 60;
sun.shadow.bias = 0.0005; // corrige l'acne
sun.shadow.normalBias = 0.02; // corrige l'acne sans peter panning
scene.add(sun);

// 3. Les meshes qui projettent des ombres
cube.castShadow = true;
capsule.castShadow = true;

// 4. Les meshes qui reçoivent les ombres
ground.receiveShadow = true;
```

`castShadow = true` ajoute le mesh à la passe 1 (rendu depuis la lumière). `receiveShadow = true` active le test d'ombre dans le matériau du receveur pendant la passe 2. Un mesh peut être **les deux** (caster et receiver) — par exemple un personnage projette une ombre et s'auto-ombrage.

Le frustum d'ombre

Pour une `DirectionalLight`, `light.shadow.camera` est une caméra **orthographique** — il faut régler manuellement son `left/right/top/bottom/far` pour couvrir la zone à ombrer. Trop petit → les ombres sont coupées au bord. Trop grand → la résolution est étalée (ombres pixelisées). C'est l'équivalent du problème que les CSM (Partie 7) viennent corriger.

Filtres d'ombre en Three.js.

```
// Les 4 modes de filtrage de Three.js :
renderer.shadowMap.type = THREE.BasicShadowMap;    // 1 échantillon – ombre dure (Partie 4)
renderer.shadowMap.type = THREE.PCFShadowMap;      // PCF 2x2 hardware (Partie 5)
renderer.shadowMap.type = THREE.PCFSoftShadowMap;   // PCF avec disque de Poisson (défaut)
renderer.shadowMap.type = THREE.VSMShadowMap;      // Variance Shadow Maps (Partie 12)

// VSM : régler le radius du flou
light.shadow.radius = 8;    // force du flou gaussien sur la shadow map
```

- **BasicShadowMap** : pas de filtre — ombre nette, utile pour démo l’acné.
- **PCFShadowMap** : PCF 2×2 « gratuit » (hardware bilinear filtering des depth samples).
- **PCFSoftShadowMap** : PCF avec échantillons en disque de Poisson — bord doux, c’est le défaut.
- **VSMShadowMap** : Variance Shadow Maps (Partie 12) — la shadow map est floutée avant la comparaison, ombres très douces mais risque de **light leaking**.

Cascaded Shadow Maps en Three.js. Three.js n'a pas de CSM dans le core, mais fournit un module officiel `examples/jsm/csm/CSM.js` :

```
import { CSM } from 'jsm/csm/CSM.js';

// Désactiver l'ombre du DirectionalLight standard – CSM gère ses propres lumières
sun.castShadow = false;

const csm = new CSM({
  maxFar: 200, // distance couverte par toutes les cascades
```

```

shadowMapSize: 2048,                // résolution de chaque cascade
shadowBias: 0.0005,
shadowNormalBias: 0.02,
lightDirection: new THREE.Vector3(-10, -15, -8).normalize(),
lightColor: new THREE.Color(0xffff4e0),
lightIntensity: 2.0,
camera: camera,                    // CSM suit le frustum de la caméra
parent: scene,
cascades: 4,                       // nombre de cascades (Partie 7)
});

// Les matériaux doivent être patchés pour supporter les cascades
scene.traverse(obj => {
    if (obj.material) csm.setupMaterial(obj.material);
});

// Dans la boucle de rendu :
csm.update(); // recalcule les cascades quand la caméra bouge
    
```

Le CSM crée **N** DirectionalLight internes, chacune avec sa propre shadow map couvrant une tranche du frustum. Le setupMaterial injecte le code shader nécessaire pour choisir la bonne cascade par fragment.

🔗 Référence Three.js

- Documentation : Three.js Shadows
- CSM addon : CSM example et CSM API
- La démo interactive `examples/session10_shadows.html` illustre tous ces concepts en direct.

Partie 10 : Ombres dans Godot et Unity — Aperçu

🔗 Pourquoi parler des moteurs ?

La démo et la pratique de cette session se font en **Three.js** (code JavaScript, même framework qu'en Session 8). Mais les concepts du shadow mapping sont universels — voici comment les mêmes réglages se présentent dans Godot et Unity, au cas où vous travailleriez avec ces moteurs pour votre projet final.

Godot — DirectionalLight3D (PSSM)

Une DirectionalLight3D a une section **Shadow** dans l'inspecteur :

- **Enabled** : active le shadow mapping.
- **Shadow Mode** : Orthogonal (1 shadow map, pas de cascades), Parallel 2 Splits (PSSM à 2 cascades), Parallel 4 Splits (PSSM à 4 cascades — équivalent au CSM de la Partie 7).
- **Split 1/2/3** : positions des coupes entre cascades, en fraction de la **Shadow Max Distance** (ex: 0.1 = première cascade sur les 10% proches). Godot laisse le contrôle **manuel** des splits, contrairement à Three.js qui les calcule automatiquement (practical / logarithmic).
- **Blend Splits** : fondu entre cascades pour éviter une coupure visible (le « chevauchement » de la Partie 7).

- **Shadow Max Distance** : distance au-delà de laquelle les ombres disparaissent (la **shadow distance** de la Partie 11).
- **Fade Start** : les ombres commencent à s'estomper à cette fraction de la max distance (ex: 0.8 = fondu sur les 20% les plus lointains).
- **Shadow Depth Range** : Stable (pas de scintillement quand la caméra bouge) ou Close (précision proche).
- **Bias / Normal Bias** : réglages d'acne/peter panning (Partie 4).
- **Shadow Atlas** (Project Settings) : résolution globale des shadow maps (2048, 4096, 8192).

Godot — OmniLight3D / SpotLight3D

- OmniLight3D (= PointLight) : utilise une **cube shadow map** (6 faces). Coût élevé — limiter le nombre d'OmniLight avec ombres.
- SpotLight3D : shadow map 2D (une seule direction), beaucoup plus économique. **Préférer les SpotLight aux OmniLight** dès qu'on peut diriger la lumière.
- Réglage **Shadow > Bias** identique au DirectionalLight.

Unity — Light shadows

- **Directional Light : Shadow Type = Soft Shadows** active PCF (ou PCSS selon le pipeline). **Cascade Count** (1 à 4) règle le CSM. **Bias** et **Normal Bias** dans la section **Shadows**.
- **Point / Spot Light** : même principe, mais les PointLight utilisent une cube shadow map (coût 6×).
- **URP / HDRP** : les pipelines modernes exposent **Soft Shadows** (PCF) et **Contact Shadows** (ombres temps écran pour le contact proche, complément du shadow mapping).
- **Référence** : Unity Lights et Shadow Casting.

Partie 11 : Performance et Qualité

! Les leviers de réglage

Le shadow mapping est l'un des postes les plus coûteux du rendu temps réel. Les leviers, du plus impactant au plus subtil :

- **Nombre de lumières avec ombres** : chaque lumière avec ombre = un rendu de scène supplémentaire (×6 pour un PointLight). C'est le levier n°1 — passer de 4 à 1 lumière ombrante double souvent le FPS.
- **Résolution des shadow maps** : $1024^2 \rightarrow 2048^2$ double la VRAM et le coût de la passe 1. 4096^2 est rare, réservé aux ombres héro.
- **Filtre** : PCF 3×3 (9 échantillons) est le standard. PCSS ou PCF 9×9 sont réservés aux plateformes haut de gamme.
- **CSM** : 4 cascades = 4× le coût d'une shadow map unique. En intérieur, désactiver le CSM (1 cascade suffit).
- **Distance d'ombre** : couper les ombres au-delà d'une distance (les objets lointains n'ont pas d'ombre). Énorme gain pour les scènes extérieures.

Optimisations courantes

- **Shadow distance / fade** : les ombres s'estompent puis disparaissent au-delà d'une distance. L'œil ne voit pas une ombre à 200 m.
- **Only cast, not receive** : un petit prop peut **projeter** une ombre sans jamais en **recevoir** — on économise le test d'ombre dans son matériau.
- **Static shadow casters** : pour un objet immobile, on peut précalculer sa shadow map une fois (au chargement) au lieu de la rendre chaque frame. C'est l'équivalent du baking (Session 6) mais juste pour la shadow map.
- **Frustum culling de la passe 1** : ne rendre dans la shadow map que ce qui est dans le frustum de la lumière (et qui peut projeter sur ce que la caméra voit).

⚠ Pièges fréquents

- **Pas de receiveShadows = true** : le sol ne reçoit pas l'ombre, tout paraît éclairé. Erreur n°1 des débutants.
- **Bias non réglé** : acne partout ou peter panning visible. Toujours ajuster après avoir placé la lumière.
- **Shadow map trop petite** : ombres crénelées (pixelisées). Augmenter la résolution ou utiliser le CSM.
- **Trop de lumières avec ombres** : chute de FPS. Compter les lumières ombrantes, pas les lumières totales.
- **Scintillement quand la caméra bouge** : utiliser le **stable cascades** (Godot : **Shadow Depth Range = Stable**), ou le **texel snapping** (Three.js CSM le fait automatiquement).

Partie 12 : Variante — Variance Shadow Maps (VSM)

Le principe des VSM

Les VSM (Donnelly & Lauritzen, 2006) stockent dans la shadow map non pas la profondeur seule, mais la **profondeur et le carré de la profondeur** (z et z^2). À partir de ces deux moments statistiques, on peut estimer la **variance** et appliquer l'inégalité de Chebyshev pour obtenir une **probabilité** d'être ombré — sans faire de test binaire.

Avantage : on peut **flouter** la shadow map elle-même (blur gaussien) avant la comparaison, ce qui donne des ombres douces **très** cheap (un blur post-process au lieu de N^2 échantillons par fragment).

⚠ Light leaking

Le défaut des VSM est le **light leaking** : quand deux ombres se chevauchent (un objet proche et un objet lointain alignés), la variance augmente et l'inégalité de Chebyshev « perd » l'ombre la plus lointaine — de la lumière apparaît là où il devrait y avoir de l'ombre. C'est pourquoi les VSM sont moins utilisées que le PCF/PCSS en production, malgré leur coût attractif.

Partie 13 : Démonstration en Direct — Three.js

🔔 Déroulement de la démo

La démo se fait en **live** avec le fichier `examples/session10_shadows.html` — une scène Three.js interactive avec une GUI lil-gui. Un sol, un cube, une capsule animée (prouve le temps réel), une caisse, et trois lumières (directionnelle, spot, point) qu'on peut permuter. Une **preview de la shadow map** en coin bas-gauche montre ce que « voit » la lumière (Partie 2). On part d'une scène **sans ombres** et on ajoute les ombres étape par étape pour **voir** chaque défaut et sa correction.

💡 Lancer la démo

```
cd examples
python3 -m http.server 8123
# ouvrir http://localhost:8123/session10_shadows.html
```

Plan de la démo (45-60 min) — mapping avec les parties du cours

Phase 1 — Scène sans ombres (5 min) [Partie 1]

- Décocher **Lumière** → **Ombres activées** dans la GUI.
- Observer : tout est éclairé uniformément, le cube **flotte** visuellement (pas de contact), la capsule n'interagit pas avec le sol.
- Message : « sans ombres, on ne sait pas où sont les objets » — c'est le rôle perceptif des ombres.

Phase 2 — Activer les ombres de base (10 min) [Partie 2 & 3]

- Re-cocher **Ombres activées**. Observer le **contact** qui apparaît — l'effet est saisissant.
- Activer **Preview shadow map** : la depth texture apparaît en coin bas-gauche. C'est **ce que voit la lumière** (Passe 1). Montrer que les objets proches de la lumière apparaissent plus clairs (profondeur faible), les objets lointains plus sombres.
- Pointer l'**acne** sur le cube et le sol (rayures sombres) — c'est l'artefact de la Partie 4.

Phase 3 — Corriger l'acne (10 min) [Partie 4]

- Cliquer **Demos rapides** → **Montrer l'acne** : la GUI règle automatiquement `bias = 0`, résolution 512, filtre Basic. L'acne est **maximal** — rayures partout.
- Monter le **Depth bias** manuellement de 0 à 0.0005 : l'acne disparaît progressivement.
- Cliquer **Demos rapides** → **Montrer peter panning** : `bias = 0.02`, l'ombre se **détache** du cube (Peter Panning).
- Revenir à un bias modéré (0.0005) + **Normal bias** 0.02 : compromis optimal.

Phase 4 — Adoucir les bords (10 min) [Partie 5 & 12]

- Changer **Filtre : Dure (Basic)** → **PCF** → **PCF Soft** → **VSM**. Observer le bord de l'ombre qui passe de tranché à doux.
- **Basic** = « jeu vidéo PS2 » (Partie 4, test binaire). **PCF Soft** = le standard moderne (Partie 5). **VSM** = Variance Shadow Maps (Partie 12) — très doux mais peut avoir du light leaking.
- Changer **Résolution** : 512 → 1024 → 2048 → 4096. Observer le crénelage qui disparaît, et le FPS qui chute. La preview de la shadow map montre aussi la différence de netteté.

Phase 5 — CSM pour une grande scène (10 min) [Partie 7]

- Changer **Taille du sol** → **Grande (200×200)**. Les cubes lointains apparaissent.

- Sans CSM : les ombres proches sont pixelisées (2048^2 étalés sur 200 m = 10 texels/m).
- Cocher **Cascaded Shadow Maps** → **Activer CSM** (4 cascades). Les ombres proches deviennent nettes, les lointaines restent acceptables.
- Changer **Nb cascades** : 2 → 4 → 5. Observer la qualité et le FPS.
- Montrer que la shadow map preview ne montre plus qu'une seule cascade (la plus proche).

Phase 6 — PointLight vs SpotLight (5-10 min) [Partie 8 & 11]

- Changer **Type** → **Point (ampoule)**. Observer les ombres dans **toutes les directions** (cube, capsule, caisse projettent vers la lumière centrale). La shadow map preview montre une face de la cube map.
- Noter le FPS (coin bas-droit) — chute notable ($\times 6$ sur la passe 1).
- Changer **Type** → **Spot (projecteur)** : même qualité d'ombre directionnelle, FPS remonte (1 seule shadow map 2D).
- Message : **préférer les SpotLight aux PointLight** dès qu'on peut diriger la lumière.

⚠ Points à insister pendant la démo

- **L'effet « wow » du contact** : la première fois qu'on active les ombres, la scène « prend vie ». C'est le message pédagogique principal — les ombres ne sont pas un détail, elles sont **structurantes**.
- **La shadow map preview** : c'est le pont visuel avec la Partie 2. Les étudiants **voient** ce que la lumière voit — c'est le déclic pour comprendre le shadow mapping.
- **Le compromis bias** : le faire vivre en direct (acne → peter panning → compromis). C'est l'erreur n°1 des étudiants.
- **Le coût des PointLight** : montrer la chute de FPS. Les étudiants veulent toujours mettre des PointLight avec ombres partout.
- **La shadow distance** : un levier de perf sous-utilisé. Les ombres lointaines ne servent à rien.

Partie 14 : Pratique — Ajouter des Ombres à une Scène Three.js

💡 Objectif de la pratique

Repartir de la démo `examples/session10_shadows.html` (ou d'une scène Three.js existante des sessions précédentes) et y ajouter des ombres temps réelles de qualité. On code en **Three.js** — même framework qu'en Session 8.

💬 Ce qu'on réutilise

- **Session 5** : la scène avec lumières et matériaux Blinn-Phong.
- **Session 8** : le concept de render target (la shadow map en est un), et le framework Three.js.
- **Session 6** : la comparaison avec les ombres baked (lightmap) — ici tout est temps réel, donc dynamique.

Cahier des charges

La scène à ombrer

Une scène Three.js avec :

- **Un sol** (grand, `receiveShadow = true`).
- **Plusieurs objets** projetant des ombres (`castShadow = true`) : un cube, une capsule (personnage), un ou deux props.
- **Une lumière directionnelle** (le soleil) avec ombres — utiliser le CSM pour les grandes scènes.
- **Une lumière spot** secondaire avec ombres (lampe d'ambiance).
- **Un objet en mouvement** (la capsule qui tourne ou se déplace) pour vérifier que les ombres sont bien dynamiques.

Étapes — Three.js

Workflow Three.js.

1. Charger la scène existante (ou copier `session10_shadows.html` comme point de départ).
2. Activer les shadow maps sur le renderer : `renderer.shadowMap.enabled = true;`
`renderer.shadowMap.type = THREE.PCFSoftShadowMap;`
3. Ajouter une `DirectionalLight`, régler `castShadow = true`, `shadow.mapSize (2048)`, le frustum `shadow.camera.left/right/top/bottom`, `shadow.bias` et `shadow.normalBias`.
4. Mettre `castShadow = true` sur tous les meshes qui projettent ; `receiveShadow = true` sur le sol et les murs.
5. Ajouter une `SpotLight` secondaire avec `castShadow = true` et sa propre `shadow.mapSize (1024)` — Three.js gère plusieurs lumières ombrantes indépendamment.
6. Animer la capsule (rotation ou translation dans la boucle de rendu), observer l'ombre qui suit en temps réel.
7. Pour les grandes scènes : importer CSM from `jsm/csm/CSM.js`, créer l'instance avec 4 cascades, appeler `csm.setupMaterial(mat)` sur chaque matériau et `csm.update()` dans la boucle.
8. **Optionnel** : afficher la shadow map en coin (comme la démo) pour visualiser la passe 1.

Étapes — Godot (alternative)

Workflow Godot.

1. Ouvrir la scène, sélectionner la `DirectionalLight3D`.
2. Section **Light > Shadow** > cocher **Enabled**. Choisir **Shadow Mode = Parallel 4 Splits** (PSSM = CSM, Partie 7).
3. Régler **Bias** (0.1 par défaut, ajuster si acne) et **Normal Bias**.
4. Vérifier que les meshes ont **Cast Shadow = On** (dans **GeometryInstance3D > Geometry > Cast Shadow**). Le sol doit être **On** aussi pour **recevoir** (en Godot, un mesh peut caster et recevoir simultanément).
5. Ajouter une `SpotLight3D`, activer son ombres. Comparer le FPS avec une `OmniLight3D` équivalente — montrer la différence de coût.
6. **Project Settings > Rendering > Shadow Atlas** : régler la résolution globale (2048 ou 4096).
7. Lancer, déplacer la capsule, observer les ombres dynamiques.

Critères de réussite

Ce qui est évalué

- **Contact visible** : l'ombre est attachée à la base de chaque objet — pas de peter panning.
- **Pas d'acne** : aucune rayure sombre sur les surfaces éclairées.
- **Bord adouci** : le bord de l'ombre est un dégradé (PCF activé), pas une coupure nette.
- **Ombres dynamiques** : quand la capsule bouge, son ombre suit en temps réel — preuve que ce n'est pas du baked.
- **Performance** : la scène reste à 60 FPS avec les ombres activées. Si chute, réduire le nombre de lumières ombrantes ou la résolution des shadow maps.
- **Deux sources** : au moins une ombre directionnelle (CSM) et une ombre spot, pour montrer qu'on maîtrise les deux types.

⚠ Pièges fréquents en pratique

- **Ombre invisible** : oubli de `receiveShadow = true` (Three.js) ou **Cast Shadow = Off** sur le sol (Godot). L'ombre est calculée mais jamais affichée.
- **Ombre invisible (Three.js)** : oubli de `renderer.shadowMap.enabled = true`. Sans ça, rien ne se passe — aucune erreur, juste pas d'ombre.
- **Acne sur tout le sol** : bias trop bas. Monter le bias **progressivement** jusqu'à disparition.
- **Ombre qui clignote quand la caméra bouge** : utiliser le CSM (qui fait le texel snapping automatique) ou vérifier la stabilité du frustum d'ombre.
- **FPS qui s'effondre** : trop de lumières avec ombres, ou résolution de shadow map trop haute. Commencer à 1024 et monter.
- **Ombre coupée au bord** : le frustum de la lumière est trop petit. Élargir `shadow.camera.left/right/top/bottom` (Three.js) ou la **shadow far** (Godot).

💡 Bonus

- **Comparaison baked vs temps réel** : reprendre la scène de la Session 9 (lightmap) et la même scène en ombres temps réel. Comparer la qualité (le baked gagne en douceur) et le FPS (le baked gagne en perf) — mais le temps réel gagne en **dynamisme** (objets en mouvement).
- **VSM** : essayer `renderer.shadowMap.type = THREE.VSMShadowMap` (Partie 12) et régler `light.shadow.radius`. Observer le **light leaking** quand deux ombres se chevauchent.
- **Contact shadows** : ajouter un effet SSAO (Partie 6 de la Session 6) pour renforcer le contact proche, en complément du shadow mapping.

Conclusion

💡 Récapitulatif

Le **shadow mapping** est la technique universelle des ombres temps réel, fondée sur une idée simple — **rendre la scène depuis la lumière pour savoir ce qu'elle voit**. Ses difficultés sont toutes dans les **détails** :

- L'**acne** et le **peter panning** (compromis de bias).

- Le **bord net** (corrigé par PCF, puis PCSS pour la pénombre variable).
- La **résolution** sur les grandes scènes (corrigée par les **cascades** CSM).
- Le **coût des lumières ponctuelles** (cube shadow maps, 6× plus chères).

Combiné avec le **baking** (Session 6) pour la géométrie statique, le shadow mapping temps réel pour les objets dynamiques donne la stratégie d'éclairage moderne : **statique précalculé, dynamique calculé**. C'est ce que font tous les moteurs professionnels — Three.js, Unity, Unreal, Godot — et c'est ce qu'on met en place dans la pratique de cette session avec la démo `examples/session10_shadows.html`.

Session 11 : Post-Processing et Render Targets

💡 Objectifs de la session

On a déjà croisé les render targets deux fois : en **Session 8** (miroir, eau) et en **Session 10** (la shadow map est un render target de profondeur). Aujourd'hui on les utilise pour le **post-processing** — appliquer des effets sur l'image finale **après** le rendu de la scène. C'est ce qui donne le « look » d'un jeu : tone mapping, bloom, flou de profondeur, vignette, corrections colorimétriques. On va voir le pipeline, les effets principaux, et l'implémentation en Three.js.

💡 Rappel : ce qu'on sait déjà

- **Session 8** : un render target est une texture dans laquelle le GPU rend au lieu d'afficher. Pipeline en deux passes : rendre dans la texture, puis l'utiliser.
- **Session 10** : la shadow map est un render target de profondeur — on rend la scène depuis la lumière, on ne garde que le z .
- **Session 5** : le fragment shader calcule la couleur finale de chaque pixel. Mais cette couleur est écrite dans le framebuffer — et si on écrivait dans une texture d'abord ?

Partie 1 : Le Pipeline de Post-Processing

1. Pourquoi rendre dans une texture ?

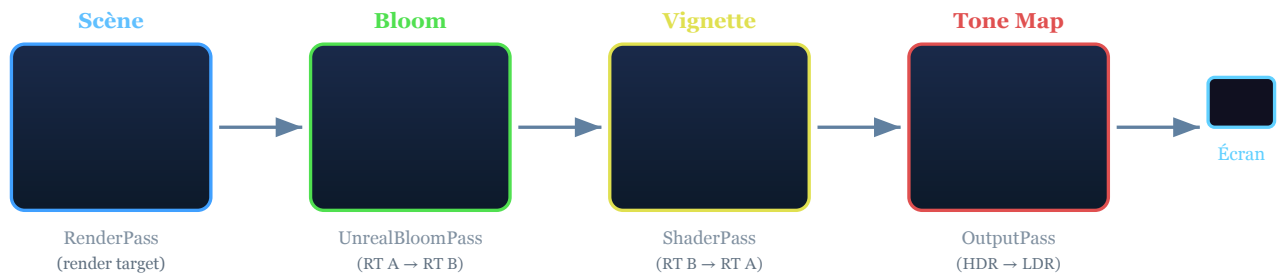
💬 Le problème

Le fragment shader produit une couleur par pixel et l'écrit directement à l'écran. Une fois affichée, cette couleur est **perdue** — on ne peut plus la modifier. Si on veut appliquer un effet (flou, bloom, tone mapping), il faut d'abord capturer l'image dans une texture, puis la traiter dans un second passage.

Le pipeline standard

1. **Passe 1 — Rendu de la scène** : la scène est rendue normalement, mais au lieu d'afficher à l'écran, on écrit dans un **render target** (couleur + profondeur).
2. **Passe 2+ — Effets** : un shader plein écran (un simple quad qui couvre tout l'écran) lit le render target, applique un effet, et écrit le résultat dans un **autre** render target.
3. **Passe finale** : le dernier render target est affiché à l'écran.

On enchaîne les effets comme une chaîne de montage : chaque effet lit l'image précédente, la transforme, produit l'image suivante.



Ping-pong : les effets alternent entre deux render targets

Fig. 45. – Pipeline de post-processing. La scène est rendue dans un render target, puis chaque effet lit l'image précédente et écrit dans la suivante. Le dernier résultat est affiché à l'écran.

2. Le fullscreen quad

Comment appliquer un shader sur toute l'image ?

On rend un simple rectangle (quad) qui couvre tout l'écran. Son vertex shader est trivial — il place les 4 coins aux bords de l'écran. Le fragment shader de ce quad est l'effet : il lit la texture d'entrée (l'image précédente) à la coordonnée UV du pixel courant, la transforme, et écrit le résultat.

C'est juste un mesh normal avec un matériau qui a la texture d'entrée en `sampler2D`. La géométrie n'a pas d'importance — seul le fragment shader compte.

Vertex shader du fullscreen quad.

```
// Deux triangles qui couvrent l'écran, UV en [0,1]
in vec2 aPosition; // [-1,1] en NDC
out vec2 vUV;

void main() {
    vUV = aPosition * 0.5 + 0.5; // [-1,1] → [0,1]
    gl_Position = vec4(aPosition, 0.0, 1.0);
}
```

3. Ping-pong rendering

Deux render targets, on alterne

On utilise deux render targets (A et B). L'effet 1 lit A, écrit dans B. L'effet 2 lit B, écrit dans A. On « ping-pong » entre les deux — pas besoin de créer une texture par effet.

$$RT_A \xrightarrow{\text{effet 1}} RT_B \xrightarrow{\text{effet 2}} RT_A \xrightarrow{\text{effet 3}} RT_B \xrightarrow{\text{afficher}} \text{écran}$$

Partie 2 : Les Effets Principaux

1. Tone Mapping — HDR vers LDR

! Le problème des couleurs > 1

En rendu HDR (High Dynamic Range), les couleurs peuvent dépasser 1.0 — une lumière brillante peut produire une valeur de 5.0 ou 10.0. Mais l'écran n'affiche que [0, 1]. Sans conversion, tout ce qui dépasse 1.0 est **écrêté** (clamped à 1.0) → les hautes lumières apparaissent comme des taches blanches plates, sans détail.

Le **tone mapping** compresse les valeurs HDR vers [0, 1] tout en préservant les détails dans les hautes et basses lumières.

Les deux opérateurs courants

Reinhard — le plus simple :

$$\text{color}_{\text{LDR}} = \text{color}_{\text{HDR}} / (\text{color}_{\text{HDR}} + 1)$$

Écrase doucement les hautes lumières. Rapide, mais peut donner un rendu « plat » — il comprime trop au milieu.

ACES (Academy Color Encoding System) — le standard du cinéma :

$$\text{color}_{\text{LDR}} = \text{ACES}(\text{color}_{\text{HDR}})$$

Préserve plus de contraste et de saturation dans les hautes lumières. C'est le tone mapper par défaut dans la plupart des moteurs modernes (Unreal, Unity, Godot, Three.js).

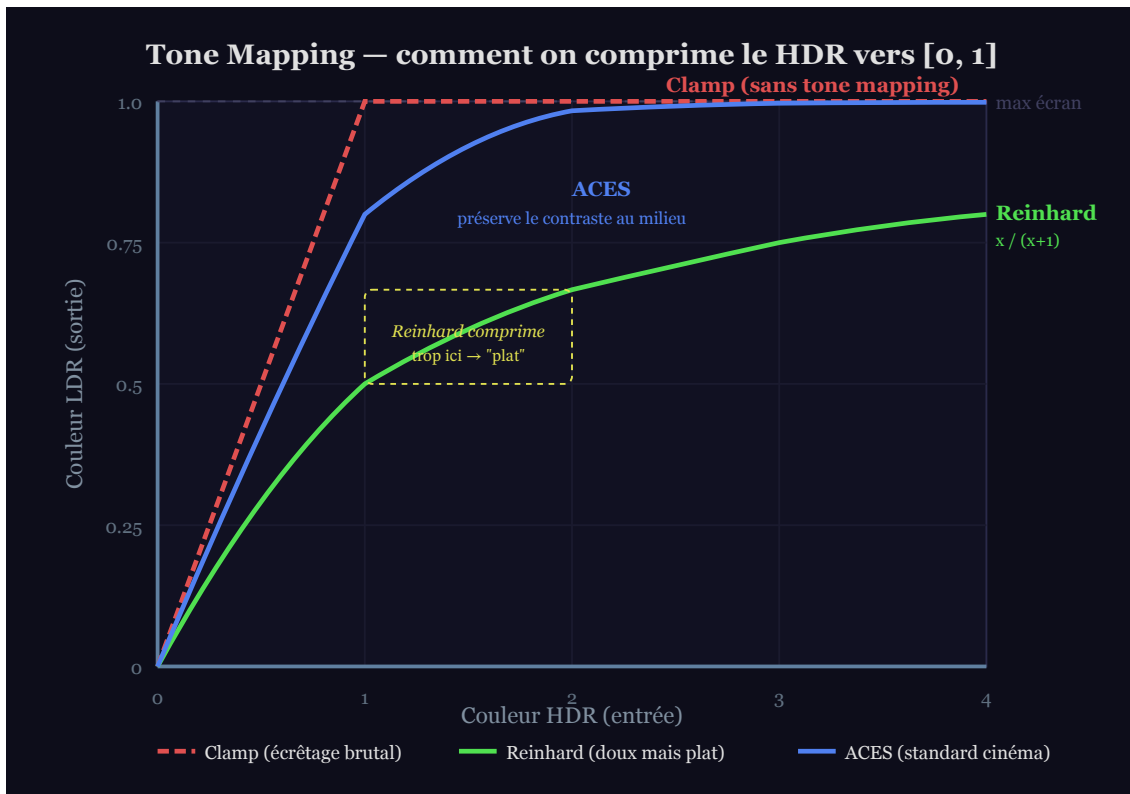


Fig. 46. – Courbes de tone mapping. Le **clamp** (rouge) écrête brutalement à 1.0 — les détails des hautes lumières sont perdus. **Reinhard** (vert) comprime doucement mais trop au milieu → rendu « plat ». **ACES** (bleu) reste linéaire plus longtemps, puis comprime en douceur — préserve le contraste.

Tone mapping en GLSL.

```
// Reinhard
vec3 reinhard(vec3 hdr) {
    return hdr / (hdr + 1.0);
}

// ACES (version simplifiée)
vec3 aces(vec3 x) {
    float a = 2.51, b = 0.03, c = 2.43, d = 0.59, e = 0.14;
    return clamp((x * (a * x + b)) / (x * (c * x + d) + e), 0.0, 1.0);
}
```

💡 Quand fait-on le tone mapping ?

Toujours en **dernier** — après tous les autres effets. Le tone mapping convertit l'HDR en LDR ; une fois fait, on est en [0,1] et les effets suivants n'auraient plus de plage dynamique pour travailler.

2. Gamma Correction — linéaire vs sRGB

Le piège classique

Les écrans affichent en sRGB (gamma ≈ 2.2). Les calculs d'éclairage doivent se faire en **linéaire**. Si on oublie la conversion, les dégradés sont trop sombres au milieu et les lumières trop saturées.

$$\text{linéaire} \xrightarrow{1/2.2} \text{sRGB} \quad (\text{affichage})$$

$$\text{sRGB} \xrightarrow{2.2} \text{linéaire} \quad (\text{calcul})$$

! En pratique

En Three.js, si le `renderer.outputColorSpace` est `SRGBColorSpace` (le défaut), la conversion gamma est faite automatiquement à la fin. En Babylon.js, c'est `BABYLON.ColorSpace.SRGB`. Il faut juste s'assurer que les textures diffusent sont marquées comme sRGB et que les calculs se font en linéaire.

3. Bloom — l'halo lumineux

! L'effet le plus reconnaissable

Le bloom fait « rayonner » les zones lumineuses — une fenêtre au soleil, une lampe, une explosion. Sans bloom, les lumières brillantes sont des pixels blancs plats. Avec bloom, elles débordent en halo doux autour de la source.

Le pipeline du bloom

1. **Bright pass** : extraire les pixels dont la luminance dépasse un seuil (ex. > 1.0). On obtient une image ne contenant que les parties brillantes.
2. **Blur** : flouter cette image (gaussian blur, plusieurs passes pour un flou large).
3. **Composite** : additionner l'image floutée à l'image originale.

$$\text{image}_{\text{finale}} = \text{image}_{\text{scène}} + \text{image}_{\text{bright}} \xrightarrow{\text{blur}} \text{halo}$$

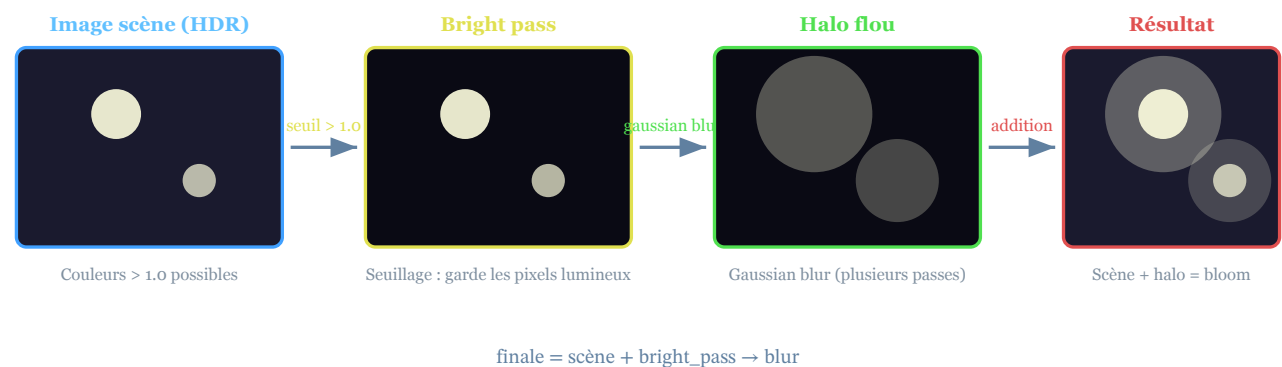


Fig. 47. — Le bloom en trois étapes : extraction des parties brillantes (bright pass), flou gaussien, puis addition à l'image originale.

💡 Pourquoi plusieurs passes de flou ?

Un gaussian blur sur un grand rayon coûte cher (beaucoup d'échantillons). On fait plusieurs passes avec un petit noyau (ex. 9×9) en alternant horizontal/vertical — c'est la **séparabilité** du gaussien. Pour un bloom large, on peut aussi **downsampler** l'image (la réduire en résolution), flouter, puis l'upsampler.

4. Depth of Field — le flou de profondeur

Principe

La caméra a une **focale** et un **ouverture**. En photographie, les objets hors de la zone nette (le **focal range**) sont flous. En temps réel, on reproduit cet effet en floutant les fragments dont la profondeur (le z du depth buffer) s'éloigne de la distance focale.

$$\text{flou} = \max(0, |z - \text{focalDistance}| - \text{focalRange})$$

💡 On réutilise le depth buffer

Le depth buffer (Session 3, Session 10) est disponible comme texture dans le post-processing. On lit z au pixel courant, on calcule la distance au focal point, et on ajuste le rayon de flou. C'est le même depth buffer que pour le shadow mapping — on l'a déjà, il est gratuit.

5. Screen-Space Ambient Occlusion (SSAO)

Principe

L'AO (Session 6) assombrit les crevasses et coins où la lumière ambiante peine à arriver. Le SSAO le calcule en temps réel, dans l'espace écran, en échantillonnant le depth buffer autour de chaque pixel. Si beaucoup de voisins sont plus proches (crevasse), le pixel est assombri.

💡 Connexion avec la Session 6

La Session 6 a vu l'AO **baked** (précalculé dans une texture, statique). Le SSAO est la version **temps réel** — dynamique, mais approximative et plus coûteuse. Beaucoup de jeux combinent les deux : AO baked pour la géométrie statique + SSAO pour les objets dynamiques.

6. Autres effets courants

Effets simples (un seul passage)

- **Vignette** : assombri les coins de l'image.

$$\text{color} * = 1.0 - \text{vignette}(\text{uv})$$

- **Color grading / LUT** : appliquer une table de correspondance de couleurs (look cinématique, désert, nuit).

- **Chromatic aberration** : décaler légèrement R, V, B (effet de lentille bon marché).
- **Film grain** : ajouter du bruit aléatoire (effet vintage).
- **Fog** : mélanger avec une couleur de brouillard selon la profondeur.

Partie 3 : Implémentation en Three.js

1. EffectComposer — la chaîne de montage

Le pattern de Three.js

Three.js fournit EffectComposer qui gère le ping-pong rendering automatiquement. On ajoute des **passes** — chaque pass est un effet. Le composer enchaîne les passes et affiche le résultat final.

```
import { EffectComposer } from 'three/addons/postprocessing/EffectComposer.js';
import { RenderPass } from 'three/addons/postprocessing/RenderPass.js';
import { UnrealBloomPass } from 'three/addons/postprocessing/UnrealBloomPass.js';

const composer = new EffectComposer(renderer);
composer.addPass(new RenderPass(scene, camera)); // passe 1 : scène
composer.addPass(new UnrealBloomPass(resolution, strength, radius, threshold));
// ... ajouter d'autres passes ...

// Dans la boucle de rendu :
composer.render(); // au lieu de renderer.render(scene, camera)
```

2. Les passes intégrées

Pass disponibles dans Three.js

- RenderPass — rend la scène dans le render target (toujours en premier).
- UnrealBloomPass — bloom de style Unreal Engine.
- ShaderPass — passe custom avec votre propre shader.
- BokehPass — depth of field.
- SAOPass — screen-space ambient occlusion.
- OutputPass — tone mapping + gamma (toujours en dernier).

3. ShaderPass — un effet custom

Vignette en ShaderPass.

```
import { ShaderPass } from 'three/addons/postprocessing/ShaderPass.js';

const vignettePass = new ShaderPass({
  uniforms: {
    tDiffuse: { value: null }, // image d'entrée (automatique)
    offset: { value: 1.0 },
    darkness: { value: 1.0 }
  },
  vertexShader: `
    varying vec2 vUV;
    void main() {
      vUV = uv;
    }
  `
});
```

```

        gl_Position = projectionMatrix * modelViewMatrix * vec4(position, 1.0);
    }
},
fragmentShader: `
    uniform sampler2D tDiffuse;
    uniform float offset;
    uniform float darkness;
    varying vec2 vUV;

    void main() {
        vec4 color = texture2D(tDiffuse, vUV);
        // Distance au centre
        vec2 uv = (vUV - 0.5) * offset;
        float vignette = 1.0 - dot(uv, uv) * darkness;
        gl_FragColor = color * vignette;
    }
`);

composer.addPass(vignettePass);

```

💡 Convention Three.js

tDiffuse est le nom conventionnel pour l'image d'entrée dans un ShaderPass. Le EffectComposer la remplit automatiquement avec le render target de la pass précédente. Ne pas la renommer.

4. Ordre des passes

💡 L'ordre compte

L'ordre des passes affecte le résultat :

1. RenderPass — toujours en premier.
2. Effets qui travaillent en HDR (bloom, DoF, SSAO) — **avant** le tone mapping.
3. OutputPass (tone mapping + gamma) — **toujours en dernier**.

Si on met le tone mapping avant le bloom, le bloom travaille en LDR et les halos seront plats. Si on le met après, le bloom peut exploiter la plage HDR complète.

Partie 4 : Performance et Bonnes Pratiques

Coût du post-processing

Chaque pass lit et écrit une texture plein écran. En 1080p, c'est $\sim 2M$ pixels par pass. Le coût principal est la **bande passante mémoire** (lire/écrire la texture), pas le calcul.

- **Réduire la résolution** : certains effets (bloom, SSAO) peuvent tourner à demi-résolution — visuellement identique, 4× moins de pixels.
- **Combiner les effets** : vignette + color grading + grain dans un seul shader au lieu de trois passes.
- **Éviter les passes inutiles** : si un effet est constant (ex. vignette fixe), on peut le baker dans une LUT.

💡 Le fullscreen quad est gratuit

Le vertex shader du fullscreen quad ne traite que 4 à 6 vertices — négligeable. C'est le fragment shader (un par pixel) qui coûte. Donc la complexité du fragment shader est ce qui compte, pas la géométrie.

Partie 5 : Démonstration en Direct — Three.js

💬 Démo interactive

On part de la démo de shadows de la Session 10 et on ajoute du post-processing :

1. Tone mapping ACES (via `OutputPass`).
2. Bloom sur les zones lumineuses (`UnrealBloomPass`).
3. Vignette custom (`ShaderPass`).
4. Un slider pour ajuster l'intensité du bloom en temps réel.

On voit l'effet de chaque pass sur l'image finale, et l'impact sur le framerate.

Partie 6 : Pratique — Ajouter du Post-Processing

Cahier des charges. Reprendre une scène Three.js (la vôtre ou celle de la Session 10) et y ajouter :

1. Tone mapping (ACES ou Reinhard).
2. Bloom avec un seuil réglable.
3. Au moins un effet custom (vignette, color grading, ou chromatic aberration).
4. Un contrôle GUI pour ajuster les paramètres en temps réel.

Étapes — Three.js

1. Installer `EffectComposer` et les passes nécessaires.
2. Remplacer `renderer.render()` par `composer.render()`.
3. Ajouter `RenderPass` en premier, `OutputPass` en dernier.
4. Insérer les effets entre les deux.
5. Pour un effet custom, créer un `ShaderPass` avec un shader GLSL.
6. Ajuster les paramètres via `lil-gui` ou `dat.gui`.

Critères de réussite

1. Le tone mapping est visible (les hautes lumières ne sont plus écrêtées).
2. Le bloom est visible sur les sources lumineuses.
3. L'effet custom est visible et paramétrable.
4. Le framerate reste acceptable (≥ 30 FPS).
5. L'ordre des passes est correct (tone mapping en dernier).

Conclusion

💬 Ce qu'on a vu

- Le post-processing applique des effets sur l'image **après** le rendu de la scène.
- Le pipeline : rendre dans un render target → enchaîner les effets (ping-pong) → afficher.

- Les effets clés : tone mapping (HDR→LDR), bloom (halo lumineux), DoF (flou de profondeur), SSAO (occlusion ambiante).
- En Three.js : EffectComposer + passes (RenderPass, UnrealBloomPass, ShaderPass, OutputPass).
- L'ordre des passes compte : tone mapping toujours en dernier.

Le post-processing est la couche finale du rendu — il transforme une image techniquement correcte en une image **belle**. C'est ce qui donne le « look » d'un jeu.

💡 La prochaine session

En Session 12, on regarde **sous le capot** : WebGPU, la nouvelle API graphique du web. On a utilisé Babylon.js et Three.js qui abstraient le GPU — maintenant on voit ce qu'il y a en dessous, et pourquoi WebGPU change la donne (compute shaders, performance, accès direct au GPU).

Session 12 : Optimisation — Structures Spatiales, LOD et Billboards

🔔 Objectifs de la session

Jusqu'ici, chaque session a ajouté de la **qualité** : éclairage, textures, ombres, post-processing. Aujourd'hui on s'attaque au problème inverse : **comment garder 60 FPS quand la scène grandit**. Une forêt avec 10 000 arbres, une ville avec 500 bâtiments, un champ d'herbe avec 50 000 brins — si on rend tout, le GPU s'effondre. Il y a deux leviers : **ne pas dessiner ce qu'on ne voit pas** (culling, structures spatiales) et **dessiner du faux pas cher** (LOD, billboards, impostors). C'est la session qui fait passer d'une démo à un jeu.

🔔 Rappel : ce qu'on sait déjà

- **Session 2** : le scene graph organise les objets en hiérarchie parent-enfant. Le culling parcourt ce graphe pour décider quoi rendre.
- **Session 7** : une texture est une image appliquée sur une surface. Le billboard est une texture sur un quad — la « 3D » est dans l'image, pas dans la géométrie.
- **Session 8** : les render targets permettent de rendre dans une texture. L'impostor utilise cette technique pour capturer un mesh dans une image.
- **Physique** : vous avez déjà vu les octrees et BVH pour le **broadphase collision** — on les retrouve ici, mais la question change : « est-ce visible ? » au lieu de « est-ce en collision ? ».

Partie 1 : Le Problème des Grandes Scènes

💡 Le coût du rendu

Chaque objet rendu coûte :

- **CPU** : calculer les matrices, tester la visibilité, préparer les draw calls. C'est le **batch** — soumettre la géométrie au GPU.
- **GPU** : traiter les vertices (vertex shader), rasteriser, traiter les fragments (fragment shader), écrire dans le framebuffer.

Avec 10 000 objets, le CPU passe son temps à préparer des draw calls, et le GPU à traiter des fragments sur des objets invisibles. Les deux sont des goulots.

Les deux leviers

1. **Côté CPU — ne pas soumettre** : si un objet n'est pas dans le champ de la caméra, ne l'envoie même pas au GPU. C'est le **culling**.
2. **Côté GPU — dessiner moins cher** : si un objet est loin, il est petit à l'écran. Remplace sa géométrie par une version simplifiée (LOD) ou par une simple image (billboard / impostor).

FPS = 60 ← ne rien faire pour ce qu'on ne voit pas, et faire simple pour ce qu'on voit mal

Partie 2 : Culling — Ne Pas Dessiner l'Invisible

1. Frustum Culling

Le frustum

La caméra voit un volume en forme de pyramide tronquée — le **frustum** (Session 2). Tout ce qui est en dehors de ce volume n'apparaîtra jamais à l'écran. Le **frustum culling** élimine ces objets **avant** de les envoyer au GPU.

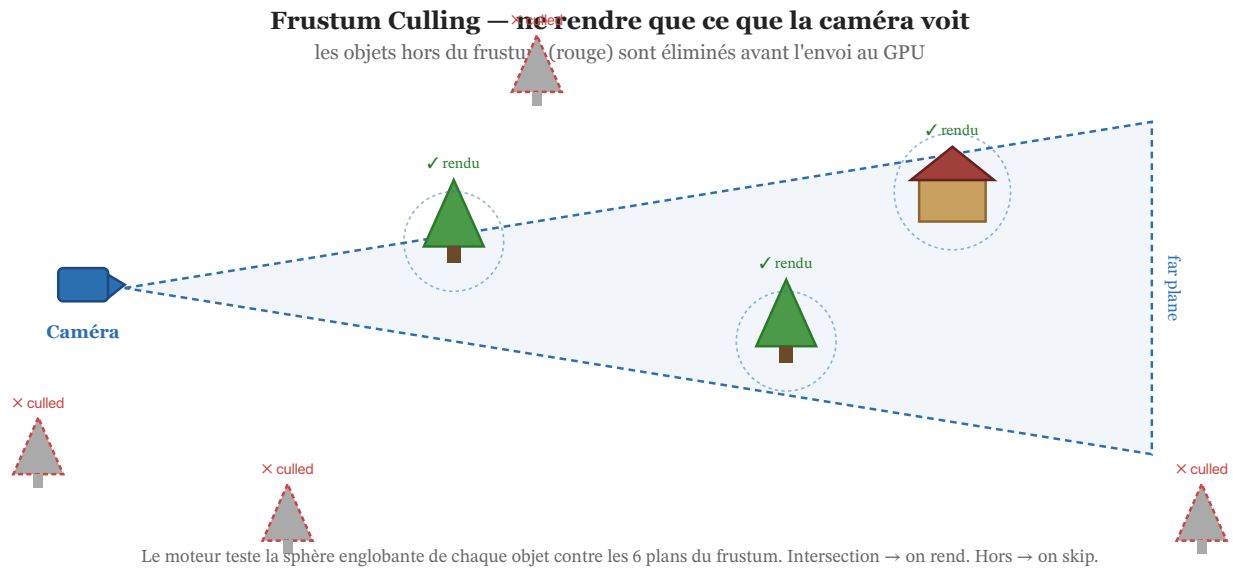


Fig. 48. – Frustum culling. Les objets verts (dans le frustum) sont rendus ; les objets rouges (hors frustum) sont éliminés sur le CPU. Le moteur teste la sphère englobante (bounding sphere) de chaque objet contre les 6 plans du frustum.

💡 Bounding volumes

Pour tester rapidement si un objet est dans le frustum, on l'enveloppe dans une forme simple — **bounding sphere** (une sphère) ou **AABB** (Axis-Aligned Bounding Box). Tester une sphère contre 6 plans est 6 produits scalaires — quasi gratuit. Si la sphère est entièrement hors d'un plan, l'objet est culled.

2. Backface Culling

Déjà vu en Session 1

Le GPU élimine les triangles dont la normale pointe **away** de la caméra. C'est gratuit — c'est un paramètre du pipeline (`gl.enable(GL_CULL_FACE)`). On l'a vu avec l'ordre des indices : `[0, 2, 1]` inverse le winding et rend le triangle visible depuis l'autre côté.

3. Occlusion Culling

! Le plus difficile

Le frustum culling élimine ce qui est hors champ. Mais un objet **dans** le frustum peut être **caché** derrière un mur. Le **occlusion culling** détecte ces cas. C'est beaucoup plus dur :

- **Portals** : diviser la scène en pièces (portails). Si le portail est hors de vue, toute la pièce derrière est culled. Utilisé dans les jeux en intérieur.
- **GPU occlusion queries** : demander au GPU « est-ce que cet objet a des pixels visibles ? » avant de le rendre. Coûteux, souvent fait avec un délai.
- **Hi-Z buffer** : utiliser une version basse résolution du depth buffer pour tester l'occlusion sur le CPU.

La plupart des moteurs utilisent une combinaison. C'est un sujet avancé — on le mentionne, on ne l'implémente pas.

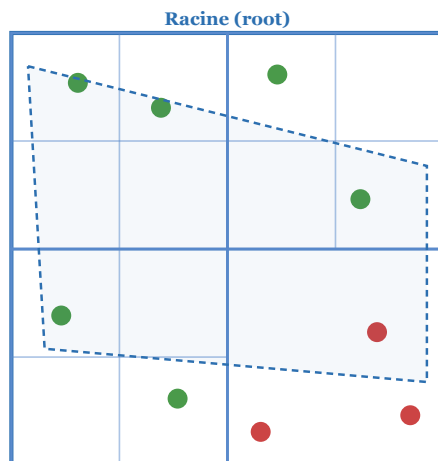
4. Structures Spatiales — Octree, BVH, Quadtree

Le problème du culling naïf

Sans structure, on teste chaque objet contre le frustum : $O(N)$ pour N objets. Avec 10 000 objets, c'est 10 000 tests par frame — sur le CPU, c'est un coût non négligeable. On peut faire mieux en **organisant** les objets dans l'espace.

Octree — subdiviser l'espace pour culer par groupes

un test rejette un sous-espace entier au lieu de tester chaque objet



Comment ça marche

1. On divise l'espace en 8 cellules (octree) ou 4 (quadtree en 2D)
2. Chaque objet est placé dans la cellule qui le contient
3. Si une cellule est entièrement hors du frustum → on skip toute la sous-branche
4. On ne teste que les cellules qui intersectent le frustum

Coût du culling :

Sans structure : $O(N)$ — tester les 10 000 objets

Avec octree : $O(\log N)$ — ~14 tests pour 10 000

Même structure qu'en physique (broadphase collision) — mais la question est "est-ce visible ?" au lieu de "est-ce en collision ?"

Fig. 49. — Octree : l'espace est divisé récursivement en 8 cellules (4 en 2D = quadtree). Chaque objet est placé dans la cellule qui le contient. Si une cellule entière est hors du frustum, on skip toute sa sous-branche — un seul test élimine des centaines d'objets.

Les structures courantes

- **Octree** : divise l'espace 3D en 8 cellules par niveau. Simple, adaptatif. Bon pour les scènes avec distribution variable.
- **BVH** (Bounding Volume Hierarchy) : arbre de bounding volumes (sphères ou AABB). Chaque nœud contient ses enfants. Bon pour les scènes avec des objets de tailles variées.
- **Quadtree** : version 2D de l'octree — 4 cellules par niveau. Utilisé pour les terrains.
- **Spatial hashing** : grille régulière avec hash. Très simple, excellent pour les objets uniformément distribués.
- **BSP** (Binary Space Partition) : division par plans. Historique (Quake), encore utilisé pour les niveaux en intérieur.

💡 Connexion avec le cours de physique

Vous avez déjà rencontré ces structures en physique pour le **broadphase collision** — l'octree ou le sweep-and-prune réduisent les $O(N^2)$ paires potentielles. En rendu, c'est la même structure, mais la requête est différente : « quels objets sont dans le frustum ? » au lieu de « quels objets sont proches les uns des autres ? ». La structure est identique, la fonction de requête change.

💡 En pratique : le moteur le fait déjà

Babylon.js et Three.js font le frustum culling automatiquement sur chaque mesh, en utilisant la bounding sphere. Pour les scènes très grandes, Three.js fournit `BatchedMesh` et Babylon.js fournit `Thin Instances` pour réduire les draw calls. Les structures spatiales custom (octree, BVH) sont rarement nécessaires dans un projet étudiant — mais comprendre **pourquoi** le moteur les utilise est essentiel pour diagnostiquer les problèmes de performance.

Partie 3 : LOD — Level of Detail

Le principe

Un objet lointain occupe peu de pixels à l'écran. Dessiner ses 17 000 triangles pour qu'ils couvrent 20 pixels est du gaspillage. Le **LOD** (Level of Detail) remplace la géométrie par une version simplifiée selon la distance à la caméra.

LOD — Level of Detail : moins de triangles au loin

l'objet lointain est petit à l'écran → on utilise une version simplifiée, visuellement identique

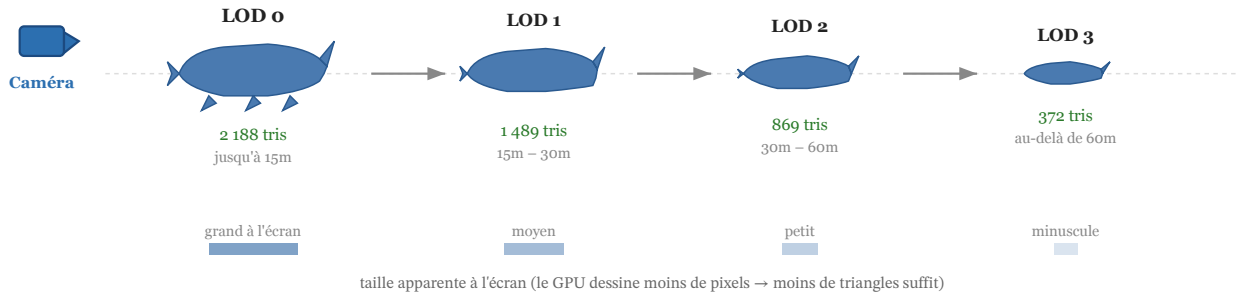


Fig. 50. – LOD sur un requin. LOD 0 (2 188 tris) près de la caméra, LOD 1 (1 489) à distance moyenne, LOD 2 (869) loin, LOD 3 (372) très loin. La taille apparente diminue, le nombre de triangles aussi — visuellement identique à l'écran.

1. Comment créer les niveaux de détail

Trois approches

- **Manuelle** : l'artiste modélise plusieurs versions. Qualité maximale, mais coûteux en temps de production.
- **Décimation** : un outil réduit automatiquement le nombre de triangles en fusionnant les vertices proches. Dans Blender, c'est le modifier **Decimate**. Rapide, qualité correcte.
- **Procedural** : générer les LOD à l'import (glTF peut contenir plusieurs meshes, ou le moteur les génère automatiquement). Godot et Unreal font ça automatiquement.

🔗 Notre approche pour la démo

On a préparé un asset de requin avec 4 niveaux de LOD dans Blender, à partir d'un mesh retopologisé (Shark_low, 1 094 polys de base, 2 188 avec le modifier Mirror appliqué). En utilisant le modifier **Decimate** avec des ratios de 0.5, 0.25 et 0.1, on obtient 4 niveaux. Le tout est exporté dans un seul fichier shark_lod.glb qui contient les 4 meshes avec textures. Dans Three.js, on charge le GLB et on construit un objet THREE.LOD qui sélectionne le bon mesh selon la distance.

2. Quand passer d'un LOD à l'autre

💡 Le seuil de distance

Chaque LOD a une **distance de transition** — la distance à laquelle on passe au niveau suivant. Trop tôt : on voit le pop-in (changement brusque de géométrie). Trop tard : on gaspille des triangles.

Règle empirique : baser le seuil sur la **taille apparente à l'écran** (en pixels), pas sur la distance brute. Un objet de 1 m à 10 m et un objet de 100 m à 1000 m ont la même taille apparente.

Pop-in et comment l'atténuer

Le **pop-in** est le changement brusque visible quand on passe d'un LOD à l'autre. Solutions :

- **Seuils bien choisis** : assez loin pour que la différence soit imperceptible.
- **LOD cross-fade** : fondre entre deux LOD sur quelques frames (dither ou alpha blend). Three.js supporte ça via `lod.fadeType`.
- **Plus de niveaux** : 4-5 LODs réduisent le pop-in par rapport à 2.
- **Impostor au dernier niveau** : remplacer le LOD le plus lointain par un billboard (Partie 4).

3. LOD dans les moteurs

Support natif

- **Three.js** : `THREE.LOD` — on ajoute des meshes avec `addLevel(mesh, distance)`. Le moteur sélectionne automatiquement.
- **Babylon.js** : `detailMap` ou `meshLODLevel` avec `addLODLevel(distance, mesh)`.
- **Godot** : génère les LOD automatiquement à l'import du glTF.
- **Unreal / Unity** : systèmes de LOD intégrés, génération automatique avec paramètres de qualité.

Partie 4 : Billboards — la 3D dans une Texture

Le billboard

Un **billboard** est un quad (2 triangles) qui **fait toujours face à la caméra**. La géométrie est plate — la « 3D » est dans la texture appliquée. Coût : 4 vertices, 2 triangles, 1 texture. C'est l'objet 3D le moins cher possible.

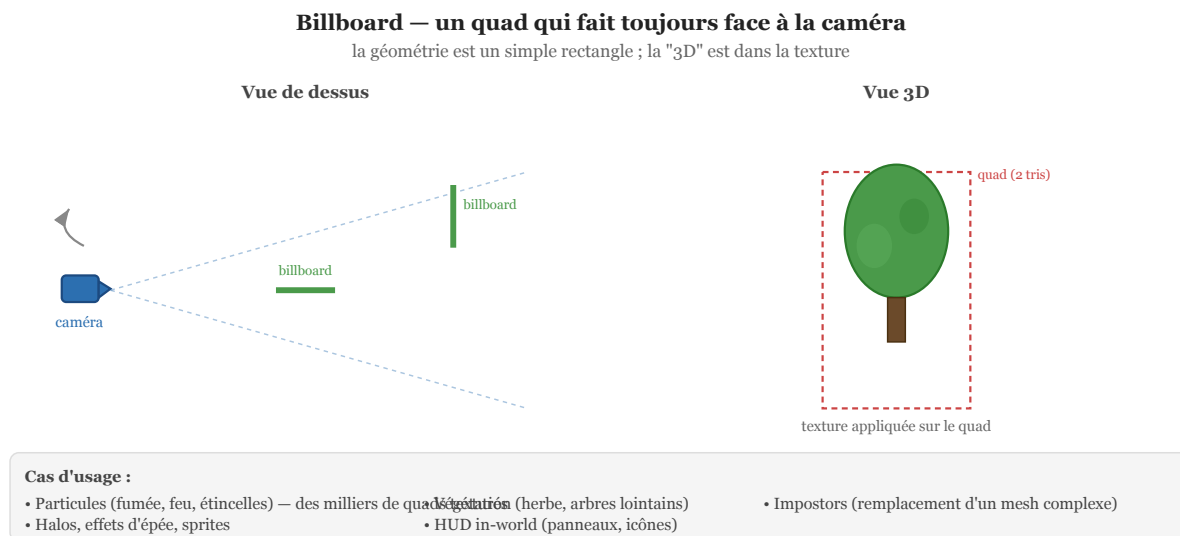


Fig. 51. — Un billboard. Vue de dessus (gauche) : le quad pivote pour rester perpendiculaire au rayon caméra. Vue 3D (droite) : la texture contient l'image de l'objet. Le résultat est un arbre qui « regarde » toujours la caméra — l'illusion tient tant qu'on ne tourne pas autour.

1. Modes de billboard

Spherical vs Cylindrical

- **Spherical** : le quad fait face à la caméra dans **tous les axes**. Utilisé pour les particules, les sprites, les halos. Problème : un arbre billboard spherical se « penche » quand on regarde vers le bas — il ne reste pas vertical.
- **Cylindrical** : le quad fait face à la caméra **autour de l'axe Y seulement**. Il reste vertical, ne fait que pivoter sur le plan horizontal. Utilisé pour les arbres, l'herbe, les objets posés au sol.

spherical : rotation(X, Y, Z) (caméra)

cylindrical : rotation(Y) (caméra) (vertical)

2. Cas d'usage

Où on les voit

- **Particules** : fumée, feu, étincelles, poussière. Des milliers de billboards animés. C'est 90% des effets visuels dans un jeu.
- **Végétation** : herbe sur un terrain (50 000 brins), arbres lointains. On ne modélise pas chaque brin — on pose des billboards.
- **Impostors** : un mesh complexe lointain remplacé par un billboard texturé avec une image du mesh (Partie 5).
- **Sprites et HUD in-world** : panneaux, icônes, marqueurs d'interaction qui doivent toujours être visibles.

3. Implémentation en Three.js

Billboard basique.

```
// Un plan avec une texture d'arbre
const texture = new THREE.TextureLoader().load('tree.png');
const material = new THREE.MeshBasicMaterial({
    map: texture,
    transparent: true,           // alpha pour les bords de l'arbre
    alphaTest: 0.5,             // rejeter les pixels transparents (pas de tri de profondeur)
    side: THREE.DoubleSide
});
const billboard = new THREE.Mesh(
    new THREE.PlaneGeometry(1, 1),
    material
);

// Faire face à la caméra (spherical) :
// Dans la boucle de rendu :
billboard.quaternion.copy(camera.quaternion);

// Ou en cylindrique (Y seulement) :
billboard.lookAt(camera.position.x, billboard.position.y, camera.position.z);
```

💡 Alpha test vs alpha blend

Les billboards utilisent des textures avec transparence (PNG avec alpha). Deux options :

- **Alpha blend** (transparent: true) : mélange la couleur avec le fond. Problème : nécessite un tri par distance (les objets transparents ne s'écrivent pas dans le depth buffer).
- **Alpha test** (alphaTest: 0.5) : rejette les pixels dont l'alpha est en dessous du seuil. Pas de mélange, écrit dans le depth buffer, pas besoin de tri. Préféré pour l'herbe et les feuilles — le bord est un peu dur, mais c'est gratuit.

4. Des milliers de billboards — Instancing

💬 Le problème des draw calls

Un billboard = 1 draw call. 50 000 brins d'herbe = 50 000 draw calls = catastrophe. Le CPU ne peut pas soumettre 50 000 commandes par frame.

Solution : **instancing**. On envoie la géométrie du quad **une seule fois** au GPU, et on fournit un tableau de **matrices de transformation** (une par instance). Le GPU duplique le quad en interne. 50 000 instances = 1 draw call.

Thin instances en Three.js.

```
const geometry = new THREE.PlaneGeometry(0.2, 0.5);
const material = new THREE.MeshBasicMaterial({
  map: grassTexture,
  alphaTest: 0.5,
  side: THREE.DoubleSide
});
const grass = new THREE.InstancedMesh(geometry, material, 50000);

// Positionner chaque brin d'herbe aléatoirement
const matrix = new THREE.Matrix4();
for (let i = 0; i < 50000; i++) {
  const x = (Math.random() - 0.5) * 100;
  const z = (Math.random() - 0.5) * 100;
  const y = 0;
  matrix.makeTranslation(x, y, z);
  grass.setMatrixAt(i, matrix);
}
grass.instanceMatrix.needsUpdate = true;
scene.add(grass); // 1 draw call pour les 50 000
```

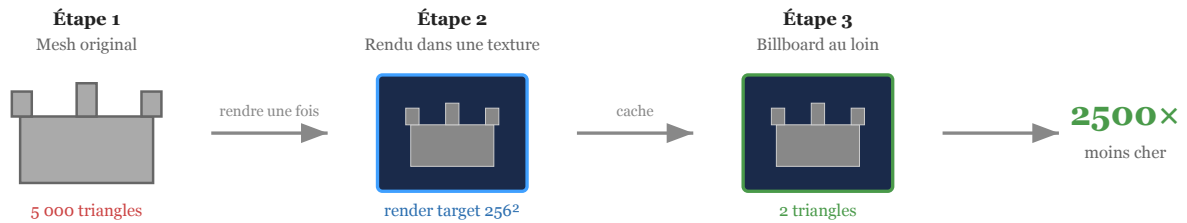
Partie 5 : Impostors — le LOD Extrême

L'impostor

Un **impostor** est la combinaison de LOD et billboard : on rend un mesh complexe **une fois** dans une texture (render target, Session 8), puis on affiche cette texture sur un billboard. Au lieu de dessiner 5 000 triangles chaque frame, on dessine 2 triangles — le quad avec l'image.

Impostor — le LOD extrême : un quad texturé au lieu d'un mesh

on rend le mesh une fois dans une texture, puis on affiche la texture sur un billboard



L'impostor est mis à jour périodiquement (ou par angle) quand l'objet se rapproche. Au-delà d'une certaine distance, il est visuellement identique au mesh.

Fig. 52. — Pipeline d'impostor. Le mesh original (5 000 tris) est rendu une fois dans une texture 256². Au loin, on affiche la texture sur un billboard (2 tris). Résultat : 2 500× moins de triangles, visuellement identique à distance.

💡 Quand l'impostor se met à jour

L'impostor est une **vue figée** du mesh. Si la caméra tourne autour, l'image ne correspond plus.

Solutions :

- **Impostor statique** : une seule vue, acceptable au très loin (l'objet est petit, l'angle change peu).
- **Impostor par angle** : pré-rendre plusieurs vues (8 ou 16 angles) et choisir la plus proche. Coût mémoire, mais qualité.
- **Mise à jour dynamique** : re-rendre l'impostor quand l'angle change trop. Coût GPU ponctuel, mais qualité maximale.

Dans la pratique, l'impostor est le dernier LOD avant de disparaître — au-delà d'une certaine distance, l'objet est trop petit pour qu'on voie la différence.

Partie 6 : Implémentation en Three.js

1. THREE.LOD — sélection automatique

Charger et utiliser les LOD du requin.

```
import { GLTFLoader } from 'three/addons/loaders/GLTFLoader.js';

const loader = new GLTFLoader();
loader.load('shark_lod.glb', (gltf) => {
    // Le GLB contient 4 meshes : LOD0, LOD1, LOD2, LOD3
    const lod = new THREE.LOD();

    // Trouver chaque mesh par nom et l'ajouter avec sa distance
    gltf.scene.traverse((child) => {
        if (child.isMesh) {
            if (child.name.includes('LOD0')) lod.addLevel(child, 0); // < 15m
            if (child.name.includes('LOD1')) lod.addLevel(child, 15); // 15-30m
            if (child.name.includes('LOD2')) lod.addLevel(child, 30); // 30-60m
            if (child.name.includes('LOD3')) lod.addLevel(child, 60); // > 60m
        }
    });
});
```

```

        }
    });

    scene.add(lod);
});

// Dans la boucle de rendu :
lod.update(camera); // Three.js sélectionne le bon mesh
    
```

2. Profiler — compter les draw calls

! Le réflexe à avoir

Avant d'optimiser, il faut **mesurer**. Three.js fournit `renderer.info` qui compte les draw calls, triangles, et textures :

```

console.log(renderer.info);
// { calls: 142, triangles: 285000, textures: 12, geometries: 8 }
    
```

Si `calls` est dans les milliers, c'est un problème de draw calls → instancing. Si `triangles` est dans les millions, c'est un problème de LOD → ajouter des niveaux.

3. La scène complète

Architecture typique

Une scène optimisée combine toutes les techniques :

1. **Frustum culling** (automatique) — élimine ce qui est hors champ.
2. **LOD** sur les objets complexes — moins de triangles au loin.
3. **Instancing** pour les objets répétés (herbe, arbres, rochers) — 1 draw call pour des milliers.
4. **Billboards** pour les particules et effets — les moins chers.
5. **Impostors** pour les objets très lointains — 2 triangles au lieu de milliers.

Partie 7 : Démonstration en Direct

! Démo interactive

On part d'une scène avec un terrain et on y ajoute :

1. Le requin avec 4 niveaux de LOD — on voit le triangle count changer dans `renderer.info` quand on zoome et dézoome.
2. Un champ d'herbe en instanced billboards (10 000 brins) — 1 draw call.
3. Un système de particules (billboards sphériques) — fumée ou étincelles.
4. Un compteur de FPS et de draw calls à l'écran.

On désactive le frustum culling et on voit les draw calls exploser. On active l'instancing et on voit les draw calls chuter.

Partie 8 : Pratique — Scène Optimisée

Cahier des charges. Construire une scène avec :

1. Le requin LOD (`shark_lod.glb`) — un `THREE.LOD` qui bascule entre les 4 niveaux.

2. Un terrain simple (plane subdivisée) avec au moins 1 000 brins d'herbe en instanced billboards.
3. Un système de particules (au choix : fumée, feu, étincelles) en billboards.
4. Un compteur de FPS et de draw calls (`renderer.info`) affiché à l'écran.
5. Un contrôle GUI pour ajuster les distances de transition LOD en temps réel.

Étapes — Three.js

1. Charger `shark_lod.glb` avec `GLTFLoader`, construire un `THREE.LOD` avec les 4 meshes.
2. Créer un terrain (plane 100×100, subdivisée).
3. Créer un `InstancedMesh` de quads texturés pour l'herbe, positionner aléatoirement.
4. Créer un système de particules (tableau de billboards ou `Points`).
5. Afficher `renderer.info.calls` et `renderer.info.triangles` dans un HUD.
6. Ajouter `lil-gui` pour ajuster les distances LOD.

Préparation de l'asset LOD dans Blender

💡 Comment on a préparé le requin

L'asset `shark_lod.glb` a été préparé dans Blender à partir d'un mesh retopologisé (`Shark_low`, 1 094 triangles de base) :

1. Le mesh original a un modifier **Mirror** (une moitié modelée, l'autre est le reflet). On applique le Mirror pour obtenir le mesh complet (2 188 triangles).
2. On duplique le mesh 3 fois → 4 copies.
3. Sur chaque copie, on ajoute un modifier **Decimate** avec un **ratio** de 0.5, 0.25 et 0.1.
4. On applique les modifieurs.
5. On exporte en GLB avec les 4 meshes nommés LOD0 à LOD3, avec la texture diffuse (2048²) et la normal map intégrées.

Le modifier **Decimate** en mode **Collapse** fusionne les vertices proches en préservant la silhouette. Le ratio n'est pas exact (1 489 au lieu de 1 094 pour 50%) car le mode Collapse préserve les bords et les contraintes topologiques. Le mode **Un-Subdivide** est une alternative qui supprime des boucles d'edges — meilleur pour les meshes avec une topologie régulière.

Critères de réussite

1. Le LOD bascule visiblement quand on s'éloigne (visible dans le compteur de triangles).
2. L'herbe s'affiche sans chute de FPS (1 draw call pour des milliers d'instances).
3. Le compteur de draw calls est visible et raisonnable (< 200).
4. Les distances LOD sont ajustables via la GUI.
5. Le pop-in est acceptable (pas de changement brusque évident à distance moyenne).

Conclusion

💬 Ce qu'on a vu

- **Culling** : frustum culling (hors champ), backface culling (dos des triangles), occlusion culling (caché derrière). Le moteur fait le frustum culling automatiquement.
- **Structures spatiales** : octree, BVH, quadtree organisent les objets pour culer par groupes. Même structure qu'en physique (broadphase), requête différente.

- **LOD** : remplacer la géométrie par une version simplifiée selon la distance. Créé par décimation dans Blender, géré par THREE.LOD dans Three.js.
- **Billboards** : un quad qui fait face à la caméra. La 3D est dans la texture. Particules, herbe, arbres lointains.
- **Instancing** : 1 draw call pour des milliers d'objets identiques. Indispensable pour l'herbe et les particules.
- **Impostors** : le LOD extrême — un mesh rendu dans une texture, affiché sur un billboard.

L'optimisation est ce qui sépare une démo d'un jeu. Une démo montre une technique ; un jeu en combine des dizaines, à 60 FPS, sur des scènes immenses. Les techniques d'aujourd'hui sont les fondations de toute scène de jeu réaliste.

💡 Récapitulatif du cours

En 12 sessions, on a parcouru tout le pipeline de rendu temps réel :

1. **Géométrie et transformations** (1-2) — comment on construit et place des objets 3D.
2. **Shaders** (3-4) — comment on contrôle le rendu au niveau du GPU.
3. **Éclairage** (5-6) — comment on simule la lumière, du direct au baked.
4. **Textures** (7-9) — comment on habille les surfaces, du simple au procédural.
5. **Ombres** (10) — comment on projette la lumière en temps réel.
6. **Post-processing** (11) — comment on traite l'image finale.
7. **Optimisation** (12) — comment on garde 60 FPS quand la scène grandit.

Vous avez maintenant les outils pour comprendre ce qui se passe dans n'importe quel moteur de jeu, et pour construire vos propres scènes optimisées.